

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC THÀNH PHỐ HỒ CHÍ
MINH

KHOA CÔNG NGHỆ THÔNG TIN

୨୨୪୪

KHÓA LUẬN TỐT NGHIỆP

**Nghiên cứu và thiết kế hệ thống
ngoại vi tốc độ cao dựa trên kỹ thuật
Double Data Rate (DDR) tích hợp
trên SoC RISC-V.**

SV thực hiện	Ngô Nguyễn Gia Huy
MSSV	23DH114340
GVHD	ThS. Hồ Lê Minh Toàn

TP. Hồ Chí Minh, tháng ... năm ...

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**Nghiên cứu và thiết kế hệ thống
ngoại vi tốc độ cao dựa trên kỹ thuật
Double Data Rate (DDR) tích hợp
trên SoC RISC-V.**

GIẢNG VIÊN HƯỚNG DẪN: ThS. Hồ Lê Minh Toàn

SINH VIÊN THỰC HIỆN: Ngô Nguyễn Gia Huy

TP. HỒ CHÍ MINH – .../...

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện luận văn tốt nghiệp, em đã nhận được nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ thầy cô, gia đình và bạn bè. Đây là nguồn động viên to lớn giúp em hoàn thành luận văn này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ths Hồ Lê Minh Toàn, người đã tận tình định hướng, góp ý chuyên môn và đồng hành cùng em trong suốt quá trình nghiên cứu. Những nhận xét và chỉ dẫn quý báu của thầy/cô đã giúp em từng bước hoàn thiện nội dung cũng như phương pháp thực hiện luận văn.

Em xin chân thành cảm ơn quý thầy cô trong khoa và nhà trường đã truyền đạt những kiến thức, kỹ năng cần thiết trong suốt thời gian học tập. Những kiến thức đó không chỉ là nền tảng để em thực hiện đề tài mà còn là hành trang quan trọng cho công việc và quá trình học tập sau này.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và tạo điều kiện thuận lợi để em có thể tập trung hoàn thành luận văn.

Mặc dù đã cố gắng nghiêm túc trong quá trình thực hiện, luận văn khó tránh khỏi những thiếu sót. Em kính mong nhận được sự góp ý của quý thầy cô và các bạn để nội dung luận văn được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp này là kết quả nghiên cứu do cá nhân tôi thực hiện dưới sự hướng dẫn của giảng viên. Toàn bộ nội dung, kết quả và số liệu được trình bày trong khóa luận đều trung thực, có nguồn gốc rõ ràng và chưa từng được sử dụng trong bất kỳ công trình nào nhằm mục đích bảo vệ một học vị khác.

Mọi kiến thức, nhận định, số liệu và kết quả nghiên cứu của các tác giả khác được sử dụng trong khóa luận đều được trích dẫn và ghi nguồn đầy đủ theo quy định. Tôi không sao chép, giả mạo hoặc sử dụng kết quả của người khác như kết quả nghiên cứu của bản thân.

Tôi xin hoàn toàn chịu trách nhiệm trước Nhà trường và pháp luật về tính trung thực, tính chính xác và toàn bộ nội dung của lời cam đoan này.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

TÓM TẮT KHÓA LUẬN

Các nền tảng hệ thống trên chip (System-on-Chip) SoC phụ thuộc nhiều vào các giao tiếp truyền thông nối tiếp như UART, SPI và I2C để tích hợp các thiết bị ngoại vi. Tuy nhiên, phương thức truyền dữ liệu đơn tốc độ truyền thông làm hạn chế thông lượng dữ liệu, trong khi việc tăng tần số xung clock thường làm gia tăng công suất tiêu thụ và gây khó khăn cho quá trình đáp ứng các ràng buộc thời gian.

Khóa luận này trình bày quá trình thiết kế và hiện thực một kiến trúc truyền thông SoC dạng mô-đun dựa trên RISC-V, có tích hợp cơ chế truyền dữ liệu tốc độ kép (Double Data Rate) DDR. Bằng cách khai thác đồng thời cạnh lên và cạnh xuống của tín hiệu clock, hệ thống được đề xuất nâng cao băng thông truyền thông mà không yêu cầu tăng tần số clock của hệ thống. Kiến trúc tích hợp các bộ điều khiển UART, SPI và I2C tùy chỉnh, cùng hoạt động trên một bus nội bộ dùng chung.

Kết quả đánh giá xác nhận hiệu quả của việc tích hợp cơ chế truyền thông theo định hướng DDR vào nền tảng RISC-V dạng mô-đun. Giải pháp cung cấp một nền tảng có khả năng mở rộng và đạt hiệu quả cao cho các ứng dụng Internet vạn vật (IoT) và điện toán biên hiện đại.

MỤC LỤC

LỜI CẢM ƠN	I
LỜI CAM ĐOAN	II
TÓM TẮT KHÓA LUẬN	III
MỤC LỤC	IV
DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	VII
DANH MỤC CÁC BẢNG	XII
DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ	XIII
1 GIỚI THIỆU	2
1.1 Lý do chọn đề tài	2
1.2 Mục tiêu nghiên cứu	4
1.2.1 Mục tiêu tổng quát	4
1.2.2 Mục tiêu cụ thể	4
1.3 Đối tượng nghiên cứu	5
1.4 Phạm vi và giới hạn nghiên cứu	5
1.4.1 Phạm vi nghiên cứu	5
1.4.2 Giới hạn đề tài	6
2 CƠ SỞ LÝ THUYẾT	7
2.1 Hệ thống SoC trên nền tảng RISC-V	7
2.2 Lỗi xử lý RISC-V PicoRV32	9
2.3 Sự khác biệt giữa truyền dữ liệu SDR và DDR	10
2.4 Giao tiếp UART	12
2.5 Giao tiếp I2C	15
2.6 Giao tiếp SPI	17
2.7 Bus APB và kết nối giữa các mô-đun	19
2.8 Timing và giới hạn tần số hoạt động	20

2.9	Công cụ Vivado	21
2.10	Nền tảng FPGA Xilinx 7 Series và thiết bị mục tiêu	23
3	CÁC NGHIÊN CỨU LIÊN QUAN	25
3.1	Tổng quan các hướng nghiên cứu về SoC trên nền RISC-V	25
3.2	Các giải pháp tối ưu băng thông và giao tiếp ngoại vi trên FPGA	25
3.3	Nghiên cứu về kỹ thuật DDR và quản lý Timing ở tần số cao	26
3.4	Bảng so sánh tổng hợp và định hướng của đề tài	28
4	MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ NGHỊ	30
4.1	Tổng quan kiến trúc hệ thống đề xuất	30
4.2	Thiết kế chi tiết các khối chức năng	31
4.2.1	Lỗi xử lý PicoRV32 và bộ giải mã địa chỉ	31
4.2.2	Cầu nối bus picorv32_apb_bridge	31
4.2.3	Khối xử lý song song và truyền dữ liệu DDR	34
4.2.4	Các khối ngoại vi chuẩn giao tiếp nối tiếp (UART, SPI, I2C)	37
4.3	Phương pháp xử lý Timing và ràng buộc XDC trên Vivado	44
4.4	Quy trình đóng gói và mô phỏng kiểm chứng mức RTL	48
4.4.1	Đóng gói các khối ngoại vi và tích hợp hệ thống (SoC Packaging)	48
4.4.2	Xây dựng môi trường mô phỏng Testbench	48
4.4.3	Kịch bản mô phỏng và kết quả kiểm chứng	49
4.4.4	Đánh giá kết quả mô phỏng	50
5	KẾT QUẢ THỰC NGHIỆM	52
5.1	Kết quả thực hiện	52
5.2	Kết quả thử nghiệm	53
5.2.1	Môi trường và phương pháp thử nghiệm	53
5.2.2	Kiểm chứng bộ điều khiển UART (UART Controller)	53
5.2.3	Kiểm chứng bộ điều khiển SPI (SPI Controller)	57
5.2.4	Kiểm chứng bộ điều khiển I ² C (I2C Controller)	59
5.2.5	Kiểm chứng khối xử lý dữ liệu song song định hướng DDR (Parallel DDR Accelerator)	62
5.2.6	Kiểm chứng tích hợp hệ thống SoC (SoC Integration Testbench)	63
5.2.7	Kết quả tổng hợp và thực thi phần cứng (RTL Implementation)	68
5.3	Đánh giá kết quả	72

5.4	Thảo luận	72
5.5	Hạn chế	73
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	74
6.1	Kết luận	74
6.2	Những kết quả đạt được	74
6.3	Những hạn chế	75
6.4	Hướng phát triển trong tương lai	76
7	CÁC CÔNG TRÌNH NGHIÊN CỨU	77
7.1	Bài báo đã gửi đăng	77
7.2	Mối liên hệ với khóa luận	77
	TÀI LIỆU THAM KHẢO	78

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
1	APB	Advanced Peripheral Bus – bus ngoại vi nâng cao
2	ASIC	Application-Specific Integrated Circuit – vi mạch tích hợp chuyên dụng
3	BRAM	Block Random Access Memory – bộ nhớ RAM khối trên FPGA
4	CAD	Computer-Aided Design – thiết kế có sự hỗ trợ của máy tính
5	CDC	Clock Domain Crossing – chuyển miền xung clock
6	DDR	Double Data Rate – kỹ thuật truyền dữ liệu trên cả hai cạnh xung clock
7	DRC	Design Rule Check – kiểm tra quy tắc thiết kế
8	EDA	Electronic Design Automation – tự động hóa thiết kế điện tử
9	EMI	Electromagnetic Interference – nhiễu điện từ
10	FPGA	Field-Programmable Gate Array – mảng cổng logic có thể lập trình
11	HDL	Hardware Description Language – ngôn ngữ mô tả phần cứng
12	IDDR	Input Double Data Rate – khối nhận dữ liệu DDR đầu vào
13	I/O	Input/Output – đầu vào/đầu ra
14	IP Core	Intellectual Property Core – lõi sở hữu trí tuệ phần cứng
15	ODDR	Output Double Data Rate – khối phát dữ liệu DDR đầu ra
16	RISC-V	Reduced Instruction Set Computer Five – kiến trúc tập lệnh RISC-V
17	RTL	Register-Transfer Level – mức truyền thanh ghi
18	SDC	Synopsys Design Constraints – định dạng ràng buộc thiết kế Synopsys
19	SDR	Single Data Rate – kỹ thuật truyền dữ liệu trên một cạnh xung clock
20	SoC	System-on-Chip – hệ thống trên chip
21	STA	Static Timing Analysis – phân tích timing tĩnh
22	XDC	Xilinx Design Constraints – định dạng ràng buộc thiết kế Xilinx
23	f	Tần số hoạt động của xung clock

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
24	$P_{dynamic}$	Công suất tiêu thụ động của hệ thống
25	t_H	Thời gian giữ dữ liệu (Hold time)
26	t_{SU}	Thời gian thiết lập dữ liệu (Setup time)
27	CPU	Central Processing Unit – bộ xử lý trung tâm
28	MCU	Microcontroller Unit – bộ vi điều khiển
29	W	Độ rộng của bus dữ liệu
30	f_{clk}	Tần số của xung clock
31	R_{SDR}	Tốc độ truyền dữ liệu lý thuyết của phương thức SDR
32	R_{DDR}	Tốc độ truyền dữ liệu lý thuyết của phương thức DDR
33	T_{clk}	Chu kỳ của xung clock
34	t_{cq}	Độ trễ từ cạnh clock đến đầu ra của thanh ghi
35	t_{logic}	Độ trễ của khối logic tổ hợp
36	t_{setup}	Thời gian thiết lập dữ liệu của thanh ghi đích
37	t_{skew}	Độ lệch thời gian giữa các tín hiệu clock
38	ACK	Acknowledgement – tín hiệu xác nhận giao dịch thành công
39	AHB	Advanced High-performance Bus – bus hiệu năng cao của AMBA
40	ARM	Advanced RISC Machines – kiến trúc bộ xử lý ARM
41	AXI	Advanced eXtensible Interface – giao diện bus mở rộng hiệu năng cao
42	CAN	Controller Area Network – mạng truyền thông điều khiển
43	CS	Chip Select – tín hiệu lựa chọn thiết bị SPI
44	DMA	Direct Memory Access – cơ chế truy cập bộ nhớ trực tiếp
45	DSP	Digital Signal Processing – khối xử lý tín hiệu số
46	FF	Flip-Flop – phần tử lưu trữ một bit dữ liệu
47	GPIO	General-Purpose Input/Output – cổng vào/ra đa dụng
48	I ² C	Inter-Integrated Circuit – bus truyền thông nối tiếp hai dây
49	IOLOGIC	Input/Output Logic – tài nguyên logic tại khối vào/ra FPGA
50	LUT	Look-Up Table – bảng tra cứu dùng để hiện thực logic tổ hợp
51	MISO	Master In Slave Out – đường dữ liệu từ thiết bị tớ đến thiết bị chủ SPI
52	MOSI	Master Out Slave In – đường dữ liệu từ thiết bị chủ đến thiết bị tớ SPI
53	NACK	Not Acknowledgement – tín hiệu không xác nhận giao dịch
54	PADDR	APB Address – địa chỉ giao dịch trên bus APB
55	PENABLE	APB Enable – tín hiệu kích hoạt pha truy cập APB
56	PL	Programmable Logic – vùng logic lập trình được của Zynq

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
57	PRDATA	APB Read Data – dữ liệu đọc trả về trên bus APB
58	PREADY	APB Ready – tín hiệu báo hoàn tất giao dịch APB
59	PS	Processing System – hệ thống xử lý cứng của Zynq
60	PSEL	APB Select – tín hiệu lựa chọn ngoại vi APB
61	PWDATA	APB Write Data – dữ liệu ghi trên bus APB
62	SCL	Serial Clock Line – đường xung clock của I ² C
63	SCLK	Serial Clock – đường xung clock của SPI
64	SDA	Serial Data Line – đường dữ liệu hai chiều của I ² C
65	SPI	Serial Peripheral Interface – giao tiếp ngoại vi nối tiếp đồng bộ
66	TX/RX	Transmit/Receive – đường truyền và nhận dữ liệu
67	UART	Universal Asynchronous Receiver/Transmitter – bộ truyền nhận nối tiếp bất đồng bộ
68	USB	Universal Serial Bus – bus nối tiếp đa dụng
69	mem_ready	Tín hiệu báo CPU rằng giao dịch bộ nhớ đã hoàn tất
70	ADC	Analog-to-Digital Converter – bộ chuyển đổi tín hiệu tương tự sang số
71	AMBA	Advanced Microcontroller Bus Architecture – kiến trúc bus cho vi điều khiển
72	CPHA	Clock Phase – pha lấy mẫu của xung clock trong giao tiếp SPI
73	CPOL	Clock Polarity – cực tính của xung clock trong giao tiếp SPI
74	DAC	Digital-to-Analog Converter – bộ chuyển đổi tín hiệu số sang tương tự
75	DRAM	Dynamic Random Access Memory – bộ nhớ truy cập ngẫu nhiên động
76	EEPROM	Electrically Erasable Programmable Read-Only Memory – bộ nhớ chỉ đọc có thể xóa và lập trình bằng điện
77	FIFO	First In, First Out – cấu trúc bộ đệm vào trước, ra trước
78	PHY	Physical Layer – lớp hoặc khối giao tiếp vật lý
79	RAM	Random Access Memory – bộ nhớ truy cập ngẫu nhiên
80	RTOS	Real-Time Operating System – hệ điều hành thời gian thực
81	SDRAM	Synchronous Dynamic Random Access Memory – bộ nhớ DRAM đồng bộ
82	VHDL	VHSIC Hardware Description Language – ngôn ngữ mô tả phần cứng VHDL

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
83	T_{ck}	Clock Period – chu kỳ xung clock
84	T_{suav}	Data valid to clock transition – thời gian thiết lập khả dụng tại đầu ra của thiết bị phát
85	T_{hav}	Clock transition to data invalid – thời gian giữ khả dụng tại đầu ra của thiết bị phát
86	T_{tcu}	Total clock uncertainty – tổng độ bất định của xung clock tại đầu ra của thiết bị phát
87	$T_{suav} + T_{hav}$	Data valid window – cửa sổ thời gian dữ liệu hợp lệ tại đầu ra
88	T_{su}	Input setup time – thời gian thiết lập yêu cầu tại đầu vào của thiết bị nhận
89	T_h	Input hold time – thời gian giữ yêu cầu tại đầu vào của thiết bị nhận
90	$T_{su} + T_h$	Required data window at input – cửa sổ dữ liệu yêu cầu tại đầu vào

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
91	WNS	Worst Negative Slack – độ dư thời gian thiết lập xấu nhất
92	WHS	Worst Hold Slack – độ dư thời gian giữ xấu nhất
93	WPWS	Worst Pulse Width Slack – độ dư độ rộng xung xấu nhất
94	TNS	Total Negative Slack – tổng độ dư thời gian âm
95	PPA	Power, Performance, Area – công suất, hiệu năng và diện tích
96	IOB	Input/Output Block – khối tài nguyên vào/ra của FPGA
97	BUFG	Global Clock Buffer – bộ đệm xung clock toàn cục
98	FSM	Finite-State Machine – máy trạng thái hữu hạn
99	LSB	Least Significant Bit – bit có trọng số thấp nhất
100	MSB	Most Significant Bit – bit có trọng số cao nhất
101	MMCM	Mixed-Mode Clock Manager – khối quản lý xung clock hỗn hợp
102	PLL	Phase-Locked Loop – vòng khóa pha
103	PCLK	APB Clock – xung clock của bus APB
104	PRESETn	APB Active-Low Reset – tín hiệu reset tích cực mức thấp của APB
105	PWRITE	APB Write Control – tín hiệu xác định giao dịch ghi APB
106	RAMB36E1	Khối Block RAM 36 Kb chuyên dụng trên FPGA 7 Series
107	OLOGIC	Output Logic – tài nguyên logic ngõ ra tại IOB
108	RV32I	Tập lệnh RISC-V cơ bản 32 bit dành cho số nguyên
109	P_{static}	Công suất tĩnh do dòng rò của hệ thống
110	P_{total}	Tổng công suất tiêu thụ trên chip
111	AI	Artificial Intelligence – trí tuệ nhân tạo
112	TinyML	Tiny Machine Learning – học máy nhẹ trên thiết bị nhúng

DANH MỤC CÁC BẢNG

STT	BẢNG	TRANG
1	Bảng 2.1: So sánh khái quát giữa SDR và DDR	11
2	Bảng 2.2: Thông số cấu hình và tài nguyên của FPGA Artix-7 mục tiêu	24
3	Bảng 3.1: Tổng hợp so sánh phạm vi các nghiên cứu liên quan và định hướng đề tài	29
4	Bảng 4.1: Bản đồ phân bố không gian địa chỉ (Memory Map) của hệ thống SoC	32
5	Bảng 5.1: Tổng hợp số liệu thực nghiệm chi tiết từng phân khối (Unit Level)	68
6	Bảng 5.2: Tổng hợp sử dụng tài nguyên phần cứng thực tế trên FPGA Artix-7 (xc7a200tfbg676-2L)	69
7	Bảng 5.3: Kết quả phân tích định thời hệ thống tại 100 MHz	70
8	Bảng 5.4: Tổng hợp công suất tiêu thụ hệ thống SoC tại tần số 100 MHz	71

DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
1	Hình 2.1: Kiến trúc tổng quan của hệ thống SoC RISC-V	8
2	Hình 2.2: Sự khác biệt giữa phương thức truyền dữ liệu SDR và DDR	11
3	Hình 2.3: Kiến trúc và luồng dữ liệu của giao tiếp UART	13
4	Hình 2.4: Dạng sóng truyền nhận dữ liệu của giao tiếp UART	14
5	Hình 2.5: Kiến trúc và trình tự truyền dữ liệu của giao tiếp I ² C	15
6	Hình 2.6: Dạng sóng truyền dữ liệu của giao tiếp I ² C	16
7	Hình 2.7: Kiến trúc và luồng dữ liệu song công của giao tiếp SPI	17
8	Hình 2.8: Dạng sóng truyền dữ liệu của giao tiếp SPI	18
9	Hình 2.9: Kiến trúc kết nối giữa APB Master và APB Slave	19
10	Hình 2.10: Giao diện phần mềm Vivado Design Suite	22
11	Hình 2.11: Bo mạch Zynq XC7Z030 tham khảo thuộc nền tảng Xilinx 7 Series	23
12	Hình 3.1: Quan hệ định thời Edge-aligned và Center-aligned tại phía nhận	26
13	Hình 4.1: Sơ đồ khối tổng thể kiến trúc SoC RISC-V đề xuất	31
14	Hình 4.2: Sơ đồ RTL của khối giao tiếp APB UART	38
15	Hình 4.3: Sơ đồ RTL của khối giao tiếp APB SPI	40
16	Hình 4.4: Sơ đồ RTL của khối giao tiếp APB I ² C	42
17	Hình 4.5: Quan hệ định thời ngõ ra Edge-aligned và Center-aligned	45

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
18	Hình 5.1: Kiểm tra tốc độ baud và cấu trúc khung truyền UART	54
19	Hình 5.2: Truyền mẫu dữ liệu UART TX 0xA5	55
20	Hình 5.3: Kiểm tra thanh ghi trạng thái Ready/Busy của UART	56
21	Hình 5.4: Kiểm tra chức năng nhận dữ liệu UART RX 0x3C	57
22	Hình 5.5: Kiểm tra CPOL và truyền nhận song công SPI	58
23	Hình 5.6: Kiểm tra thao tác thanh ghi Chip Select của SPI	59
24	Hình 5.7: Kiểm tra thao tác đọc/ghi thanh ghi I ² C qua APB	60
25	Hình 5.8: Truyền byte và nhận phản hồi ACK của I ² C	61
26	Hình 5.9: Truyền byte và nhận phản hồi NACK của I ² C	61
27	Hình 5.10: Kiểm tra điều kiện STOP của giao tiếp I ² C	62
28	Hình 5.11: Kiểm tra khối xử lý song song và truyền kết quả DDR	63
29	Hình 5.12: Kiểm tra truy cập và giải mã địa chỉ APB của hệ thống SoC	64
30	Hình 5.13: Kiểm tra UART và SPI hoạt động đồng thời	65
31	Hình 5.14: Kiểm tra UART, SPI và I ² C hoạt động đồng thời	66
32	Hình 5.15: Kiểm tra các giao dịch APB đan xen giữa nhiều ngoại vi	67

Chương 1

GIỚI THIỆU

1.1 Lý do chọn đề tài

Kiến trúc hệ thống trên chip (System-on-Chip – SoC) được sử dụng rộng rãi trong các hệ thống nhúng, thiết bị Internet vạn vật (IoT) và nền tảng điện toán biên nhờ khả năng tích hợp bộ xử lý, bộ nhớ, hệ thống liên kết và các bộ điều khiển ngoại vi trên cùng một nền tảng phần cứng. Cấu trúc này giúp giảm kích thước hệ thống và tạo điều kiện thuận lợi cho việc mở rộng thiết kế. Trong số các kiến trúc bộ xử lý hiện nay, RISC-V là một lựa chọn phù hợp cho nghiên cứu SoC nhờ tính mở, cấu trúc mô-đun và khả năng hỗ trợ xây dựng các nền tảng phần cứng tùy chỉnh [1].

Một SoC thực tế thường cần nhiều giao tiếp truyền thông để kết nối với cảm biến, bộ nhớ, màn hình và các mô-đun bên ngoài. UART, SPI và I²C là những lựa chọn phổ biến vì có cấu trúc đơn giản và được nhiều thiết bị nhúng hỗ trợ. UART thường được sử dụng cho mục đích gỡ lỗi và trao đổi dữ liệu bất đồng bộ; SPI hỗ trợ truyền song công với tốc độ cao; trong khi I²C phù hợp khi nhiều thiết bị tốc độ thấp dùng chung một bus. Các nghiên cứu về thiết kế SoC bằng Verilog và triển khai trên FPGA cho thấy các khối truyền thông nối tiếp vẫn giữ vai trò quan trọng trong hệ thống phần cứng nhúng [2–4].

Mặc dù các giao tiếp này đã được tích hợp trong nhiều vi điều khiển thương mại, cấu trúc bên trong thường cố định nên khó điều chỉnh cho mục đích nghiên cứu, giảng dạy hoặc phát triển SoC tùy chỉnh. Vì vậy, một nền tảng có khả năng tái sử dụng và được mô tả ở mức RTL là cần thiết, đặc biệt khi thiết kế cần được kiểm chứng, tổng hợp và thử nghiệm trên FPGA trước khi hướng đến hiện thực ASIC. Các công trình về thiết kế ngoại vi dựa trên bus AMBA đã cho thấy tầm quan trọng của một cấu trúc bus rõ ràng khi kết nối bộ xử lý với các khối phần cứng tùy chỉnh [5]. Bên cạnh đó, kiến trúc liên kết AXI cũng đã được nghiên cứu để kết nối các ngoại vi tùy chỉnh với bộ xử lý RISC-V [6].

Một vấn đề quan trọng khác trong truyền thông SoC là thông lượng dữ liệu. Với phương thức truyền đơn tốc độ, dữ liệu chỉ được phát hoặc lấy mẫu tại một cạnh xung clock. Việc tăng tần số clock có thể cải thiện băng thông nhưng đồng thời làm cho quá trình đáp ứng ràng buộc thời gian và kiểm soát công suất trở nên khó khăn hơn. Kỹ thuật truyền dữ liệu tốc độ kép (Double Data Rate – DDR) là một giải pháp phù hợp hơn nhờ khai thác đồng thời cạnh lên và cạnh xuống của xung clock. Cơ chế này cho phép nâng cao tốc độ truyền dữ liệu mà không cần trực tiếp tăng gấp đôi tần số clock chính. Các kỹ thuật dựa trên DDR đã được ứng dụng trong bộ điều khiển bộ nhớ và các hệ thống truyền thông tốc độ cao trên chip [7]. Những nghiên cứu về liên kết mạng trên chip sử dụng DDR cũng cho thấy việc truyền dữ liệu trên hai cạnh clock có thể cải thiện thông lượng truyền thông nội bộ [8, 9].

Từ những vấn đề trên, khóa luận đề xuất một kiến trúc SoC dạng mô-đun dựa trên RISC-V, tích hợp các kênh truyền thông và cơ chế truyền dữ liệu DDR. Hệ thống gồm các bộ điều khiển UART, SPI và I²C dưới dạng các mô-đun RTL độc lập, được kết nối thông qua bus APB dùng chung. Cơ chế DDR hỗ trợ phát dữ liệu tại cả cạnh lên và cạnh xuống của xung clock nhằm nâng cao thông lượng. Thiết kế được mô tả bằng Verilog/SystemVerilog có khả năng tổng hợp, sau đó được kiểm chứng bằng mô phỏng và các báo cáo sau hiện thực trên FPGA. Quy trình này cho phép đánh giá chức năng, đặc tính định thời và chi phí tài nguyên trước khi xem xét mở rộng sang thiết kế ASIC [10].

Các đóng góp chính của khóa luận được tóm tắt như sau:

- Thiết kế kiến trúc SoC dạng mô-đun dựa trên RISC-V, tích hợp các bộ điều khiển truyền thông UART, SPI và I²C cho các ứng dụng truyền thông nhúng.
- Tích hợp cơ chế truyền dữ liệu theo kỹ thuật DDR nhằm cải thiện thông lượng bằng cách khai thác cả cạnh lên và cạnh xuống của xung clock.
- Xây dựng thiết kế RTL bằng Verilog/SystemVerilog có khả năng tổng hợp, phục vụ triển khai trên FPGA và tái sử dụng trong các thiết kế hướng ASIC sau này.
- Thực hiện mô phỏng chức năng và kiểm chứng sau hiện thực để xác nhận hoạt động của hệ thống đề xuất.
- Đánh giá thiết kế theo các tiêu chí về mức sử dụng tài nguyên, tần số hoạt động, đặc tính định thời và chức năng truyền thông.

Phần còn lại của khóa luận được tổ chức như sau: Chương 2 trình bày kiến thức tổng quan về các giao tiếp UART, SPI, I²C, hệ thống bus và kiến trúc SoC. Chương 3 tổng hợp các nghiên cứu liên quan. Chương 4 trình bày mô hình và phương pháp đề nghị. Chương 5 trình bày các kết quả thực nghiệm và đánh giá. Cuối cùng, Chương 6 đưa ra kết luận và đề xuất các hướng phát triển trong tương lai.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu và thiết kế hệ thống ngoại vi tốc độ cao dựa trên kỹ thuật Double Data Rate (DDR), tích hợp trên SoC RISC-V nhằm nâng cao thông lượng xử lý và truyền dữ liệu. Hệ thống được mô tả bằng Verilog ở mức RTL, bao gồm các mô-đun chức năng chính, mạch kết nối nội bộ và môi trường kiểm chứng hoạt động.

1.2.2 Mục tiêu cụ thể

Các mục tiêu cụ thể của đề tài gồm:

- Khảo sát và phân tích thiết kế hiện hữu: Nghiên cứu nguyên lý truyền nhận dữ liệu Double Data Rate (DDR); đánh giá kiến trúc các mô-đun ngoại vi Single Data Rate (SDR) hiện có để xác định phương án cải tiến và chuyển đổi sang cấu trúc DDR.
- Thiết kế mô-đun ngoại vi DDR và cơ chế kiểm soát lỗi: Xây dựng mô tả RTL cho khối ngoại vi DDR, đồng thời tích hợp cơ chế kiểm soát lỗi nhằm bảo đảm tính toàn vẹn dữ liệu khi truyền nhận ở tốc độ cao.
- Xử lý chuyển miền xung clock (CDC): Thiết kế mạch đồng bộ giữa miền clock của ngoại vi DDR và miền clock hệ thống nhằm hạn chế rủi ro bất ổn định trạng thái (Metastability).
- Tối ưu hóa pipeline và thiết kế cầu nối (Bridge): Áp dụng kỹ thuật pipelining để rút ngắn đường truyền tới hạn (Critical Path); thiết kế mô-đun cầu nối để tích hợp ngoại vi DDR vào bus nội bộ của SoC RISC-V.

- Đặt ràng buộc và phân tích định thời: Thiết lập các tệp ràng buộc thời gian SDC/XDC; phân tích và tối ưu các thông số setup, hold và slack nhằm nâng cao tần số hoạt động tối đa của hệ thống.
- Tổng hợp và đánh giá kết quả: Tổng hợp kết quả mô phỏng, hiện thực phần cứng, mức sử dụng tài nguyên và báo cáo định thời để đánh giá thiết kế theo các tiêu chí đã đề ra.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là khối ngoại vi tốc độ cao áp dụng kỹ thuật Double Data Rate (DDR), được mô tả ở mức RTL bằng ngôn ngữ Verilog. Nghiên cứu tập trung vào nguyên lý thiết kế và mô phỏng chức năng của khối ngoại vi; phương thức tích hợp vào bus APB của SoC; kỹ thuật xử lý chuyển miền xung clock (Clock Domain Crossing – CDC); cùng quy trình thiết lập và đáp ứng các ràng buộc thời gian. Đồng thời, đề tài đánh giá ảnh hưởng của độ lệch clock và độ lệch dữ liệu đến tính toàn vẹn dữ liệu khi hệ thống hoạt động ở tần số cao.

Các thành phần chính được nghiên cứu bao gồm lõi xử lý RISC-V, các mô-đun truyền nhận dữ liệu, mạch điều khiển clock, cơ chế phát dữ liệu ở hai cạnh xung, mạch đồng bộ tín hiệu, kết nối giữa các khối chức năng và các báo cáo định thời sau tổng hợp, hiện thực. Ngoài ra, đề tài cũng xem xét ảnh hưởng của việc tăng tần số clock đến độ ổn định, độ trễ đường truyền, khả năng đáp ứng định thời và tính đúng đắn của dữ liệu.

1.4 Phạm vi và giới hạn nghiên cứu

1.4.1 Phạm vi nghiên cứu

Đề tài tập trung vào quy trình thiết kế, tích hợp và kiểm chứng phần cứng ở mức Register-Transfer Level (RTL) cho hệ thống ngoại vi tốc độ cao sử dụng kỹ thuật Double Data Rate (DDR). Phạm vi nghiên cứu bao gồm:

- Thiết kế phần cứng RTL: Sử dụng ngôn ngữ Verilog để mô tả mô-đun ngoại vi DDR (chuyển đổi từ mô-đun SDR hiện có), tích hợp cơ chế kiểm soát lỗi, mạch xử lý chuyển miền xung clock (Clock Domain Crossing – CDC) bất đồng bộ,

cấu trúc pipeline và mô-đun cầu nối (Bridge) để ghép nối vào bus APB của SoC RISC-V.

- Mô phỏng chức năng (Functional Simulation): Xây dựng môi trường testbench để xác minh tính đúng đắn về logic, trình tự thời gian và khả năng kiểm soát lỗi trong các kịch bản truyền nhận dữ liệu DDR.
- Tổng hợp và hiện thực phần cứng (Synthesis & Place-and-Route): Tổng hợp thiết kế trên FPGA AMD/Xilinx Artix-7 xc7a200tfbg676-2L để đánh giá mức sử dụng tài nguyên phần cứng, bao gồm LUT, FF, BRAM và I/O.
- Tối ưu định thời: Thiết lập các ràng buộc thời gian SDC/XDC; phân tích setup slack, hold slack và clock/data skew ở các mức tần số khác nhau để xác định tần số hoạt động tối đa khả thi của hệ thống.

1.4.2 Giới hạn đề tài

Để bảo đảm tính khả thi và tập trung vào các bài toán kỹ thuật trọng tâm trong thời gian thực hiện khóa luận, đề tài xác định các giới hạn sau:

- Mô hình giao tiếp DDR: Khỏi ngoại vi DDR được thiết kế phục vụ truyền nhận dữ liệu hai cạnh xung cho giao diện ngoại vi/bus nội bộ, không tập trung xây dựng toàn bộ bộ điều khiển bộ nhớ DRAM thương mại phức tạp (như DDR3/DDR4/DDR5 SDRAM Controller với các quy trình Calibration/PHY riêng biệt).
- Môi trường kiểm chứng: Đề tài dừng lại ở bước kiểm chứng chức năng qua mô phỏng RTL và phân tích định thời, tài nguyên sau bước Place & Route trên AMD Vivado Design Suite 2025.2; chưa thực hiện đo đặc tính hiệu năng bằng máy hiện sóng hoặc bộ phân tích logic trên phần cứng thực tế.
- Hệ sinh thái phần mềm: Đề tài không xây dựng trình điều khiển, RTOS/Linux hoặc trình biên dịch cho SoC RISC-V.

Chương 2

CƠ SỞ LÝ THUYẾT

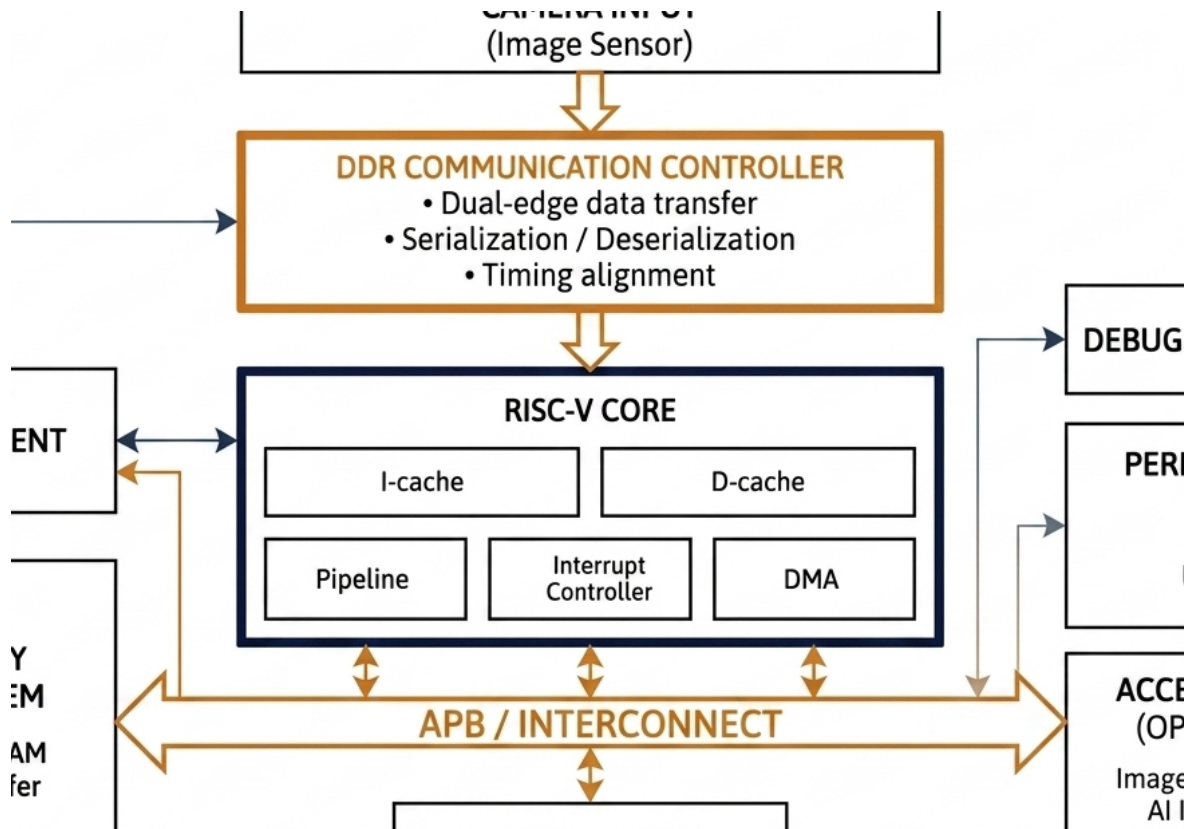
2.1 Hệ thống SoC trên nền tảng RISC-V

SoC trong đề tài được xây dựng theo hướng một hệ thống xử lý nhúng gọn nhẹ, trong đó lõi RISC-V đóng vai trò điều khiển trung tâm và các mô-đun ngoại vi được kết nối thông qua bus nội bộ. Thành phần xử lý chính là PicoRV32, một lõi RISC-V 32 bit có kích thước nhỏ, phù hợp để tích hợp trên FPGA và thuận tiện cho việc quan sát luồng tín hiệu ở mức RTL. Lõi này phát sinh các yêu cầu truy cập bộ nhớ thông qua giao diện *native memory*, gồm địa chỉ, dữ liệu ghi, dữ liệu đọc, tín hiệu hợp lệ, tín hiệu sẵn sàng và tín hiệu cho phép ghi theo byte [11, 12].

Hình 2.1 minh họa kiến trúc tổng quan và mối liên kết giữa các thành phần chính của hệ thống. Ở mức cao nhất, mô-đun soc_top kết nối PicoRV32 với bộ nhớ lệnh/dữ liệu nội bộ và các ngoại vi APB. Vùng địa chỉ thấp được dùng cho BRAM 4 KB, trong đó chương trình được nạp từ tập tin firmware.hex. Khi CPU cần lấy lệnh, yêu cầu được đưa trực tiếp đến BRAM để giảm độ phức tạp của đường truy cập lệnh. Khi CPU thực hiện đọc hoặc ghi dữ liệu đến vùng ngoại vi, yêu cầu từ giao diện native memory được chuyển sang giao thức APB thông qua mô-đun picorv32_apb_bridge [11, 13].

Khối bridge có nhiệm vụ biến một truy cập của CPU thành giao dịch APB gồm hai pha: pha thiết lập và pha kích hoạt. Trong pha thiết lập, địa chỉ, hướng truyền và dữ liệu ghi được giữ ổn định trên bus. Trong pha kích hoạt, hệ thống chờ tín hiệu PREADY từ mô-đun đích rồi mới trả mem_ready về cho CPU. Cách tổ chức này giúp lõi RISC-V không cần trực tiếp xử lý chi tiết của từng ngoại vi, đồng thời tạo ra một lớp kết nối thống nhất cho toàn bộ hệ thống [13].

Phần ngoại vi của SoC được chia theo vùng địa chỉ. Mô-đun soc_apb_top giải mã các vùng 0x4000_0000, 0x4000_1000 và 0x4000_2000 lần lượt cho UART, SPI và I2C. Ngoài ra, khối parallel được ánh xạ riêng tại vùng 0x4000_3000. Đây là



Hình 2.1: Kiến trúc tổng quan của hệ thống SoC RISC-V.

khối xử lý song song gồm bốn lane, mỗi lane nhận hai toán hạng 32 bit và tạo kết quả cộng tương ứng. Các thanh ghi A, B, thanh ghi điều khiển/trạng thái và thanh ghi kết quả đều được truy cập thông qua APB, nhờ đó phần mềm chạy trên PicoRV32 có thể ghi dữ liệu đầu vào, kích hoạt xử lý và đọc kết quả trả về [11, 13].

Như vậy, trọng tâm của SoC này không nằm ở việc xây dựng một MCU thương mại đầy đủ, mà là tạo ra một nền tảng RISC-V có đủ các thành phần cần thiết để kiểm chứng luồng truyền dữ liệu ở mức phần cứng. PicoRV32 đảm nhiệm vai trò phát lệnh và điều khiển, APB bridge bảo đảm việc chuyển giao dịch giữa CPU và ngoại vi, còn các khối UART, SPI, I2C và parallel cung cấp môi trường để đánh giá khả năng truyền nhận, xử lý song song và mở rộng băng thông của hệ thống. Kiến trúc này là nền tảng để các phần sau phân tích cơ chế truyền dữ liệu hai cạnh clock, timing và khả năng nâng tần số hoạt động [11, 12].

2.2 Lỗi xử lý RISC-V PicoRV32

RISC-V là kiến trúc tập lệnh mở, được tổ chức theo một tập lệnh cơ sở kết hợp với các phần mở rộng tùy chọn. Trong cấu hình RV32I, độ rộng thanh ghi nguyên *XLEN* bằng 32 bit; kiến trúc cung cấp 32 thanh ghi nguyên từ x0 đến x31 và một thanh ghi bộ đếm chương trình. Thanh ghi x0 luôn có giá trị bằng không, trong khi các thanh ghi còn lại được sử dụng để lưu toán hạng, địa chỉ và kết quả trung gian của chương trình [12].

PicoRV32 là một lõi RISC-V mã nguồn mở được mô tả bằng Verilog, hướng đến việc tối ưu diện tích logic và tần số hoạt động trên FPGA hoặc ASIC. Dự án PicoRV32 cung cấp ba biến thể chính: *picorv32* sử dụng giao diện bộ nhớ nguyên bản, *picorv32_axi* sử dụng giao diện AXI4-Lite và *picorv32_wb* sử dụng giao diện Wishbone. Trong khóa luận này, biến thể *picorv32* được lựa chọn vì giao diện bắt tay đơn giản, thuận lợi cho việc kết nối BRAM và xây dựng cầu nối sang bus APB [11].

Giao diện bộ nhớ nguyên bản của PicoRV32 hoạt động theo cơ chế *valid/ready* và xử lý một giao dịch tại mỗi thời điểm. Các tín hiệu chính bao gồm:

- *mem_valid*: báo hiệu CPU đang phát sinh một yêu cầu truy cập bộ nhớ;
- *mem_instr*: phân biệt chu kỳ lấy lệnh với chu kỳ truy cập dữ liệu;
- *mem_ready*: xác nhận bộ nhớ hoặc thiết bị đích đã hoàn tất giao dịch;
- *mem_addr*: mang địa chỉ truy cập 32 bit;
- *mem_wdata* và *mem_rdata*: lần lượt mang dữ liệu ghi và dữ liệu đọc 32 bit;
- *mem_wstrb*: gồm bốn bit cho phép ghi, tương ứng với bốn byte của một từ dữ liệu 32 bit.

Khi bắt đầu một giao dịch, PicoRV32 đưa *mem_valid* lên mức cao và giữ ổn định các tín hiệu địa chỉ, dữ liệu và điều khiển cho đến khi nhận được *mem_ready*. Đối với giao dịch đọc, *mem_wstrb* bằng không và dữ liệu được nhận qua *mem_rdata*. Đối với giao dịch ghi, một hoặc nhiều bit của *mem_wstrb* được kích hoạt để xác định các byte cần cập nhật trong từ dữ liệu. Cơ chế này cho phép ghép nối trực tiếp với bộ nhớ đơn giản hoặc chuyển đổi sang một giao thức bus khác thông qua mạch cầu nối [11].

PicoRV32 còn hỗ trợ nhiều tham số cấu hình nhằm cân bằng giữa diện tích và hiệu năng, chẳng hạn lựa chọn tập thanh ghi một cổng hoặc hai cổng đọc, bộ dịch barrel, hỗ trợ ngắt và giao diện đồng xử lý PCPI. Các phép nhân, chia có thể được bổ sung thông qua các khối PCPI tương ứng. Nhờ khả năng cấu hình này, lõi có thể được điều chỉnh cho các hệ thống nhúng nhỏ hoặc mở rộng khi ứng dụng cần thêm chức năng xử lý [11].

Trong SoC của khóa luận, PicoRV32 thực hiện lấy lệnh từ BRAM nội bộ và truy cập các ngoại vi thông qua mô-đun `picorv32_apb_bridge`. Cầu nối tiếp nhận yêu cầu từ giao diện bộ nhớ nguyên bản, chuyển đổi thành chu kỳ APB và chỉ đưa `mem_ready` lên mức cao sau khi ngoại vi hoàn tất giao dịch. Vì vậy, PicoRV32 đóng vai trò bộ điều khiển trung tâm, trong khi bus APB cung cấp không gian địa chỉ thống nhất cho UART, SPI, I²C và khối xử lý song song DDR.

2.3 Sự khác biệt giữa truyền dữ liệu SDR và DDR

Ở phương thức SDR (Single Data Rate), dữ liệu thường được phát hoặc lấy mẫu tại một cạnh cố định của xung clock, chẳng hạn cạnh lên hoặc cạnh xuống. Với tần số clock f_{clk} và bus dữ liệu có độ rộng W , tốc độ truyền lý thuyết có thể biểu diễn gần đúng bởi [7, 14]:

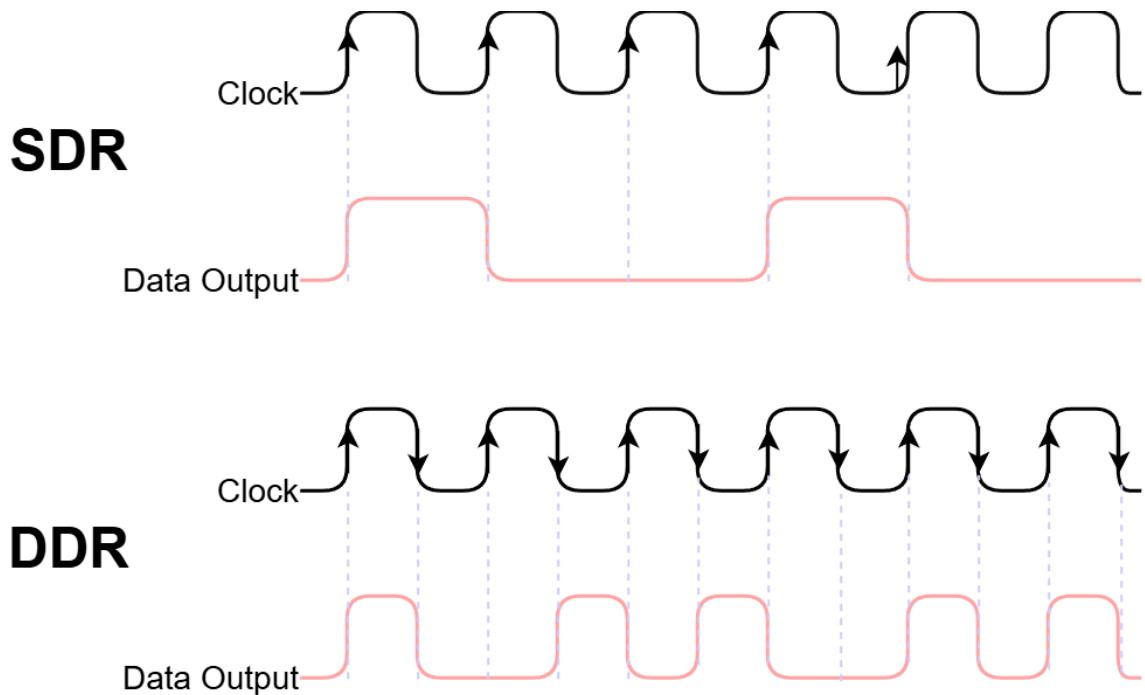
$$R_{SDR} = W \times f_{clk}. \quad (2.1)$$

Trong khi đó, DDR (Double Data Rate) sử dụng cả cạnh lên và cạnh xuống của xung clock để truyền dữ liệu. Như vậy, trong một chu kỳ clock có thể truyền hai lần dữ liệu. Tốc độ truyền lý thuyết khi đó được xác định bởi [9, 14]:

$$R_{DDR} = 2 \times W \times f_{clk}. \quad (2.2)$$

Việc khai thác hai cạnh clock cũng được áp dụng trong các giao diện nối tiếp tốc độ cao, chẳng hạn chế độ truyền DDR trên các bộ điều khiển nhớ ngoài của vi điều khiển NXP [15].

Theo đó, DDR có thể tăng gấp đôi số lần truyền trong cùng một khoảng thời gian mà không nhất thiết phải tăng độ rộng bus. Đây là điểm trọng tâm của đề tài. Tuy nhiên, việc sử dụng hai cạnh clock cũng làm yêu cầu về thời điểm phát dữ liệu, thời điểm lấy mẫu, độ lệch clock và thời gian ổn định của tín hiệu trở nên nghiêm ngặt hơn [8, 9, 16].



Hình 2.2: Minh họa sự khác biệt giữa phương thức truyền dữ liệu SDR và DDR.

Hình 2.2 cho thấy SDR chỉ thay đổi dữ liệu theo một cạnh clock, trong khi DDR khai thác cả cạnh lên và cạnh xuống để tăng gấp đôi số lần truyền trong mỗi chu kỳ so với SDR [14].

Bảng 2.1: So sánh khái quát giữa SDR và DDR

Tiêu chí	SDR	DDR
Số lần truyền trong một chu kỳ	Một lần	Hai lần
Điểm phát hoặc lấy mẫu	Một cạnh clock	Cả cạnh lên và cạnh xuống
Băng thông lý thuyết	Thấp hơn ở cùng tần số	Cao hơn ở cùng tần số
Yêu cầu timing	Tương đối đơn giản	Chặt chẽ hơn
Độ phức tạp thiết kế	Thấp hơn	Cao hơn do phải quản lý hai cạnh clock

Sự khác biệt trực quan về mật độ dữ liệu giữa hai phương thức được thể hiện trên Hình 2.2. Việc chuyển sang sử dụng cả hai cạnh xung nhịp mang lại cho kỹ thuật DDR những thế mạnh cốt lõi so với phương thức SDR [8, 9]:

Thứ nhất, DDR giúp tăng gấp đôi băng thông truyền tải mà không cần nâng cao tần số xung nhịp hệ thống. Để đạt cùng một hiệu năng xử lý, phương thức SDR phải tăng tần số f_{clk} , kéo theo nguy cơ gia tăng nhiễu điện từ (EMI), làm phức tạp mạng phân phối clock (*Clock Tree*) và dễ vi phạm ràng buộc timing. DDR giải quyết vấn đề này bằng cách nhân đôi lưu lượng dữ liệu trên một chu kỳ clock [9, 16].

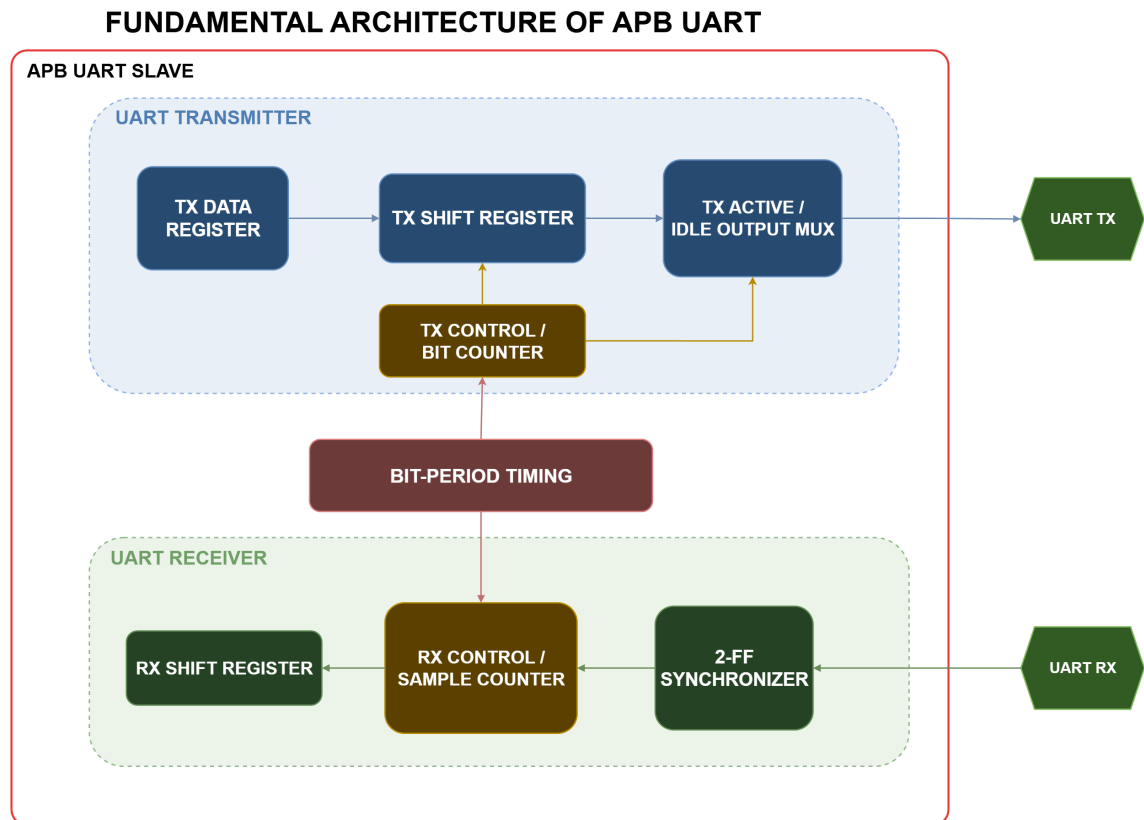
Thứ hai, DDR hỗ trợ tiết kiệm tài nguyên chân I/O khi cần tăng băng thông. Với SDR, một phương án tăng băng thông mà không tăng tần số là mở rộng độ rộng bus dữ liệu, làm tăng số lượng đường tín hiệu. Kỹ thuật DDR cho phép giữ nguyên số đường dữ liệu vật lý nhưng tăng số lần truyền trong mỗi chu kỳ [7, 14].

Thứ ba, DDR có thể cải thiện hiệu suất năng lượng khi đạt cùng mức băng thông ở tần số clock thấp hơn. Trong mạch số, công suất động phụ thuộc vào tần số chuyển mạch; vì vậy việc hạn chế tăng clock có thể giảm áp lực công suất trên mạng xung nhịp [16].

Mặc dù sở hữu nhiều ưu điểm về mặt hiệu năng và tài nguyên, phương thức DDR cũng đặt ra thách thức lớn hơn về quản lý timing. Do thời gian dành cho một lần truyền bị thu hẹp xuống còn $T_{clk}/2$, các thông số như thời gian thiết lập (t_{setup}), thời gian giữ (t_{hold}) và độ lệch clock (*clock skew*) phải được kiểm soát chặt chẽ. Mô hình DDR trong đề tài tập trung vào việc thiết kế và tối ưu khối giao tiếp RTL hai cạnh clock nhằm chứng minh tính hiệu quả trên nền tảng RISC-V SoC [9, 16].

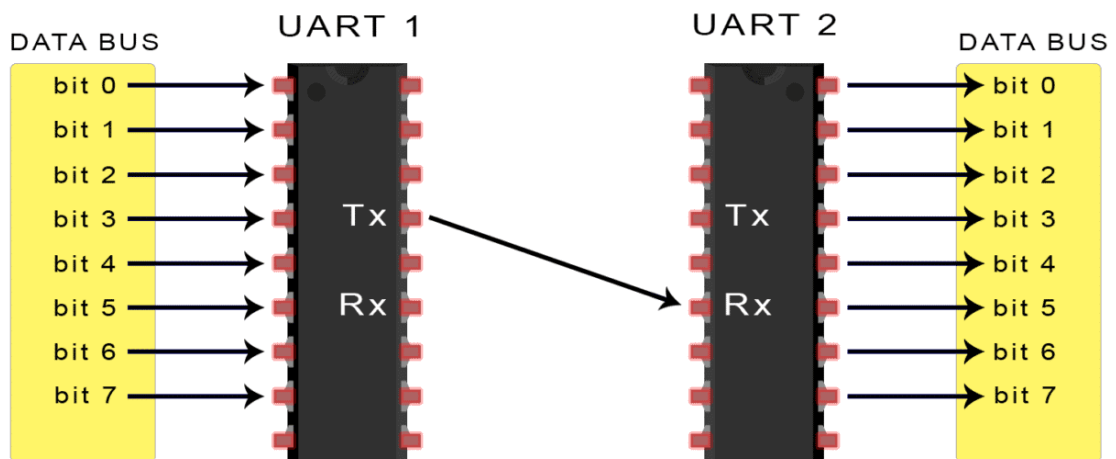
2.4 Giao tiếp UART

UART (Universal Asynchronous Receiver/Transmitter) là chuẩn giao tiếp nối tiếp bất đồng bộ, được sử dụng rộng rãi trong các hệ thống nhúng để trao đổi dữ liệu giữa vi điều khiển, máy tính, module ngoại vi hoặc mạch kiểm thử. Đặc điểm quan trọng của UART là hai thiết bị không dùng chung một đường clock truyền kèm dữ liệu. Thay vào đó, bên truyền và bên nhận phải thống nhất trước các tham số như tốc độ baud, số bit dữ liệu, bit kiểm tra chẵn lẻ nếu có và số bit dừng [2, 17, 18].



Hình 2.3: Kiến trúc và luồng dữ liệu của giao tiếp UART [18].

Hình 2.3 trình bày các thành phần chính của bộ điều khiển UART, bao gồm khối thanh ghi dùng để lưu dữ liệu truyền, dữ liệu nhận, cấu hình và trạng thái hoạt động. Khối chia xung clock xác định chu kỳ của từng bit. Máy trạng thái điều khiển trình tự phát và thu, trong khi thanh ghi dịch truyền bổ sung bit bắt đầu và bit dừng rồi đưa dữ liệu tuần tự ra đường TX. Theo chiều ngược lại, khối lấy mẫu giám sát đường RX, nhận lần lượt các bit và ghép chúng thành dữ liệu song song để CPU đọc qua bus APB [3,4].



Hình 2.4: Dạng sóng truyền nhận dữ liệu của giao tiếp UART [18].

Hình 2.4 minh họa cấu trúc thời gian của một khung UART, gồm trạng thái rỗi, bit bắt đầu, các bit dữ liệu và bit dừng [17].

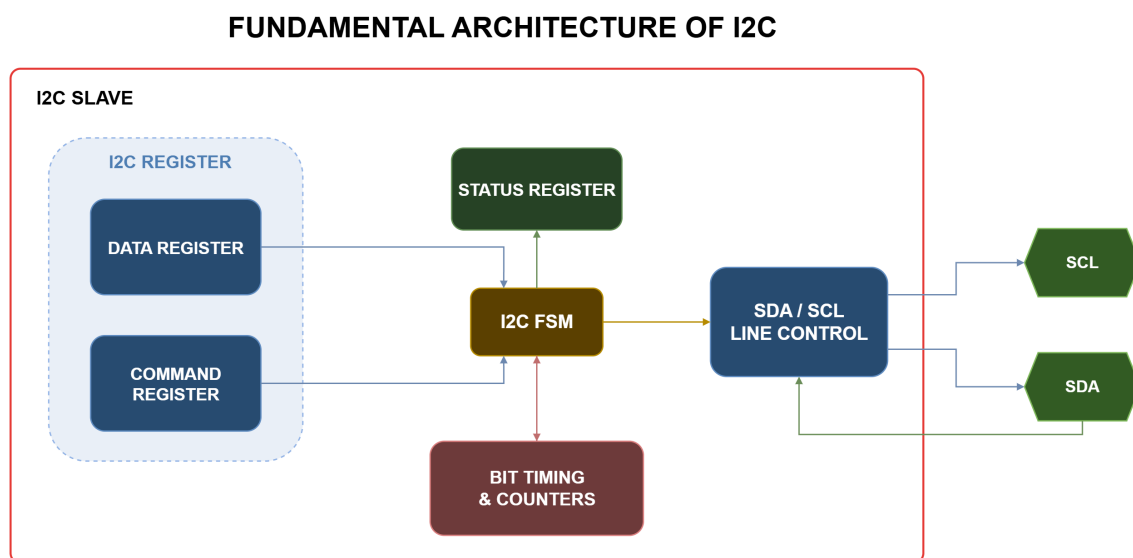
Về nguyên lý, dữ liệu song song bên trong hệ thống được UART chuyển thành chuỗi bit nối tiếp trước khi đưa ra chân truyền. Khi không có dữ liệu, đường truyền thường ở trạng thái rỗi. Một khung truyền bắt đầu bằng bit start để báo cho bên nhận biết thời điểm lấy mẫu, tiếp theo là các bit dữ liệu, sau đó có thể có bit parity và kết thúc bằng một hoặc nhiều bit stop. Bên nhận phát hiện bit start, dùng tốc độ baud đã cấu hình để lấy mẫu các bit kế tiếp và ghép lại thành dữ liệu song song [2, 17].

UART có ưu điểm là cấu trúc đơn giản, ít chân tín hiệu và dễ dùng trong giai đoạn kiểm thử. Chỉ cần hai đường chính là TX và RX, hệ thống đã có thể gửi log, truyền lệnh điều khiển hoặc kiểm tra dữ liệu đầu ra. Hạn chế của UART nằm ở tốc độ không cao bằng các giao tiếp đồng bộ và độ chính xác phụ thuộc vào sai số baud rate giữa hai phía. Nếu tần số lấy mẫu bị lệch nhiều, bên nhận có thể đọc sai bit, đặc biệt khi khung dữ liệu dài hoặc tốc độ truyền cao [17].

Trong SoC, UART được ánh xạ tại địa chỉ APB 0x4000_0000. CPU RISC-V điều khiển UART thông qua các thanh ghi APB. Khi cần gửi dữ liệu, phần mềm ghi byte hoặc word vào thanh ghi truyền; module UART sẽ tuần tự hóa dữ liệu và phát ra chân uart_tx. Khi có dữ liệu đi vào từ chân uart_rx, UART lấy mẫu, phục hồi dữ liệu và đặt vào thanh ghi nhận để CPU đọc lại. Nhờ cơ chế này, UART đóng vai trò kênh giao tiếp cơ bản giữa SoC và môi trường bên ngoài, đồng thời hỗ trợ quan sát hoạt động của hệ thống trong quá trình mô phỏng hoặc chạy trên FPGA [5, 13].

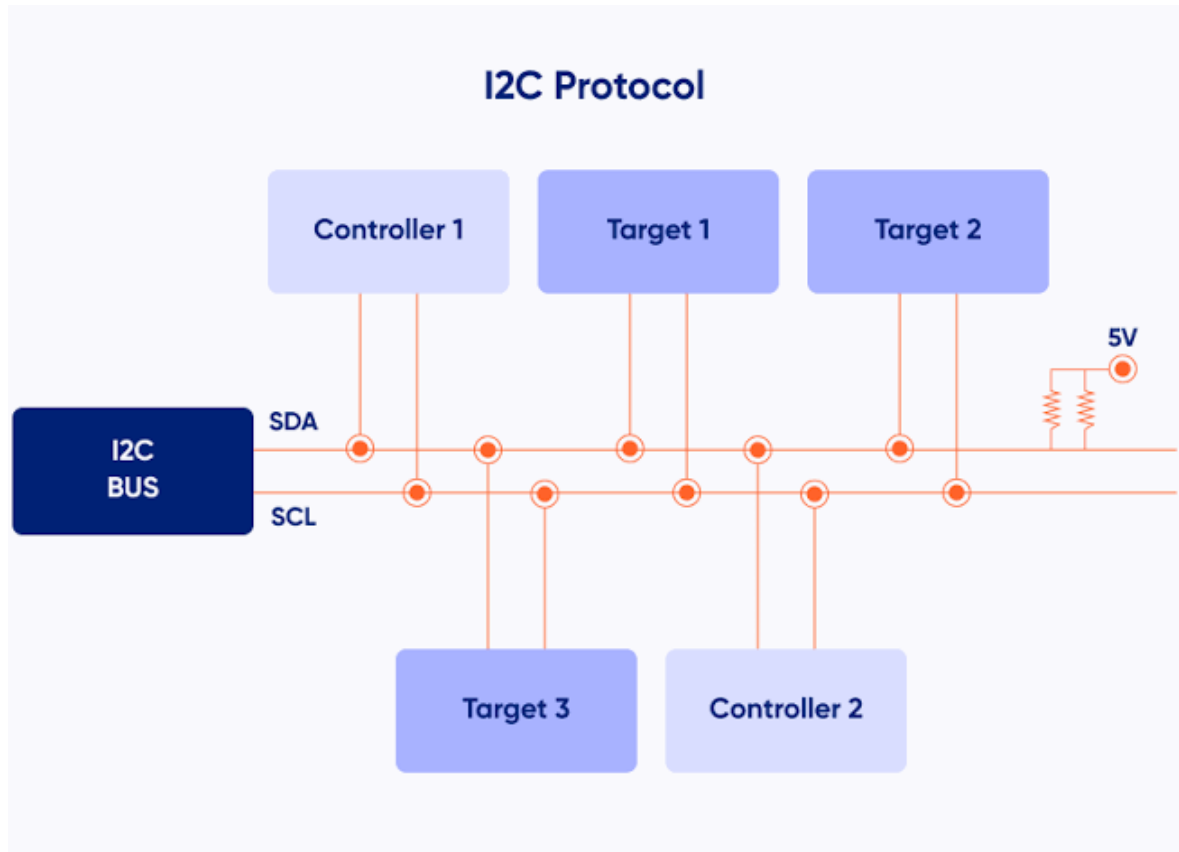
2.5 Giao tiếp I2C

I2C (Inter-Integrated Circuit) là chuẩn giao tiếp nối tiếp đồng bộ sử dụng hai đường tín hiệu chính: SCL và SDA. SCL mang xung clock, còn SDA mang dữ liệu hai chiều. Khác với UART, I2C cho phép nhiều thiết bị cùng chia sẻ một bus thông qua cơ chế địa chỉ slave. Điều này làm I2C phù hợp với các hệ thống cần kết nối nhiều cảm biến, EEPROM, mạch quản lý nguồn hoặc IC cấu hình mà không muốn tiêu tốn quá nhiều chân I/O [19, 20].



Hình 2.5: Kiến trúc và trình tự truyền dữ liệu của giao tiếp I²C [20].

Hình 2.5 minh họa cấu tạo của bộ điều khiển I²C. Khối thanh ghi lưu địa chỉ thiết bị, dữ liệu, lệnh điều khiển và trạng thái ACK/NACK. Khối định thời và bộ đếm bit tạo các pha cần thiết trên đường SCL. Máy trạng thái điều khiển lần lượt các điều kiện START, truyền địa chỉ, truyền hoặc nhận dữ liệu, kiểm tra xác nhận và tạo điều kiện STOP. Khối điều khiển đường truyền quản lý hai tín hiệu SCL và SDA theo cơ chế cực máng hở; thiết bị chỉ kéo tín hiệu xuống mức thấp hoặc nhả đường bus để điện trở kéo lên tạo mức cao [19].



Hình 2.6: Dạng sóng truyền dữ liệu của giao tiếp I^2C [20].

Hình 2.6 minh họa quan hệ thời gian giữa hai đường SCL và SDA trong một giao dịch I^2C , bao gồm điều kiện START, các bit địa chỉ và dữ liệu, bit xác nhận ACK/NACK và điều kiện STOP [19].

Về mặt hoạt động, bus I^2C thường có một master điều khiển quá trình truyền thông và một hoặc nhiều slave phản hồi. Master tạo điều kiện start bằng cách kéo SDA xuống mức thấp khi SCL đang ở mức cao. Sau đó, master gửi địa chỉ slave cùng bit xác định hướng đọc hoặc ghi. Nếu slave nhận ra địa chỉ của mình, nó phản hồi bằng bit ACK. Dữ liệu tiếp tục được truyền theo từng byte, mỗi byte đi kèm một bit ACK hoặc NACK. Giao dịch kết thúc bằng điều kiện stop, khi SDA chuyển từ thấp lên cao trong lúc SCL ở mức cao [19].

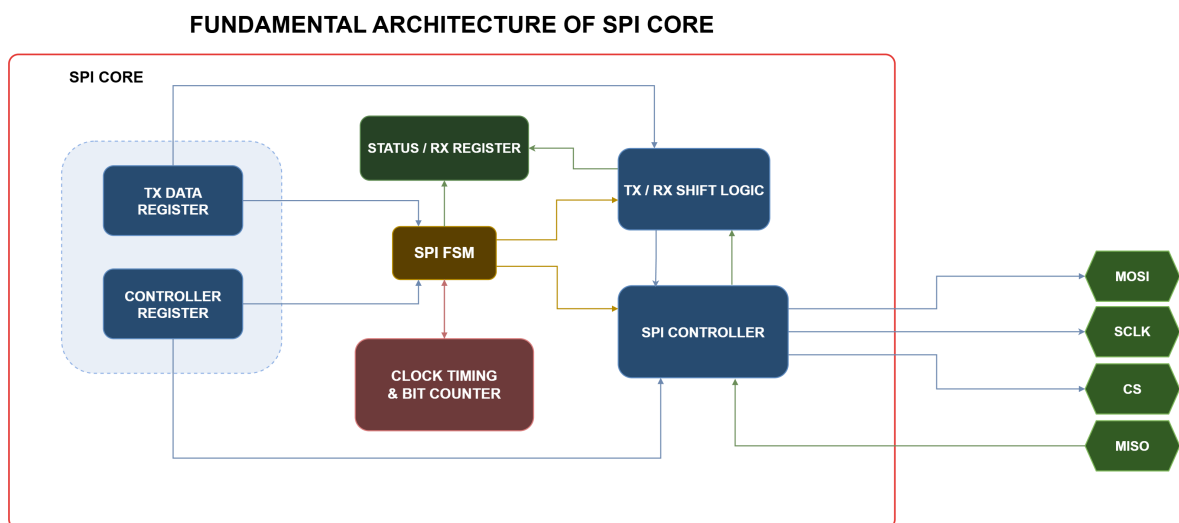
Do SDA là đường hai chiều, các thiết bị trên bus không được tự ý đẩy mức cao một cách chủ động như ngõ ra thông thường. Thực tế, đường SDA và SCL thường hoạt động theo kiểu open-drain hoặc open-collector, dùng điện trở kéo lên để tạo mức cao. Thiết bị chỉ kéo đường truyền xuống thấp khi cần phát logic 0, còn logic 1 được tạo bằng cách nhả đường truyền. Cách tổ chức này giúp nhiều thiết bị cùng nối trên bus mà hạn chế xung đột điện, nhưng cũng làm tốc độ bị ảnh hưởng bởi điện trở kéo

lên và điện dung đường dây [19].

Trong SoC, I2C được ánh xạ tại vùng 0x4000_2000. Các tín hiệu scl_out, sda_out, sda_oe và sda_in thể hiện cách điều khiển đường dữ liệu hai chiều. Tín hiệu sda_oe quyết định lúc module được phép kéo SDA xuống và lúc phải nhả bus để slave phản hồi. CPU cấu hình module I2C thông qua APB để tạo start, gửi địa chỉ, truyền hoặc nhận dữ liệu, kiểm tra ACK và tạo stop. Nhờ đó, SoC có khả năng giao tiếp với nhiều thiết bị tốc độ thấp hoặc trung bình bằng số lượng chân tín hiệu rất nhỏ [13, 19].

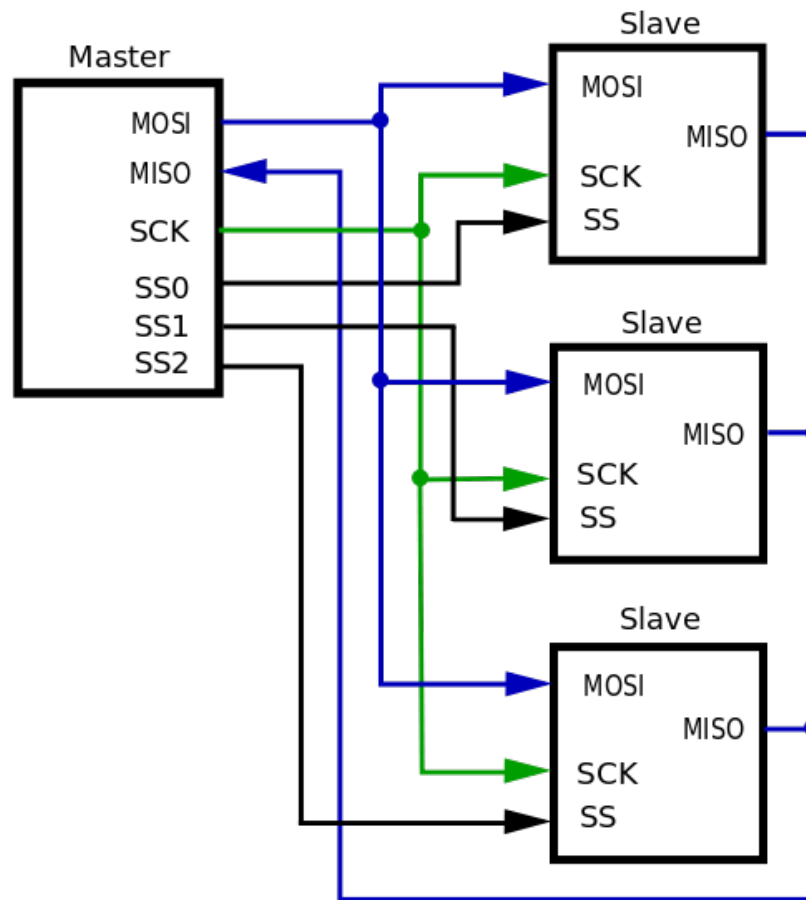
2.6 Giao tiếp SPI

SPI (Serial Peripheral Interface) là chuẩn giao tiếp nối tiếp đồng bộ thường dùng giữa một master và một hoặc nhiều slave. Bus SPI phổ biến với bốn tín hiệu: SCLK, MOSI, MISO và CS. Master tạo xung clock trên SCLK, gửi dữ liệu đến slave qua MOSI, nhận dữ liệu từ slave qua MISO và chọn thiết bị cần giao tiếp bằng tín hiệu CS. Do có clock riêng, SPI thường đạt tốc độ cao hơn UART và đơn giản hơn I2C ở phần định địa chỉ [21, 22].



Hình 2.7: Kiến trúc và luồng dữ liệu song công của giao tiếp SPI [22].

Hình 2.7 trình bày các thành phần chính của bộ điều khiển SPI, bao gồm khối thanh ghi lưu dữ liệu truyền, dữ liệu nhận, cấu hình chế độ và trạng thái giao dịch. Bộ chia clock tạo tín hiệu SCLK, còn bộ đếm bit xác định thời điểm hoàn tất một từ dữ liệu. Máy trạng thái điều khiển tín hiệu chọn thiết bị CS và các pha lấy mẫu theo cấu hình CPOL/CPHA. Hai thanh ghi dịch thực hiện đồng thời việc đưa từng bit ra MOSI và nhận từng bit từ MISO, nhờ đó SPI hỗ trợ truyền dữ liệu song công toàn phần [21].



Hình 2.8: Dạng sóng truyền dữ liệu của giao tiếp SPI [22].

Hình 2.8 minh họa quan hệ thời gian giữa tín hiệu chọn thiết bị, xung clock nối tiếp và các đường dữ liệu MOSI/MISO trong một giao dịch SPI [21].

Hoạt động của SPI dựa trên sự dịch bit đồng thời ở hai hướng. Trong mỗi chu kỳ clock, master phát một bit trên MOSI và đồng thời lấy mẫu một bit từ MISO. Vì vậy, SPI có thể truyền song công toàn phần nếu cả master và slave đều hỗ trợ. Thứ tự bit, cực tính clock và pha lấy mẫu được quyết định bởi cấu hình SPI mode. Hai tham số quan trọng là CPOL và CPHA: CPOL xác định mức rỗi của clock, còn CPHA xác định cạnh clock dùng để lấy mẫu dữ liệu. Master và slave phải dùng cùng chế độ thì dữ liệu mới được nhận đúng [21].

Ưu điểm của SPI là tốc độ cao, giao thức gọn và dễ hiện thực bằng phần cứng. Chuẩn này phù hợp với ADC, DAC, cảm biến tốc độ cao, màn hình, thẻ nhớ hoặc flash ngoài. Nhược điểm chính là số chân chọn chip tăng theo số lượng slave nếu dùng cấu trúc truyền thống. Ngoài ra, SPI không có cơ chế địa chỉ và xác nhận dữ liệu chuẩn

hóa như I2C, nên việc kiểm tra lỗi hoặc quản lý nhiều thiết bị thường do tầng giao thức phía trên đảm nhiệm [21].

Trong hệ thống được xây dựng, SPI nằm ở vùng địa chỉ 0x4000_1000. CPU ghi dữ liệu cần truyền vào thanh ghi của module, cấu hình trạng thái giao dịch và đọc dữ liệu nhận được sau khi quá trình truyền hoàn tất. Các chân spi_sclk, spi_mosi, spi_miso và spi_cs là giao diện vật lý nối ra ngoài. Nhờ kết nối qua APB, SPI trở thành một ngoại vi có thể điều khiển thống nhất bằng phần mềm chạy trên PicoRV32, đồng thời cung cấp kênh truyền tốc độ cao hơn cho các thiết bị ngoại vi cần trao đổi dữ liệu liên tục [13, 21].

2.7 Bus APB và kết nối giữa các mô-đun

APB (Advanced Peripheral Bus) thuộc tập giao thức AMBA của Arm, là chuẩn bus đơn giản được thiết kế cho các thiết bị ngoại vi có yêu cầu băng thông thấp. So với các bus hệ thống như AXI hay AHB, APB có giao diện và quy trình truyền đơn giản hơn, thuận lợi cho việc kết nối các thanh ghi ngoại vi [5, 13].

Bổ sung hình kiến trúc APB tại images/APB.jpg

Hình 2.9: Kiến trúc kết nối tín hiệu giữa APB Master và APB Slave [23].

Sơ đồ kết nối tín hiệu giữa khối APB Master và APB Slave được minh họa trên Hình 2.9. Một giao dịch truyền nhận cơ bản trên bus APB diễn ra tuần tự qua hai pha tín hiệu chính gắn liền với các cạnh lên của xung clock hệ thống (PCLK) [13]:

- Pha thiết lập (Setup Phase): Giai đoạn này bắt đầu khi khối Master đưa địa chỉ cần truy cập lên bus PADDR, đặt dữ liệu ghi lên PWDATA (nếu là giao dịch ghi), xác định hướng truyền qua tín hiệu PWRITE (1 cho ghi, 0 cho đọc), đồng thời bật tín hiệu chọn thiết bị PSEL lên mức cao. Trạng thái này duy trì đúng một chu kỳ clock để ổn định bus địa chỉ và điều khiển.

- Pha kích hoạt (Enable Phase): Ngay ở chu kỳ clock tiếp theo, tín hiệu cho phép PENABLE được Master kéo lên mức cao để đánh dấu khởi động quá trình trao đổi dữ liệu thực sự. Khối Slave phản hồi bằng cách kéo tín hiệu sẵn sàng PREADY lên mức cao khi hoàn tất xử lý. Đối với giao dịch đọc, dữ liệu trả về từ Slave sẽ được chốt trên bus PRDATA tại thời điểm này trước khi cả PENABLE và PSEL hạ xuống mức thấp để kết thúc giao dịch.

Các tín hiệu và trình tự hai pha nêu trên tuân theo đặc tả AMBA APB [13].

Trong kiến trúc SoC được xây dựng, mô-đun cầu nối `picorv32_apb_bridge` đóng vai trò là APB Master. Khối này tiếp nhận các yêu cầu đọc/ghi từ giao diện *native memory* của lõi xử lý PicoRV32, sau đó tự động biên dịch và điều phối các tín hiệu bắt tay kể trên thành một giao dịch APB chuẩn. Đi kèm với đó, bộ giải mã địa chỉ (*Address Decoder*) trong khối top-level ngoại vi sẽ phân tích vùng địa chỉ PADDR để kích hoạt đường PSEL tương ứng cho từng ngoại vi Slave (UART, SPI, I2C, hoặc khối parallel) [11, 13].

Nhờ cơ chế tổ chức này, SoC có thể điều khiển và trao đổi dữ liệu với toàn bộ các mô-đun phần cứng thông qua một không gian địa chỉ thống nhất. Mặc dù APB không trực tiếp đảm nhiệm cơ chế truyền dữ liệu hai cạnh clock, bus này cung cấp lớp truy cập thanh ghi để CPU PicoRV32 khởi tạo cấu hình, phát lệnh và giám sát trạng thái của các khối giao tiếp trong hệ thống [6, 13].

2.8 Timing và giới hạn tần số hoạt động

Trong các thiết kế mạch số đồng bộ (Synchronous Digital Circuits), việc đảm bảo tính đúng đắn về mặt dữ liệu phụ thuộc vào các ràng buộc thời gian (Timing Constraints). Dữ liệu sau khi chốt tại phần tử lưu trữ nguồn (Source Flip-Flop) phải lan truyền qua mạng logic tổ hợp và đến được phần tử lưu trữ đích (Destination Flip-Flop) trước cạnh chốt clock tiếp theo. Để tránh vi phạm thời gian thiết lập (*Setup Time Violation*), chu kỳ xung clock hệ thống T_{clk} phải thỏa mãn điều kiện bất đẳng thức cơ bản [16]:

$$T_{clk} \geq t_{cq} + t_{logic} + t_{setup} + t_{skew}, \quad (2.3)$$

trong đó t_{cq} (*Clock-to-Q Delay*) là độ trễ từ lúc xuất hiện cạnh clock đến khi dữ liệu ổn định ở ngõ ra thanh ghi nguồn; t_{logic} biểu diễn tổng độ trễ đường truyền qua logic tổ hợp và dây dẫn kết nối (*Routing Delay*); t_{setup} là thời gian duy trì dữ liệu ổn định tối thiểu trước cạnh clock chốt tại thanh ghi đích; và t_{skew} (*Clock Skew*) đại

diện cho độ lệch thời điểm xuất hiện cạnh clock giữa hai thanh ghi nguồn và đích trên mạng phân phối xung nhịp [16].

Khi nâng cao tần số hoạt động của SoC (tương ứng với việc thu hẹp chu kỳ T_{clk}), khoảng thời gian dành cho dữ liệu lan truyền bị thu hẹp, khiến bất đẳng thức trên khó đáp ứng hơn. Nếu dữ liệu đến thanh ghi đích trễ hơn thời gian cho phép, thiết kế sẽ vi phạm ràng buộc setup và có thể hoạt động không ổn định trên phần cứng [16].

Đối với phương thức truyền dữ liệu hai cạnh clock DDR, yêu cầu timing closure trở nên chặt chẽ hơn. Do dữ liệu được phát và lấy mẫu ở cả cạnh lên và cạnh xuống, khoảng thời gian giữa hai lần truyền chỉ còn một nửa chu kỳ xung nhịp ($T_{clk}/2$). Điều này đòi hỏi nhà thiết kế không chỉ kiểm soát t_{setup} mà còn phải đảm bảo ràng buộc thời gian giữ t_{hold} . Bên cạnh đó, các yếu tố như độ nhiễu pha xung nhịp (*Clock Jitter*), độ lệch pha giữa các đường truyền song song (*Data-to-Data Skew*) và khả năng đồng bộ miền xung nhịp (CDC) cũng ảnh hưởng đến tính toàn vẹn tín hiệu [9, 16].

Trong phạm vi đề tài, quá trình phân tích và tối ưu timing được thực hiện định lượng thông qua bộ công cụ Vivado. Thiết kế SoC được khảo sát qua từng mức tần số clock tăng dần để đánh giá đường truyền tới hạn (*Critical Path*). Giá trị slack, đặc biệt là chỉ số WNS (*Worst Negative Slack*), được dùng làm tiêu chí đánh giá [16, 24]:

- Slack ≥ 0 (Slack dương): Thiết kế hoàn toàn đạt ràng buộc timing, dữ liệu truyền nhận chính xác ở tần số khảo sát.
- Slack < 0 (Slack âm): Hệ thống bị vi phạm timing. Lúc này, các giải pháp kỹ thuật RTL sẽ được áp dụng như: chèn các tầng thanh ghi trung gian (*Pipelining*) để cắt ngắn độ trễ t_{logic} , tái cấu trúc logic tổ hợp, tối ưu hóa sơ đồ đặt và nối dây (*Place & Route*), hoặc sử dụng các khối quản lý xung nhịp chuyên dụng (như MMCM/PLL) để điều chỉnh pha xung nhịp linh hoạt.

Các tiêu chí slack và phương pháp tối ưu nêu trên phù hợp với quy trình timing closure của Vivado [16].

Việc tăng tần số clock chỉ mang lại giá trị hiệu năng khi thiết kế vẫn đạt timing closure và duy trì tính đúng đắn trên phần cứng FPGA [16].

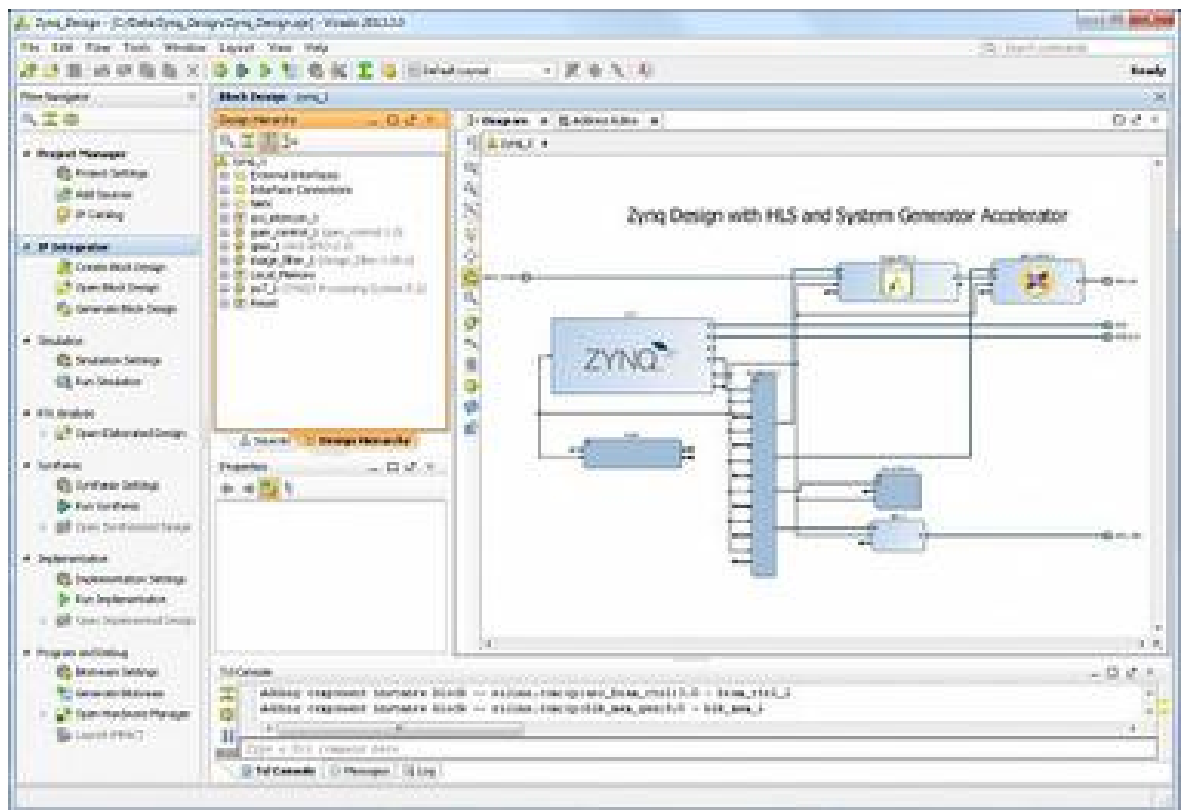
2.9 Công cụ Vivado

Vivado Design Suite là môi trường phát triển phần cứng tích hợp do Xilinx, hiện thuộc AMD, phát triển cho quy trình thiết kế, kiểm thử và triển khai hệ thống số trên FPGA

và Adaptive SoC. Công cụ tích hợp các tác vụ quản lý mã nguồn RTL, mô phỏng chức năng, tổng hợp logic, đặt và nối dây, phân tích timing và tạo tập tin cấu hình bitstream trong một môi trường thống nhất [24].

Quy trình triển khai SoC RISC-V trên Vivado được thực hiện qua các giai đoạn khởi tạo dự án, tích hợp nguồn RTL, mô phỏng, tổng hợp và hiện thực. Các tệp Verilog cấu thành SoC gồm lõi picorv32.v, cầu nối apb_bridge.v, bộ giải mã soc_apb_top.v, khối parallel.v và các ngoại vi APB. Vivado Simulator hỗ trợ chạy testbench để kiểm tra từng khối và toàn bộ hệ thống trước khi tổng hợp [11, 24].

Sau khi hoàn tất mô phỏng chức năng, tập tin ràng buộc .xdc được thiết lập để định nghĩa tần số clock mục tiêu, gán chân I/O và khai báo chuẩn điện áp. Vivado tổng hợp mã RTL thành netlist, sau đó thực hiện placement và routing nhằm đáp ứng các ràng buộc timing trước khi tạo bitstream để nạp lên phần cứng [16, 24].



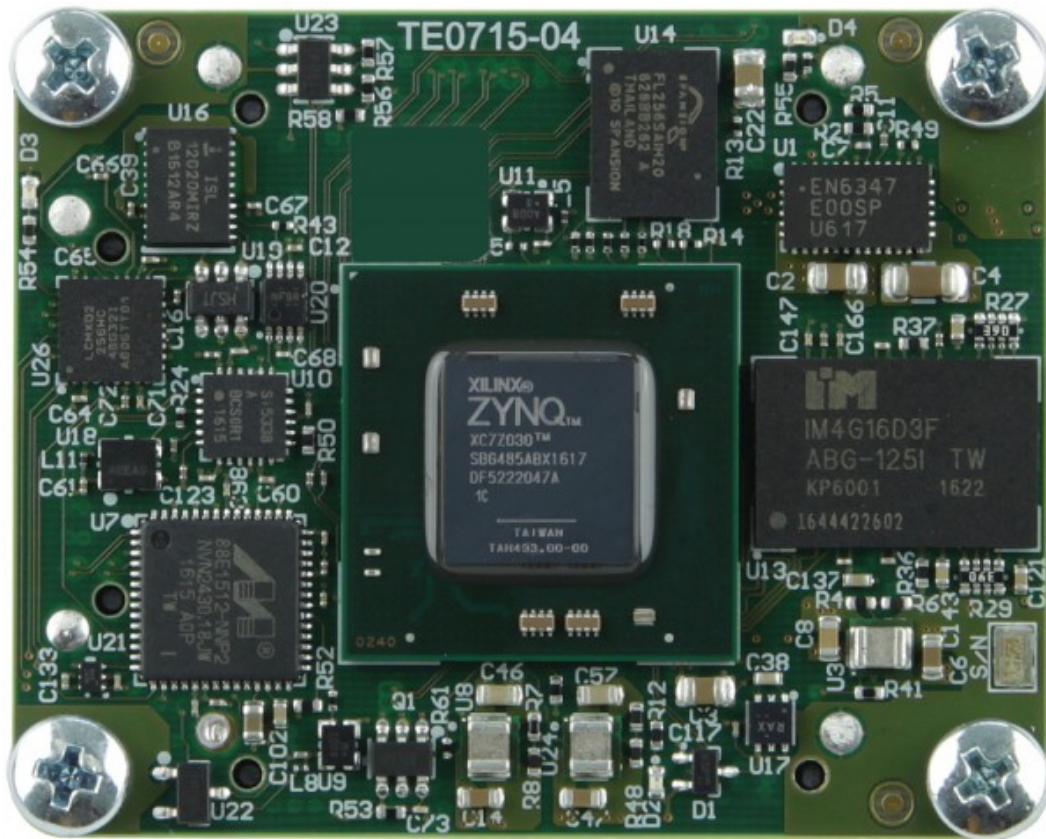
Hình 2.10: Giao diện phần mềm Vivado Design Suite [25].

Vivado cung cấp các báo cáo sau tổng hợp và hiện thực để tinh chỉnh kiến trúc RTL. Báo cáo sử dụng tài nguyên thống kê LUT, thanh ghi, Block RAM và DSP; báo cáo timing cung cấp WNS, TNS và đường truyền tối hạn; trong khi công cụ phân tích công suất ước tính công suất tĩnh và động. Các dữ liệu này tạo cơ sở định lượng cho

việc tối ưu thiết kế SoC [16, 24].

2.10 Nền tảng FPGA Xilinx 7 Series và thiết bị mục tiêu

Họ FPGA Xilinx 7 Series cung cấp các tài nguyên LUT, Flip-Flop, Block RAM, bộ đệm clock toàn cục và phần tử I/O chuyên dụng để hiện thực các hệ thống SoC tùy biến. Hình 2.11 minh họa một bo mạch Zynq XC7Z030 thuộc cùng thế hệ công nghệ 7 Series, trong đó vùng logic lập trình được có thể chứa lõi RISC-V, bus APB và các ngoại vi tùy biến [26–28]. Tuy nhiên, thiết bị tổng hợp và đánh giá chính thức của khóa luận là FPGA Artix-7 xc7a200tfbg676-2L [29].



Hình 2.11: Bo mạch Zynq XC7Z030 thuộc nền tảng Xilinx 7 Series [28].

Ảnh bo mạch được sử dụng để minh họa cách các tài nguyên FPGA 7 Series được tích hợp với bộ nhớ và ngoại vi trên một nền tảng phát triển [28]. Thiết kế trong khóa luận được tổng hợp như một hệ thống FPGA độc lập trên Artix-7; vì vậy, các kết quả tài nguyên, timing và công suất ở Chương 5 đều được tính theo cấu hình xc7a200tfbg676-2L trên AMD Vivado 2025.2 [24, 29].

Bảng 2.2: Thông số cấu hình và tài nguyên của FPGA Artix-7 mục tiêu

Nhóm thông số	Giá trị hoặc đặc điểm kỹ thuật
Họ thiết bị	Xilinx/AMD Artix-7
Mã thiết bị	xc7a200tfbg676-2L
Slice LUTs khả dụng	134,600
Slice Registers khả dụng	269,200
Block RAM Tile (RAMB36E1)	365
Bonded I/O (IOB)	400
Global Clock Buffer (BUFG)	32
Clock hệ thống mục tiêu	100 MHz
Tốc độ dữ liệu DDR mục tiêu	200 Mbps/pin

Các thông số thiết bị trong Bảng 2.2 được tổng hợp từ tài liệu DS181 và báo cáo thiết bị của Vivado [24, 29]. Các tài nguyên này tạo không gian đủ lớn để triển khai lõi PicoRV32, bộ nhớ chương trình, bus APB, các ngoại vi UART/SPI/I²C và khối phát DDR. Việc sử dụng cùng mã thiết bị trong mô tả nền tảng và báo cáo thực nghiệm giúp các tỷ lệ sử dụng tài nguyên ở Chương 5 được đối chiếu nhất quán.

Bên cạnh tài nguyên logic, việc triển khai yêu cầu khai báo đúng clock, reset, chân I/O và chuẩn điện áp trong tệp .xdc. Các tín hiệu UART, SPI, I2C và DDR phải được ánh xạ đúng chân vật lý; đồng thời chuẩn điện áp của từng I/O bank phải phù hợp với phần cứng để bảo đảm tính toàn vẹn tín hiệu [16].

Quy trình kiểm chứng dựa trên mô phỏng RTL và báo cáo sau hiện thực để đánh giá clock, routing và các ràng buộc I/O. Đo dạng sóng trực tiếp bằng máy hiện sóng hoặc bộ phân tích logic là bước kiểm chứng phần cứng tiếp theo.

Chương 3

CÁC NGHIÊN CỨU LIÊN QUAN

3.1 Tổng quan các hướng nghiên cứu về SoC trên nền RISC-V

Sự phát triển mạnh mẽ của kiến trúc tập lệnh mở RISC-V [12] đã thúc đẩy nhiều nghiên cứu thiết kế vi xử lý và hệ thống trên chip (SoC) tích hợp trên nền tảng FPGA. Nhằm tối ưu hóa diện tích thực thi và mức tiêu thụ tài nguyên cho các ứng dụng nhúng gọn nhẹ, lõi PicoRV32 [11] được phát triển như một bộ xử lý 32-bit (RV32I/IMC) có kích thước siêu nhỏ gọn. Tuy nhiên, bản thân công trình PicoRV32 [11] chỉ tập trung vào việc cung cấp lõi CPU thuần túy với giao diện bộ nhớ đơn giản (Native Memory Interface). Việc triển khai lõi này vào một SoC hoàn chỉnh đòi hỏi phải ghép nối thêm hệ thống cầu nối bus chuẩn hóa và các khối giao tiếp ngoại vi phù hợp.

Ở quy mô phức tạp hơn, các khung làm việc như BlackParrot [1] hay Simply-V [30] được đề xuất để xây dựng các hệ thống SoC RISC-V đa nhân, hỗ trợ tích hợp các bộ tăng tốc phần cứng chuyên dụng. Bên cạnh đó, các nghiên cứu như của Ma và cộng sự [31] tập trung tích hợp các lõi mã hóa mật độ thấp vào SoC. Mặc dù các công trình này [1, 30, 31] giải quyết rất tốt bài toán nâng cao năng lực tính toán và tính linh hoạt của hệ thống, phạm vi nghiên cứu của họ chủ yếu tập trung vào kiến trúc CPU và bus hệ thống tốc độ cao (như AXI), chưa hướng tới việc tối ưu hóa băng thông cho các khối ngoại vi APB gọn nhẹ bằng kỹ thuật truyền dữ liệu hai cạnh clock.

3.2 Các giải pháp tối ưu băng thông và giao tiếp ngoại vi trên FPGA

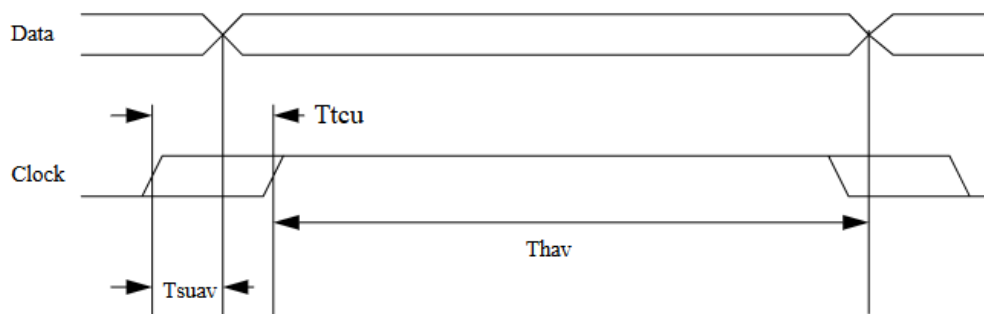
Để truyền tải dữ liệu giữa lõi xử lý và các mô-đun ngoại vi, các chuẩn giao tiếp nối tiếp kinh điển như UART [2–4, 17], SPI [21] và I2C [19] được triển khai rất phổ biến trên FPGA. Nhiều công trình đã tập trung chuẩn hóa các khối giao tiếp này thông qua việc kết nối với hạ tầng bus AMBA APB [5, 13] hoặc AXI [6]. Các nghiên cứu của Ananthu [32] cũng như Liu [5] chỉ ra rằng bus APB [13] là giải pháp phù hợp cho các

ngoại vi tốc độ trung bình nhờ giao thức điều khiển tối giản, giúp tiết kiệm tài nguyên LUT và giảm độ phức tạp định tuyến so với bus AXI.

Mặc dù vậy, đa số các nghiên cứu tích hợp ngoại vi qua bus APB hiện nay [2–5, 13, 32] đều áp dụng cơ chế truyền dữ liệu theo một cạnh clock (SDR - Single Data Rate). Với phương thức truyền SDR, khi cần gia tăng lưu lượng truyền nhận dữ liệu qua giao diện ngoại vi, giải pháp thường dùng là nâng tần số clock hệ thống. Tuy nhiên, việc tăng tần số clock trên toàn bộ hệ thống bus có thể làm gia tăng công suất tiêu thụ động và đặt ra thách thức lớn hơn về tối ưu thời gian (Timing Closure) cho các tuyến mạch tổ hợp [16].

3.3 Nghiên cứu về kỹ thuật DDR và quản lý Timing ở tần số cao

Kỹ thuật truyền dữ liệu hai cạnh clock (DDR) [14] là phương pháp lấy mẫu dữ liệu tại cả cạnh lên và cạnh xuống của xung nhịp, giúp tăng gấp đôi lưu lượng truyền lý thuyết tại cùng một tần số clock. Trong các hệ thống số, kỹ thuật DDR được ứng dụng ở hai mức độ khác nhau: bộ điều khiển bộ nhớ chính DDR SDRAM/DDR3/DDR4 (Memory Controller) [7, 10, 33–36] và khối giao tiếp truyền nhận dữ liệu tốc độ cao (DDR I/O Interface) [8, 9, 37].



Hình 3.1: So sánh mối quan hệ định thời ngõ vào (Input Timing) giữa Clock và Data ở phương thức căn lề cạnh (Edge-aligned) và căn lề giữa (Center-aligned) tại phía nhận (chuyển thể từ [38]).

Như được minh họa trên Hình 3.1, một thách thức lớn khi thiết kế khối nhận (Receiver) trong giao tiếp DDR tốc độ cao là đảm bảo tín hiệu xung nhịp ngõ vào lấy mẫu dữ liệu một cách tin cậy [38]:

- Mô hình căn lề cạnh ngõ vào (Edge-aligned Input): Tín hiệu Clock và Data đến các chân ngõ vào (Input ports) của FPGA có các cạnh chuyển trạng thái trùng

nhau. Do cạnh lấy mẫu của Clock nằm ngay tại thời điểm dữ liệu bắt đầu thay đổi, phía nhận (Receiver) sẽ gặp rủi ro cao vi phạm thời gian thiết lập (t_{setup}) và giữ (t_{hold}) nếu không thực hiện dịch pha xung nhịp nội bộ [38].

- Mô hình căn lề giữa ngõ vào (Center-aligned Input): Tín hiệu Clock ngõ vào đã được thiết bị phát (Transmitter) dịch pha 90° (hoặc được FPGA chủ động dịch pha thông qua khối MMCM/PLL nội bộ). Cạnh lấy mẫu của Clock do đó rơi vào chính giữa mắt dữ liệu (Data Eye), giúp tối đa hóa biên độ dư thừa thời gian (Timing Margin) cho cả t_{setup} (T_{suav}) và t_{hold} (T_{hav}) tại các thanh ghi đầu vào [38].

Do đó, việc thiết lập chính xác các ràng buộc thời gian ngõ vào (set_input_delay) trong tệp XDC [39] trên phần mềm Vivado [24] đóng vai trò quan trọng. Ràng buộc này mô tả độ lệch pha giữa dữ liệu ngoài chip và clock tham chiếu tại giao diện FPGA, qua đó cung cấp cơ sở để Vivado phân tích setup/hold và để người thiết kế lựa chọn cấu trúc IDELAY hoặc MMCM phù hợp [40].

Đối với nhóm nghiên cứu về bộ điều khiển bộ nhớ DDR SDRAM [7, 35, 36], mục tiêu chính là quản lý chu kỳ định thời phức tạp, chẳng hạn t_{RP} , t_{RCD} và độ trễ CAS, của các chip RAM vật lý ngoài chip. Một số thiết kế [35, 36] được xây dựng dưới dạng các khối độc lập (stand-alone), chưa đóng gói thành ngoại vi theo các chuẩn bus nhúng thông dụng như APB để CPU có thể truy xuất trực tiếp. Trong khi đó, các nghiên cứu ứng dụng đường truyền DDR cho mạch đệm hoặc mạng trên chip (NoC) [8, 9, 37] đã chứng minh hiệu quả của việc nhân đôi tốc độ truyền dữ liệu nội bộ.

Cần làm rõ rằng, luận văn này không hướng tới việc thiết kế một bộ điều khiển bộ nhớ DDR SDRAM/DDR3 phức tạp, mà tập trung nghiên cứu ứng dụng kỹ thuật truyền hai cạnh clock (DDR I/O) cho khối giao tiếp ngoại vi kết nối qua bus APB.

Khi triển khai các đường truyền DDR ở tần số cao, bài toán quản lý Timing đóng vai trò quyết định. Tài liệu SelectIO của AMD mô tả các ô phần cứng chuyên dụng IDDR/ODDR, trong khi hướng dẫn XDC và phương pháp UltraFast cung cấp quy trình khai báo, phân tích và đóng định thời [16, 39, 40]. Nếu ràng buộc không phản ánh đúng quan hệ clock–data ngoài chip, hệ thống có thể không đạt Timing Closure ($WNS < 0$) và tiềm ẩn nguy cơ sai dữ liệu khi vận hành thực tế.

3.4 Bảng so sánh tổng hợp và định hướng của đề tài

Từ việc tổng hợp và phân tích phạm vi của các công trình công bố trước đây, Bảng 3.1 tóm tắt sự so sánh giữa các nghiên cứu liên quan [1–14, 16, 17, 19, 21, 24, 29–37, 39, 40] với định hướng tiếp cận của luận văn.

Dựa trên việc phân tích phạm vi và kế thừa kết quả từ các nghiên cứu trước, luận văn xác định mục tiêu xây dựng một hệ thống SoC RISC-V hoàn chỉnh dựa trên lõi PicoRV32 [11], tích hợp hệ thống bus AMBA APB [13] và khối giao tiếp ngoại vi hỗ trợ truyền dữ liệu hai cạnh clock (DDR) [14]. Các mục tiêu cụ thể bao gồm:

1. Đóng gói chuẩn hóa ngoại vi APB: Xây dựng và đóng gói khối giao tiếp hai cạnh clock DDR [14] thành một mô-đun ngoại vi chuẩn APB [13], cho phép lõi CPU PicoRV32 [11] cấu hình và trao đổi dữ liệu một cách trực tiếp và nhất quán.
2. Tối ưu lưu lượng giao tiếp ngoại vi: Tăng gấp đôi lưu lượng truyền nhận dữ liệu trên giao diện ngoại vi so với phương thức SDR truyền thống [5, 32] tại cùng một tần số xung nhịp hệ thống.
3. Đạt trạng thái Timing Closure: Áp dụng phương pháp luận UltraFast [16] và thiết lập tệp ràng buộc XDC [39] trên Vivado [24] nhằm kiểm soát các chỉ số dư thừa thời gian, bảo đảm chỉ số $WNS \geq 0$ khi hệ thống hoạt động ở tần số mục tiêu.
4. Kiểm chứng thực nghiệm: Tổng hợp và thực thi thiết kế SoC cho FPGA Artix-7, mã thiết bị xc7a200tfbg6762L [29]. Tài nguyên sử dụng, timing và công suất được đánh giá bằng các báo cáo của Vivado [24].

Bảng 3.1: Bảng tổng hợp so sánh phạm vi các nghiên cứu liên quan và định hướng đề tài

Nghiên cứu / Tài liệu	Nền tảng & Kiến trúc	Phạm vi tập trung của nghiên cứu	Định hướng trong luận văn
PicoRV32 [11], Simply-V [30]	Lõi RV32I, Native / AXI Interface	Tối ưu hóa cấu trúc lõi CPU [11] hoặc xây dựng SoC đa nhân [30]; chưa áp dụng DDR cho ngoại vi APB.	Sử dụng lõi PicoRV32 [11] kết hợp cầu nối APB [13] và khối giao tiếp ngoại vi DDR.
Giao tiếp APB [5, 32] & UART/SPI [2, 21]	SoC Bus, Bus AMBA APB / AHB	Chuẩn hóa giao tiếp ngoại vi gọn nhẹ; dữ liệu truyền theo phương thức SDR 1 cạnh clock.	Mở rộng khối ngoại vi APB hỗ trợ truyền dữ liệu 2 cạnh clock (DDR) để tăng lưu lượng dữ liệu.
Bộ điều khiển DDR SDRAM [7, 35, 36]	Stand-alone Memory Controller	Điều khiển chip nhớ DDR RAM ngoài; khối thiết kế riêng biệt, chưa đóng gói chuẩn APB.	Tối ưu khối truyền nhận DDR I/O gọn nhẹ, đóng gói thành ngoại vi APB chuẩn cho CPU điều khiển.
DDR Buffer / NoC [8, 9, 37]	High-Speed Buffer, DDR-NoC	Tăng tốc độ truyền nội bộ; phạm vi chưa đi sâu vào phân tích WNS và Timing Closure trên APB.	Tối ưu đường truyền tối hạn (Critical Path) và thiết lập XDC [39] để đạt $WNS \geq 0$.
Tài liệu AMD/Xilinx [16, 29, 39, 40]	FPGA 7 Series, Vivado Engine	Cung cấp đặc tính thiết bị, tài nguyên SelectIO, quy tắc ràng buộc và phương pháp luận UltraFast.	Tổng hợp và đánh giá mô hình SoC trên FPGA Artix-7 XC7A200T.

Chương 4

MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ NGHỊ

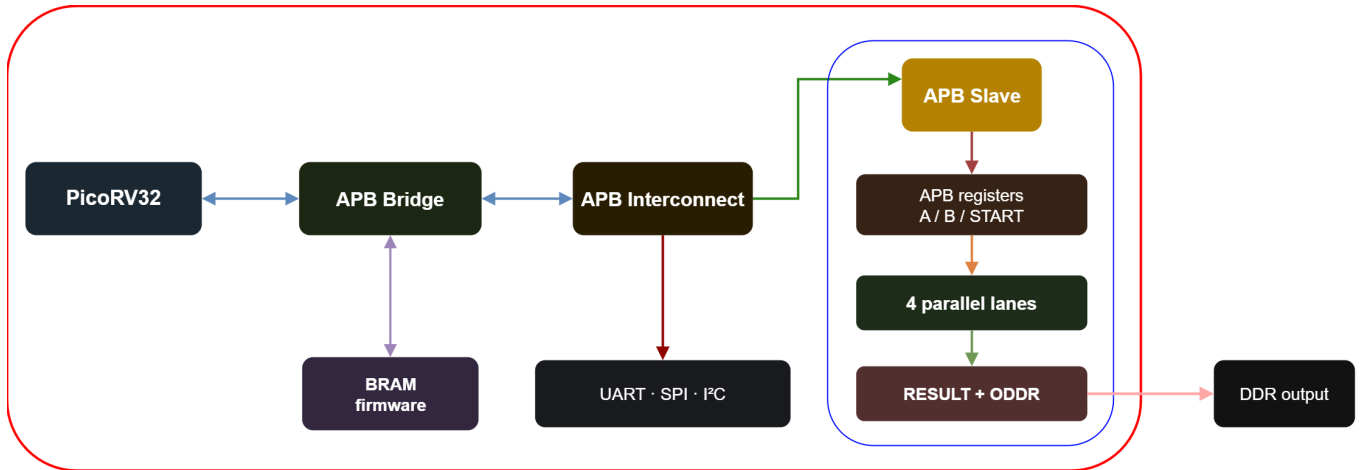
4.1 Tổng quan kiến trúc hệ thống đề xuất

Kiến trúc hệ thống trên chip (SoC) đề xuất được thiết kế nhằm tối ưu hóa băng thông giao tiếp ngoại vi và kiểm chứng khả năng truyền dữ liệu hai cạnh clock (DDR) trên nền tảng FPGA. Trung tâm của hệ thống là lõi xử lý RISC-V PicoRV32 đảm nhiệm vai trò điều khiển và phát các yêu cầu truy cập bộ nhớ. Để đảm bảo tính đóng gói và tiêu tốn ít tài nguyên phần cứng, hạ tầng bus AMBA APB được chọn làm tuyến bus nội bộ kết nối CPU với các mô-đun ngoại vi [11, 13, 14].

Nhằm khẳng định rõ đóng góp khoa học của đề tài cũng như đảm bảo tính minh bạch học thuật, phạm vi thiết kế phần cứng được phân định cụ thể thành hai nhóm thành phần chính:

- Các khối kế thừa từ mã nguồn mở: Lõi RISC-V PicoRV32 đảm nhiệm vai trò CPU trung tâm, còn BRAM chứa mã chương trình khởi tạo (`firmware.hex`). Việc kế thừa giúp bảo đảm tính chuẩn hóa của hạ tầng xử lý lệnh và độ tin cậy khi thực thi phần mềm [11, 12].
- Các khối tự thiết kế và tích hợp: Hệ thống gồm cầu nối `picorv32_apb_bridge`, bộ giải mã địa chỉ APB, khối xử lý song song đa làn và bộ phát dữ liệu hai cạnh clock. Các thành phần được thiết kế ở mức RTL bằng Verilog HDL và đóng gói vào SoC.

Toàn bộ hệ thống được chia thành hai miền chức năng chính gồm miền xử lý trung tâm và miền ngoại vi APB. PicoRV32 lấy lệnh từ BRAM nội bộ thông qua giao diện Native Memory; các giao dịch dữ liệu đến ngoại vi được chuyển sang APB bởi khối `picorv32_apb_bridge`. Hệ thống ngoại vi gồm UART, SPI, I2C và khối parallel. Khối parallel vừa thực hiện bốn phép cộng độc lập vừa phát bốn kết quả đó dưới dạng luồng DDR 32 bit. Sơ đồ tổng thể hệ thống được minh họa trên Hình 4.1.



Hình 4.1: Sơ đồ khối tổng thể kiến trúc SoC RISC-V với khối xử lý song song và đầu ra DDR.

4.2 Thiết kế chi tiết các khối chức năng

4.2.1 Lỗi xử lý PicoRV32 và bộ giải mã địa chỉ

Lỗi xử lý PicoRV32 được cấu hình hoạt động theo tập lệnh RV32I cơ bản, sử dụng giao diện Native Memory để trao đổi dữ liệu với bên ngoài. Giao diện này bao gồm các tín hiệu địa chỉ `mem_addr`, dữ liệu ghi `mem_wdata`, dữ liệu đọc `mem_rdata`, tín hiệu cho phép `mem_valid`, tín hiệu phản hồi `mem_ready` và tín hiệu chọn byte `mem_wstrb` [11, 12].

Không gian địa chỉ 32 bit được phân bổ như trong Bảng 4.1. Vùng địa chỉ thấp dành cho BRAM 4 KB, trong đó tệp `firmware.hex` được nạp làm chương trình khởi động. Các giao dịch đến vùng ngoại vi từ `0x4000_0000` đến `0x4000_3FFF` được chuyển qua APB bridge.

Tại phía slave, `soc_apb_top` và `soc_top` giải mã tín hiệu `PSEL` để chọn đúng ngoại vi.

4.2.2 Cầu nối bus `picorv32_apb_bridge`

Khối `picorv32_apb_bridge` chuyển các giao dịch dữ liệu của giao diện Native Memory sang APB. Giao dịch lấy lệnh được nhận biết bằng `mem_instr` và trả về từ BRAM qua `bram_rdata`; giao dịch dữ liệu được giữ địa chỉ, hướng truyền và dữ liệu ghi trong các thanh ghi APB. Cầu nối sử dụng ba trạng thái `S_IDLE`, `S_SETUP` và `S_ENABLE`. Cách tổ chức này bảo đảm `PADDR`, `PWRITE` và `PWDATA` ổn định trong hai

Bảng 4.1: Bản đồ phân bố không gian địa chỉ (Memory Map) của hệ thống SoC

Vùng địa chỉ	Mô-đun mục tiêu	Chức năng kỹ thuật
0x0000_0000--0x0000_0FFF	BRAM (4 KB)	Chứa firmware khởi động
0x4000_0000--0x4000_0FFF	APB UART	Giao tiếp nối tiếp bất đồng bộ
0x4000_1000--0x4000_1FFF	APB SPI	Giao tiếp nối tiếp đồng bộ
0x4000_2000--0x4000_2FFF	APB I2C	Giao tiếp chuẩn 2 dây
0x4000_3000--0x4000_3FFF	Parallel Block	Bộ xử lý song song 4 lanes

pha Setup và Enable của APB [11, 13].

Ở trạng thái S_SETUP, PSEL đã ở mức cao còn PENABLE bằng không. Sang trạng thái S_ENABLE, cầu nối đưa PENABLE lên mức cao và chờ PREADY từ ngoại vi được giải mã. Với giao dịch đọc, PRDATA được chép vào mem_rdata; sau đó mem_ready được phát trong một chu kỳ để hoàn tất giao dịch với PicoRV32. Mã nguồn 4.1 là phần RTL tương ứng trong thiết kế.

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state      <= S_IDLE;
        PSEL       <= 1'b0;
        PENABLE    <= 1'b0;
        mem_ready  <= 1'b0;
    end else begin
        mem_ready <= 1'b0;
        case (state)
            S_IDLE: begin
                PSEL    <= 1'b0;
                PENABLE <= 1'b0;
                if (mem_valid) begin
                    if (mem_instr) begin
                        mem_rdata <= bram_rdata;
                        mem_ready <= 1'b1;
                    end else begin
                        PADDR    <= mem_addr;
                        PWRITE   <= |mem_wstrb;
                    end
                end
            end
        endcase
    end
end

```

```

        PWDATA  <= mem_wdata;
        PSEL    <= 1'b1;
        state   <= S_SETUP;
    end
end
end
S_SETUP: begin
    PENABLE <= 1'b1;
    state   <= S_ENABLE;
end
S_ENABLE: begin
    if (PREADY) begin
        PSEL      <= 1'b0;
        PENABLE   <= 1'b0;
        mem_ready <= 1'b1;
        if (!PWRITE)
            mem_rdata <= PRDATA;
        state <= S_IDLE;
    end
end
endcase
end
end

```

Listing 4.1: Máy trạng thái trong picorv32_apb_bridge.

Cơ chế hoạt động và phân tích chi tiết mã nguồn FSM

Mã nguồn trong Listing 4.1 thực hiện quá trình chuyển đổi giao thức (Protocol Translation) giữa giao diện bus Native Memory của PicoRV32 và bus AMBA APB. Luồng xử lý chi tiết theo các trạng thái diễn ra như sau:

1. Trạng thái rảnh (S_IDLE):

- Cầu nối giám sát tín hiệu `mem_valid`. Khi `mem_valid = 1`, CPU bắt đầu phát sinh một truy xuất bộ nhớ.
- **Lấy lệnh** (`mem_instr = 1`): Cầu nối nhận biết đây là chu kỳ nạp lệnh (Instruction Fetch). Tín hiệu `mem_rdata` lập tức được gán sang `mem_rdata`, đồng thời `mem_ready = 1` được kích hoạt ngay chu kỳ này để trả kết quả cho CPU mà không qua bus APB, giúp giảm thiểu tối đa độ trễ nạp lệnh (Instruction Fetch Latency).

- **Truy xuất dữ liệu** (`mem_instr = 0`): Cầu nối chuyển sang giao dịch đọc/ghi ngoại vi APB. Địa chỉ `PADDR` được gán từ `mem_addr`, hướng truyền `PWRITE` xác định qua mặt nạ byte `|mem_wstrb`, và dữ liệu ghi `PWDATA` lấy từ `mem_wdata`. Cầu nối kéo `PSEL = 1` để khởi động pha Setup và chuyển sang trạng thái `S_SETUP`.

2. Pha thiết lập (`S_SETUP`):

- Tín hiệu `PENABLE` được đưa lên mức 1 trong khi `PSEL` tiếp tục giữ ở mức 1. Điều này đảm bảo tuân thủ quy định của chuẩn AMBA APB: tín hiệu `PENABLE` được kích hoạt sau pha thiết lập với `PSEL`. Kết thúc chu kỳ này, FSM tự động chuyển sang `S_ENABLE` [13].

3. Pha kích hoạt và hoàn tất (`S_ENABLE`):

- Cầu nối duy trì các tín hiệu điều khiển và chờ tín hiệu phản hồi `PREADY = 1` từ phía ngoại vi.
- Khi tín hiệu `PREADY` đạt mức 1, giao dịch hoàn tất. Nếu `PWRITE` bằng 0, dữ liệu `PRDATA` được lưu vào `mem_rdata`.
- Các tín hiệu `PSEL` và `PENABLE` được đưa về 0 để giải phóng bus, đồng thời `mem_ready = 1` được phát ra trong một chu kỳ để báo cho CPU hoàn tất giao dịch và quay về trạng thái `S_IDLE`.

Tác dụng chính của đoạn mã là đồng bộ hóa sự khác biệt giao thức giữa Native Memory (vốn dựa trên cơ chế bắt tay đơn giản `valid/ready`) và bus AMBA APB (yêu cầu chuỗi pha `PSEL` → `PENABLE` chặt chẽ), đồng thời chốt giữ ổn định các đường tín hiệu địa chỉ và dữ liệu để tránh hiện tượng trôi hay vấp tín hiệu (glitch) trong suốt chu kỳ bus.

4.2.3 Khối xử lý song song và truyền dữ liệu DDR

Khối parallel được ánh xạ tại vùng `0x4000_3000--0x4000_3FFF` và đóng vai trò bộ tăng tốc song song đa lần. Khối tích hợp đầu ra DDR để thực hiện đồng thời các phép tính do CPU giao và tuần tự hóa kết quả phát ra ngoài chip.

1. Cơ chế ghi/đọc thanh ghi và kích hoạt tính toán

Hệ thống bố trí tập thanh ghi APB để CPU tương tác với khối xử lý song song:

- Thanh ghi toán hạng A (Offset 0x10--0x1C): Chứa 4 giá trị toán hạng $A[0]$ đến $A[3]$ (mỗi giá trị 32 bit).
- Thanh ghi toán hạng B (Offset 0x20--0x2C): Chứa 4 giá trị toán hạng $B[0]$ đến $B[3]$ (mỗi giá trị 32 bit).
- Thanh ghi điều khiển START (Offset 0x00): Khi CPU ghi bit 1 vào thanh ghi này, tín hiệu kích hoạt được phát ra để 4 lane đồng thời thực hiện phép cộng 32 bit: $Result[i] = A[i] + B[i]$ (với $i = 0 \dots 3$).
- Thanh ghi kết quả (Offset 0x30--0x3C): Chứa 4 kết quả tính toán. Đường đọc APB cho phép CPU truy xuất lại các thanh ghi này bất kỳ lúc nào.

2. Cơ chế đóng gói và phân tích mã nguồn bộ phát DDR

Song song với việc lưu kết quả vào thanh ghi để CPU đọc, toàn bộ 128 bit kết quả ($4 \text{ lane} \times 32 \text{ bit}$) được nạp vào bộ tuần tự hóa `ddr_result_serializer` nhằm đẩy ra các chân I/O ngoài chip qua 2 chu kỳ clock (tương ứng 4 cạnh xung nhịp).

Mã nguồn 4.2 thể hiện đoạn kết nối giữa luồng kết quả tính toán và khối tuần tự hóa DDR.

```
// Generate the load condition for the DDR serializer
assign fire_start = start_pending & core_in_ready & {NUM_LANES{ddr_ready}};

// Serialize four 32-bit results over four clock edges
ddr_result_serializer #(DATA_WIDTH(32)) u_ddr_result_serializer (
    .clk          (clk),
    .rst_n        (rst_n),
    .load         (!fire_start),
    .data_rise_0   (result_flat[31:0]),
    .data_fall_0   (result_flat[63:32]),
    .data_rise_1   (result_flat[95:64]),
    .data_fall_1   (result_flat[127:96]),
    .ddr_data      (ddr_data),
    .ddr_valid     (ddr_valid),
    .ddr_done      (ddr_done),
    .ddr_ready     (ddr_ready)
);
```

Listing 4.2: Giao tiếp giữa kết quả các lần tính toán và bộ phát DDR.

Giải thích mã nguồn Listing 4.2:

1. Dòng 2 (assignfire_start): Tín hiệu fire_start đóng vai trò là điều kiện bắt tay (handshake). Lệnh tính toán chỉ thực sự được nạp khi có yêu cầu thực thi (start_pending), lõi tính toán rảnh (core_in_ready), và quan trọng nhất là khối phát DDR đang sẵn sàng (ddr_ready=1). Điều này ngăn chặn hiện tượng ghi đè dữ liệu khi batch trước chưa phát xong.
2. Dòng 8 (.load): Sử dụng phép OR thu gọn (|fire_start) để tạo xung nạp dữ liệu. Khi có hiệu lệnh, 128 bit kết quả ghép từ 4 lane (result_flat) lập tức được chốt vào các thanh ghi đầu vào của bộ phát.
3. Dòng 9–12 (.data_rise_X, .data_fall_X): Mảng dữ liệu 128 bit được chia thành 4 từ dữ liệu 32 bit:
 - data_rise_0 (Lane 0): Phát tại cạnh lên của chu kỳ clock thứ nhất.
 - data_fall_0 (Lane 1): Phát tại cạnh xuống của chu kỳ clock thứ nhất.
 - data_rise_1 (Lane 2): Phát tại cạnh lên của chu kỳ clock thứ hai.
 - data_fall_1 (Lane 3): Phát tại cạnh xuống của chu kỳ clock thứ hai.
4. Dòng 13–16 (ddr_data, ddr_valid, ddr_done, ddr_ready): Quản lý luồng đầu ra. ddr_valid=1 kéo lên trong suốt 2 chu kỳ phát dữ liệu; ddr_done phát ra một xung ngắn đánh dấu hoàn tất truyền dữ liệu của một đợt tính toán.

3. Nguyên lý phát dữ liệu hai cạnh xung nhịp qua Primitive ODDR

Để đưa bus dữ liệu 32 bit ra chân vật lý của FPGA với tốc độ gấp đôi mà không cần tăng gấp đôi tần số clock hệ thống, thiết kế sử dụng phần tử phần cứng chuyên dụng ODDR (Output Double Data Rate) nằm tại khối IOB (I/O Block) của chip AMD/Xilinx [40].

Mã nguồn 4.3 mô tả cách khởi tạo primitive ODDR cho từng đường truyền dữ liệu.

```
// Instantiate one ODDR primitive for each output bit
ODDR #(
    .DDR_CLK_EDGE ("SAME_EDGE"),
    .INIT          (1'b0),
    .SRTYPE        ("ASYNC")
) u_oddr (
    .Q (ddr_data[bit_index]),
    .C (clk),
    .CE (1'b1),
    .D1 (rise_word[bit_index]),
    .D2 (fall_word[bit_index]),
    .R (~rst_n),
    .S (1'b0)
);
```

Listing 4.3: Khởi tạo phần tử cứng ODDR trong AMD/Xilinx FPGA.

Phân tích chuyên sâu về phần tử cứng ODDR (Listing 4.3):

- **Cấu hình DDR_CLK_EDGE="SAME_EDGE":** Trong chế độ này, dữ liệu đưa vào D1 và D2 được tổ chức để phát lần lượt tại cạnh lên và cạnh xuống của xung nhịp C, trong khi việc nạp dữ liệu được đồng bộ theo cùng một cạnh clock. Cấu hình này đơn giản hóa đường định thời dữ liệu trong lớp logic FPGA [40].
- **Ngõ ra Q:** Được nối thẳng tới chân I/O vật lý của chip FPGA (ddr_data[bit_index]). Nhờ sử dụng primitive IOB chuyên dụng thay vì dùng Flip-Flop thông thường trong lát logic (CLB), độ trễ đường truyền từ thanh ghi ra chân chip được giảm thiểu và có tính đồng đều cao giữa các đường truyền của bus 32 bit.

4.2.4 Các khối ngoại vi chuẩn giao tiếp nối tiếp (UART, SPI, I2C)

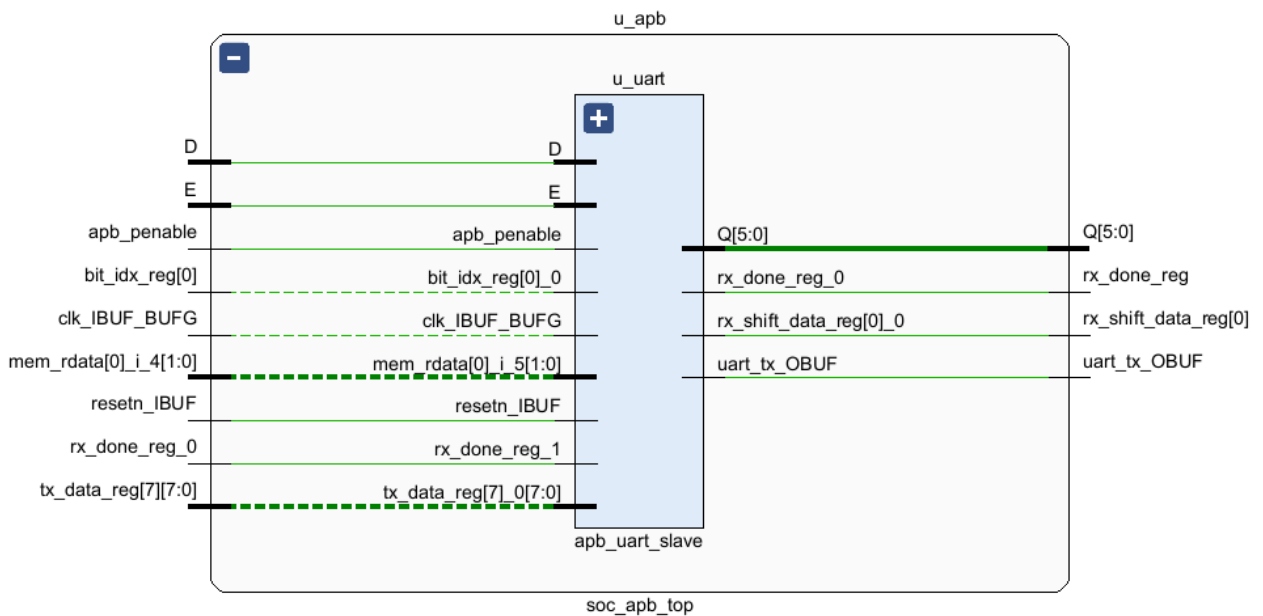
Bên cạnh khối xử lý song song và đầu ra DDR, hệ thống SoC còn tích hợp các khối ngoại vi chuẩn giao tiếp nối tiếp đóng vai trò là giao diện kết nối giữa SoC với các linh kiện ngoài chip và hệ thống giám sát:

- **Khối APB UART (0x4000_0000):** Đảm nhiệm vai trò truyền nhận dữ liệu chuỗi bất đồng bộ với máy tính chủ (Host PC) phục vụ mục đích truyền nhận dữ liệu kiểm thử và xuất log điều khiển (debug).

- Khối APB SPI (0x4000_1000): Cung cấp giao diện truyền dữ liệu nối tiếp đồng bộ tốc độ cao, hỗ trợ kết nối với các thiết bị ngoại vi như bộ nhớ Flash ngoài hoặc màn hình hiển thị.
- Khối APB I2C (0x4000_2000): Cung cấp giao diện giao tiếp 2 dây chuẩn (SDA, SCL) phục vụ việc đọc/ghi dữ liệu cấu hình từ các cảm biến hoặc IC quản lý nguồn.

Tất cả các mô-đun ngoại vi trên đều được đóng gói chuẩn giao diện APB Slave, kết nối trực tiếp vào bộ giải mã địa chỉ của soc_apb_top và được quản lý thông qua tập thanh ghi cấu hình tương ứng trong bảng bản đồ bộ nhớ (Bảng 4.1). Nguyên lý hoạt động của các giao thức UART, SPI và I2C được đối chiếu lần lượt với các tài liệu kỹ thuật [17, 19, 21]; giao diện thanh ghi ngoại vi tuân theo trình tự truyền APB [13].

Khối giao tiếp nối tiếp bất đồng bộ (APB UART Slave)



Hình 4.2: Sơ đồ RTL của khối giao tiếp APB UART.

Mô-đun apb_uart_slave thực hiện việc truyền nhận dữ liệu bất đồng bộ chuẩn UART. Kiến trúc chính bao gồm giao diện APB, bộ phát (TX) và bộ nhận (RX) độc lập. Cấu trúc khung truyền và nguyên tắc xác định tốc độ baud được xây dựng theo đặc tả UART thông dụng [17]. Các đoạn mã Verilog cốt lõi được thể hiện trong Listing 4.4.

```
// 1. Giao ma giao dien APB Read/Write
wire apb_write = UART_PSEL && UART_PENABLE && UART_PWRITE;
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        tx_data <= 8'h0; tx_start <= 1'b0;
    end else begin
        tx_start <= 1'b0;
        if (apb_write && (UART_PADDR[7:0] == 8'h00)) begin
            tx_data <= UART_PWDATA[7:0];
            tx_start <= 1'b1; // Kich hoat truyen TX
        end
    end
end

// 2. TX shift register (PISO)
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin ... end
    else if (tx_start && !tx_active) begin
        tx_active <= 1'b1;
        tx_shift_reg <= {1'b1, tx_data, 1'b0}; // Khung: Stop(1) + Data(8) + Start
        (0)
    end else if (tx_active && clk_cnt == BIT_PERIOD - 1) begin
        tx_shift_reg <= tx_shift_reg >> 1; // Dich phai tung bit
        if (bit_idx == 9) tx_active <= 1'b0;
    end
end

// 3. Two-stage RX synchronizer and start-bit detector
reg [1:0] rx_sync;
always @(posedge PCLK) rx_sync <= {rx_sync[0], uart_rx};

always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin ... end
    else begin
        if (rx_clear) rx_done <= 1'b0;
        if (!rx_active && (rx_sync[1] == 1'b0)) begin // Phat hien canh xuong Start
            bit
            rx_active <= 1'b1;
        end else if (rx_active && rx_clk_cnt == BIT_PERIOD - 1) begin
            if (rx_bit_idx >= 1 && rx_bit_idx <= 8)
                rx_shift_data <= {rx_sync[1], rx_shift_data[7:1]}; // Lay mau du
            lieu RX
        end
    end
end
```

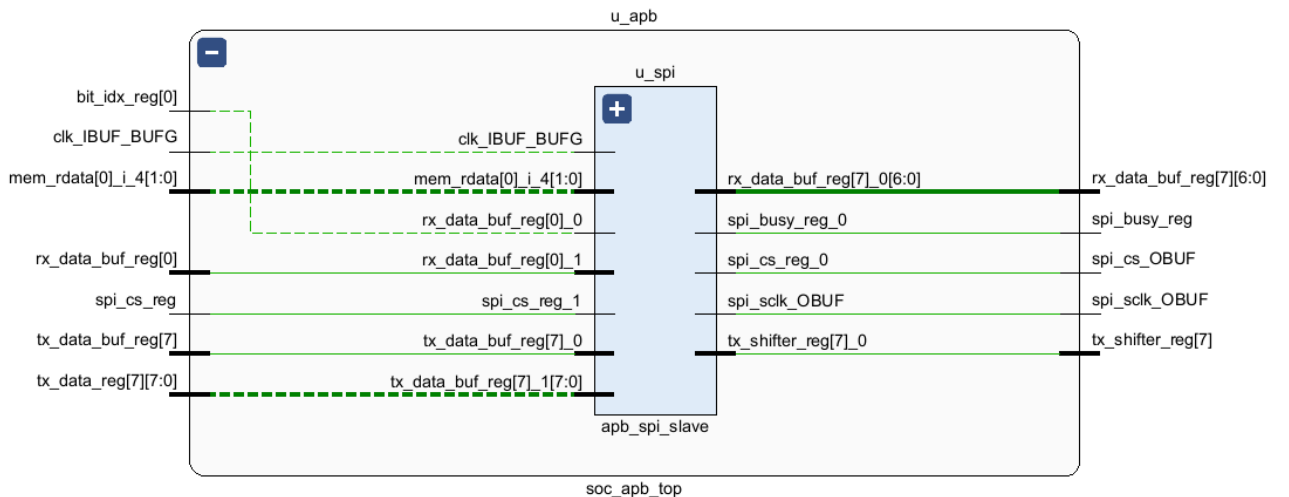
end

Listing 4.4: Các đoạn mã RTL cốt lõi của mô-đun *apb_uart_slave*.

giải thích mã nguồn khối *apb_uart_slave*:

- Phân giải baud rate: Tốc độ truyền được tính toán qua hằng số $BIT_PERIOD = CLK_FREQ/BAUD_RATE$. Mạch đếm *clk_cnt* chia tần số *PCLK* để định thời chính xác khoảng thời gian của 1 bit.
- Khối truyền (TX): Thanh ghi dịch *tx_shift_reg* 10 bit được nạp định dạng bao gồm 1 bit Start (0), 8 bit dữ liệu *tx_data*, và 1 bit Stop (1). Dữ liệu được dịch phải tuần tự ra đường phát *uart_tx*.
- Khối nhận (RX) và chống hiện tượng bất ổn định: Đường vào *uart_rx* được đưa qua thanh ghi đồng bộ 2 tầng *rx_sync* nhằm loại bỏ hiện tượng bất ổn định trạng thái (Metastability) khi tín hiệu bất đồng bộ đi vào miền xung nhịp *PCLK*. Khi phát hiện cạnh xuống (bit Start), mạch nhận thực hiện dịch 8 bit dữ liệu vào *rx_shift_data* và kích hoạt cờ *rx_done*. Cờ này tự động xóa (*rx_clear*) khi CPU thực hiện thao tác đọc thanh ghi dữ liệu nhận qua bus APB.

Khối giao tiếp nối tiếp đồng bộ (APB SPI Slave)



Hình 4.3: Sơ đồ RTL của khối giao tiếp APB SPI.

Mô-đun *apb_spi_slave* chuyển đổi các truy xuất APB thành giao thức truyền nối tiếp 4 dây SPI Master. Quan hệ giữa cực tính, pha xung nhịp và thời điểm lấy mẫu

dữ liệu được thiết lập theo SPI Mode 0 [21]. Các đoạn mã FSM và dịch dữ liệu quan trọng được trình bày tại Listing 4.5.

```
// 1. Gia ma thanh ghi Chip Select (CS) qua APB
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) spi_cs <= 1'b1;
    else if (SPI_PSEL && SPI_PENABLE && SPI_PWRITE && SPI_PADDR[7:0] == 8'h04)
        spi_cs <= SPI_PWDATA[0]; // Dieu khien spi_cs bang phan mem
end

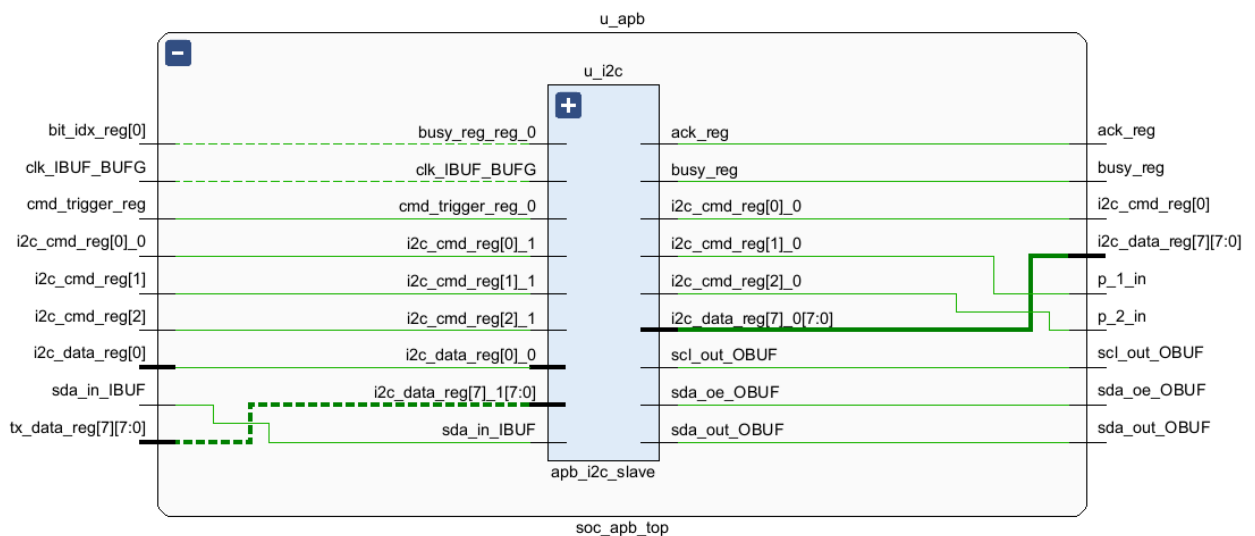
// 2. May trang thai (FSM) dieu khien SCLK va truyen nhan SPI (Mode 0)
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin ... end
    else begin
        case (state)
            0: begin // IDLE
                spi_sclk <= 1'b0;
                if (start_tx && !spi_cs) begin
                    spi_busy <= 1'b1;
                    tx_shifter <= tx_data_buf;
                    state <= 1;
                end
            end
            1: begin // Canh len SCLK - Lay mau du lieu MISO
                spi_sclk <= 1'b1;
                if (clk_div == 8'd5) begin
                    rx_data_buf <= {rx_data_buf[6:0], spi_miso};
                    state <= 2;
                end
            end
            2: begin // Falling SCLK edge: drive the next MOSI bit
                spi_sclk <= 1'b0;
                if (clk_div == 8'd5) begin
                    if (bit_cnt < 7) begin
                        bit_cnt <= bit_cnt + 1;
                        tx_shifter <= tx_shifter << 1; // Dich bit MSB ra MOSI
                        state <= 1;
                    end else state <= 0; // Hoan thanh 8-bit
                end
            end
        endcase
    end
end
end
```

Listing 4.5: Các đoạn mã RTL cốt lõi của mô-đun *apb_spi_slave*.

Giải thích mã nguồn khối `apb_spi_slave`:

- Điều khiển chọn chip: Phần mềm điều khiển `spi_cs` bằng thanh ghi tại offset `0x04`. Khi `spi_cs` bằng 0, giao tiếp SPI sẵn sàng hoạt động.
- Máy trạng thái SPI: FSM dùng bộ đếm `clk_div` để tạo tín hiệu `spi_sclk`.
- Truyền nhận Mode 0: Ở cạnh lên của `spi_sclk`, dữ liệu `spi_miso` được chốt vào `rx_data_buf`. Ở cạnh xuống, `tx_shifter` đẩy bit kế tiếp ra `spi_mosi`. Sau 8 bit, FSM trở về trạng thái nghỉ và hạ cờ `spi_busy`.

Khởi giao tiếp 2 dây (APB I2C Slave)



Hình 4.4: Sơ đồ RTL của khối giao tiếp APB I²C.

Mô-đun `apb_i2c_slave` tạo giao diện giao tiếp I2C Master. Các điều kiện START, STOP, chu kỳ ACK và đặc tính ngõ ra kiểu open-drain được triển khai theo đặc tả I2C-bus [19]. Đoạn mã trọng tâm điều khiển các trạng thái và chân ba trạng thái được thể hiện trong Listing 4.6.

```
// 1. Máy trạng thái điều khiển giao thực I2C (START, STOP, DATA, ACK)
always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin ... end
    else begin
        case (state)
            0: begin // IDLE: Tra SDA và SCL về mức cao
                scl_out <= 1'b1; sda_out <= 1'b1; sda_oe <= 1'b1;
```

```

        if (cmd_trigger) begin
            if (i2c_cmd[0])      state <= 1; // Lehn START
            else if (i2c_cmd[1]) state <= 3; // Lehn STOP
            else if (i2c_cmd[2]) begin bit_idx <= 7; state <= 5; end // Ghi
        end
    end
end

// Tao dieu kien START (SDA gia xuong khi SCL dang cao)
1: begin sda_out <= 1'b0; if (clk_cnt == 50) state <= 2; end
2: begin scl_out <= 1'b0; if (clk_cnt == 50) state <= 0; end

// Truyen 8-bit du lieu (MSB truoc)
5: begin
    sda_out <= i2c_data[bit_idx];
    if (clk_cnt == 25) scl_out <= 1'b1;
    if (clk_cnt == 50) begin
        scl_out <= 1'b0;
        if (bit_idx > 0) bit_idx <= bit_idx - 1;
        else state <= 6; // Chuyen sang kiem tra ACK
    end
end

// Release SDA and sample the ACK bit from the slave
6: begin
    sda_oe <= 1'b0; // FPGA tha noi chan SDA (Tri-state)
    if (clk_cnt == 25) begin scl_out <= 1'b1; ack_reg <= sda_in; end //
    Lay mau ACK
    if (clk_cnt == 50) begin scl_out <= 1'b0; state <= 0; end
end
endcase
end
end
end

```

Listing 4.6: Các đoạn mã RTL cốt lõi của mô-đun *apb_i2c_slave*.

Giải thích nguồn khởi *apb_i2c_slave*:

- Điều khiển SDA ba trạng thái: Tín hiệu *sda_oe* điều khiển tầng đệm ở wrapper trên cùng. Khi tín hiệu bằng 1, FPGA chủ động điều khiển SDA; khi bằng 0, FPGA nhả đường SDA để thiết bị slave phản hồi ACK.
- Thực thi lệnh bằng FSM: Máy trạng thái giải mã thanh ghi *i2c_cmd* tại offset 0x04:

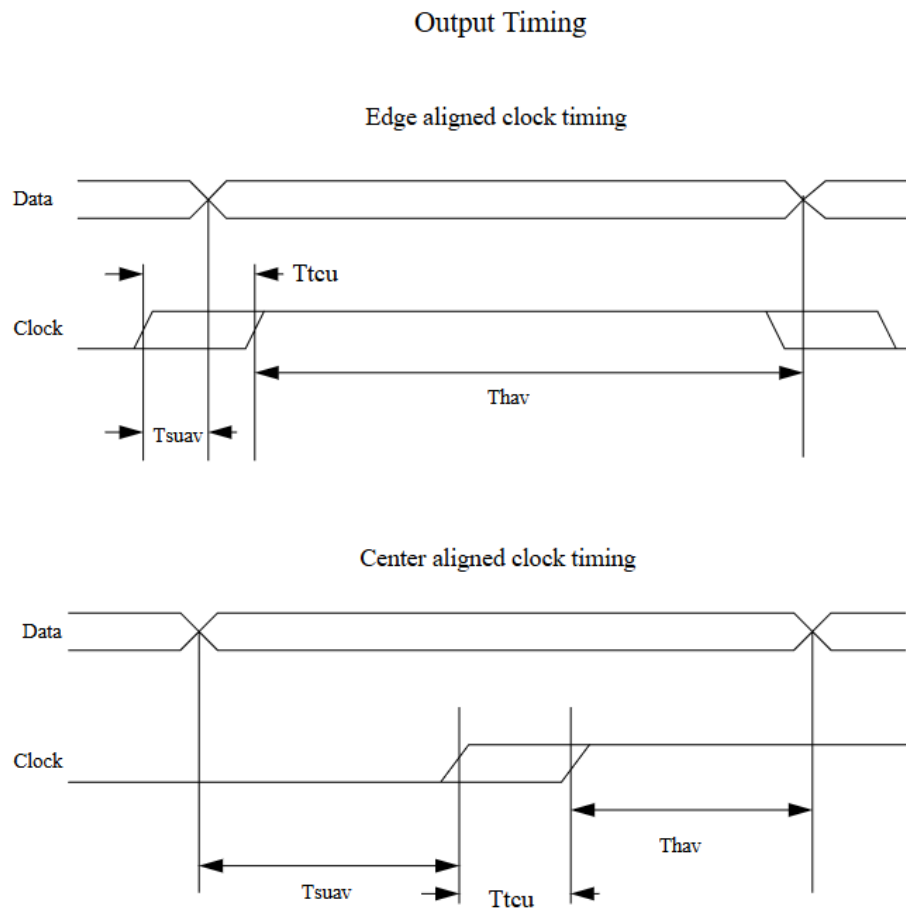
- START (trạng thái 1–2): Tạo cạnh xuống trên SDA khi SCL đang ở mức cao.
- STOP (trạng thái 3–4): Tạo cạnh lên trên SDA khi SCL đang ở mức cao.
- Truyền byte và đọc ACK (trạng thái 5–6): Dịch dữ liệu từ MSB đến LSB, sau đó lấy mẫu `sda_in` vào `ack_reg` tại cạnh lên của SCL.

4.3 Phương pháp xử lý Timing và ràng buộc XDC trên Vivado

Việc phát dữ liệu tại cả hai cạnh của xung nhịp làm khoảng thời gian giữa hai lần thay đổi dữ liệu bị thu hẹp xuống xấp xỉ $T_{clk}/2$ (tương ứng 5.0 ns tại tần số 100 MHz). Do đó, phân tích định thời phải bao phủ cả yêu cầu thiết lập và giữ của các đường tới ô ODDR cũng như giao tiếp từ IOB đến thiết bị nhận [39, 40].

Phân tích mã nguồn RTL tập trung vào khối parallel và bộ tuần tự hóa DDR cho thấy thiết kế hiện tại bộc lộ ba điểm hạn chế về định thời phần cứng cần được nâng cấp khi triển khai thực tế:

1. **Đường tới hạn (Critical Path) chưa có tầng chốt Pipeline:** Kết quả từ các bộ cộng tổ hợp $A[i] + B[i]$ trong `parallel_lane` được kết nối trực tiếp vào các cổng vào `data_rise_X` và `data_fall_X` của khối phát DDR trong cùng chu kỳ kích hoạt `fire_start`. Đường mạch tổ hợp kéo dài từ thanh ghi toán hạng qua bộ cộng 32-bit đến ô ODDR tạo ra trễ mạch tổ hợp (t_{logic}) lớn, dễ gây vi phạm thời gian thiết lập (t_{setup}) khi hệ thống vận hành ở tần số cao.
2. **Thiếu tín hiệu xung nhịp đồng hành (Forwarded Clock Out):** Dữ liệu DDR phát ra chân chip nhưng chưa có đường xung nhịp đồng bộ đi kèm. Do ảnh hưởng của trễ đệm I/O (IOB delay) biến thiên theo nhiệt độ và điện áp, phía thiết bị nhận bên ngoài sẽ khó xác định chính xác vị trí cạnh xuống để lấy mẫu nửa chu kỳ sau nếu không có một đường clock đồng dạng truyền kèm.
3. **Rủi ro cạnh tranh dữ liệu (Race Condition) trong FSM:** Trong bộ tuần tự hóa DDR, tại trạng thái `PHASE_FIRST`, từ dữ liệu `rise_word` và `fall_word` bị ghi đè ngay ở cạnh lên xung nhịp trong khi ODDR đang lấy mẫu cặp dữ liệu đầu tiên. Việc cập nhật dữ liệu ngay tại thời điểm chốt của primitive cứng I/O có thể dẫn đến hiện tượng chốt sai giá trị giữa các lần.



Hình 4.5: *Mối quan hệ định thời giữa Clock và Data ở hai phương thức Edge-aligned (trên) và Center-aligned (dưới) trong giao tiếp truyền dữ liệu DDR ngõ ra (chuyển thể từ [38]).*

Nguyên tắc căn lề thời gian cần xem xét khi mở rộng giao tiếp DDR ra ngoài FPGA được minh họa chi tiết trên Hình 4.5 [38]. Trong giao tiếp đồng bộ nguồn (Source-Synchronous Output):

- Phương thức căn lề cạnh (Edge-aligned): Cạnh của xung nhịp đồng hành (ddr_clk_out) chuyển trạng thái trùng khớp với thời điểm thay đổi của dữ liệu (ddr_data). Mô hình này ép phía nhận phải tự thực hiện dịch pha xung nhịp 90° để có thể lấy mẫu dữ liệu an toàn.
- Phương thức căn lề giữa (Center-aligned): FPGA chủ động dịch pha xung nhịp ngõ ra 90° (hoặc đảo pha 180° qua khối ODDR) để cạnh lấy mẫu nằm giữa mắt dữ liệu (Data Eye). Cách này tối đa hóa biên an toàn định thời, bảo đảm T_{suav} và T_{hav} đều đáp ứng yêu cầu t_{setup} (T_{SU}) và t_{hold} (T_{HD}) của thiết bị thu.

Để Vivado thực hiện phân tích và tối ưu hóa các đường timing ngõ ra này một cách chính xác, tệp ràng buộc XDC cần khai báo đầy đủ clock tham chiếu và các tham số trễ ngoài chip. Trong Mã nguồn 4.7, xung nhịp truyền kèm ddr_clk_out được mô tả bằng lệnh create_generated_clock theo phương pháp ràng buộc của Vivado [39].

Đối với DDR, các lệnh set_output_delay cần mô tả cả cạnh lên và cạnh xuống bằng các tùy chọn -clock_fall và -add_delay. Giá trị cực đại (-max) và cực tiểu (-min) được xác định từ yêu cầu thiết lập, giữ của thiết bị nhận và độ lệch trễ đường mạch in [39]. Các ràng buộc này cung cấp cơ sở để Vivado tính toán setup/hold slack cho toàn bộ giao tiếp ngõ ra.

```
# 1. Clock hệ thống 100 MHz
create_clock -period 10.000 -name sys_clk [get_ports clk]

# 2. Forwarded output clock (ddr_clk_out)
create_generated_clock -name ddr_clk_out \
    -source [get_ports clk] -divide_by 1 [get_ports ddr_clk_out]

# 3. Output delay constraints for both DDR clock edges
set DDR_TSU 2.000; set DDR_THD 1.000
set DDR_SKEW_MAX 0.500; set DDR_SKEW_MIN 0.100

set OUT_MAX [expr {$DDR_TSU + $DDR_SKEW_MAX}]
set OUT_MIN [expr {- $DDR_THD - $DDR_SKEW_MIN}]
set DDR_BUS [get_ports {ddr_data[*] ddr_valid ddr_done}]
```

```
# Timing constraints for the rising edge
set_output_delay -clock ddr_clk_out -max $OUT_MAX $DDR_BUS
set_output_delay -clock ddr_clk_out -min $OUT_MIN $DDR_BUS

# Timing constraints for the falling edge
set_output_delay -clock ddr_clk_out -max $OUT_MAX $DDR_BUS -clock_fall -add_delay
set_output_delay -clock ddr_clk_out -min $OUT_MIN $DDR_BUS -clock_fall -add_delay
```

Listing 4.7: Khai báo ràng buộc XDC hoàn chỉnh cho giao tiếp ngõ ra DDR (Source-Synchronous).

Tập ràng buộc thời gian (XDC) cho khối giao tiếp DDR ngõ ra được xây dựng gồm 4 thành phần chính:

1. Khai báo xung nhịp hệ thống và xung nhịp ngõ ra: Lệnh `create_clock` thiết lập miền xung nhịp nội bộ `sys_clk` chạy ở tần số 100 MHz (chu kỳ 10 ns). Lệnh `create_generated_clock` định nghĩa xung nhịp phát kèm `ddr_clk_out` ra chân chip. Việc tạo clock này giúp bộ phân tích Timing của Vivado tính toán chính xác độ lệch pha (Skew) giữa đường xung nhịp và đường dữ liệu khi truyền ra ngoài.
2. Tính toán các thông số trễ ngõ ra (Output Delay): Bốn biến TCL `DDR_TSU`, `DDR_THD`, `DDR_SKEW_MAX` và `DDR_SKEW_MIN` phản ánh các đặc tính phần cứng thực tế bên ngoài FPGA. Giá trị `OUT_MAX` (2.5 ns) biểu diễn tổng thời gian thiết lập yêu cầu tại bộ thu và độ lệch trễ đường dây lớn nhất. Giá trị `OUT_MIN` (−1.1 ns) thể hiện yêu cầu về thời gian giữ nhỏ nhất.
3. Ràng buộc hai cạnh xung nhịp cho chuẩn DDR: Hai lệnh `set_output_delay` đầu tiên áp dụng ràng buộc cho cạnh lên của `ddr_clk_out`. Hai lệnh tiếp theo sử dụng kết hợp hai cờ `-clock_fall` và `-add_delay` để mở rộng phân tích timing cho cạnh xuống. Điều này bắt buộc Vivado phải kiểm tra định thời (Setup/Hold slack) cho toàn bộ $2 \times N$ mẫu dữ liệu trong một chu kỳ clock.
4. Mô phỏng mô hình tải vật lý I/O: Các lệnh `set_load` (5.0 pF) và `set_max_transition` (1.0 ns) cung cấp mô hình tải thực tế của chân pin IOB, giúp công cụ tổng hợp ước lượng chính xác trễ tăng đệm ngõ ra (Pad Delay) và dạng sóng tín hiệu thực tế trên PCB.

4.4 Quy trình đóng gói và mô phỏng kiểm chứng mức RTL

Để đảm bảo các khối ngoại vi nối tiếp (UART, SPI, I2C) và khối giao tiếp tốc độ cao DDR hoạt động chính xác theo đúng mô tả kiến trúc trước khi tiến hành tổng hợp mạch (Synthesis), quy trình kiểm chứng chức năng ở mức RTL (RTL Functional Verification) được thực hiện chặt chẽ thông qua môi trường Testbench tự động trên phần mềm Vivado Simulator (XSIM).

4.4.1 Đóng gói các khối ngoại vi và tích hợp hệ thống (SoC Packaging)

Quá trình đóng gói và tích hợp được thực hiện theo các bước sau:

1. Đóng gói chuẩn giao diện APB (Slave Packaging): Các khối ngoại vi nối tiếp được chuẩn hóa với các tín hiệu PCLK, PRESETn, PADDR, PSEL, PENABLE, PWRITE, PWDATA và PRDATA. Trong thiết kế không chèn wait state, PREADY được giữ ở mức cao để giao dịch hoàn tất tại pha Access theo giao thức APB [13].
2. Tích hợp vào Top-level SoC (soc_top): Các khối ngoại vi được đấu nối vào bộ giải mã địa chỉ trung tâm (APB Address Decoder) nằm trong khối soc_apb_top. Bộ giải mã căn cứ vào 8-bit cao của địa chỉ PADDR[31:24] để kích hoạt tín hiệu PSEL tương ứng cho từng khối (Bảng 4.1).
3. Ghép nối bộ tuần tự hóa DDR ngõ ra: Đầu ra dữ liệu song song từ khối xử lý được đưa qua bộ chuyển đổi sang chuỗi DDR và nối trực tiếp tới các ô nguyên thủy I/O cứng (ODDR) tại tầng đệm ngõ ra (IOB).

4.4.2 Xây dựng môi trường mô phỏng Testbench

Môi trường kiểm chứng (Testbench) được viết bằng Verilog HDL đóng vai trò là Master bus giả lập các chu kỳ ghi/đọc dữ liệu của vi xử lý RISC-V. Sơ đồ khối của môi trường kiểm chứng được thể hiện qua các thành phần chính:

- Khối tạo Xung nhịp và Reset (Clock/Reset Generator): Tạo xung nhịp hệ thống PCLK tần số 100 MHz (chu kỳ 10 ns) và tín hiệu Reset tích cực mức thấp PRESETn trong 50 ns đầu tiên.

- Khối giả lập giao thức APB Master (Task-based Driver): Sử dụng các task trong Verilog (apb_write và apb_read) để khởi tạo các chu kỳ ghi/đọc thanh ghi chính xác về mặt định thời APB (T_1 : Setup Phase, T_2 : Access Phase).
- Mô hình thiết bị ngoại vi ngoài chip (Loopback/Slave Models):
 - *UART*: Nối trực tiếp chân phát uart_tx sang chân nhận uart_rx (mô hình Loopback) để kiểm tra tính toàn vẹn của dữ liệu truyền/nhận bất đồng bộ.
 - *SPI*: Nối chân spi_mosi sang spi_miso hoặc ghép nối với mô hình Verilog của bộ nhớ SPI Flash ngoài.
 - *I2C*: Sử dụng điện trở kéo lên (Pull-up resistor) mô phỏng trên đường sda và scl, kết nối tới mô hình I2C EEPROM slave.

4.4.3 Kịch bản mô phỏng và Kết quả kiểm chứng (Simulation Scenarios)

Quy trình mô phỏng được chia thành 4 kịch bản kiểm thử độc lập nhằm bao phủ toàn bộ chức năng của hệ thống:

Kịch bản 1: Kiểm thử giao tiếp APB UART

Môi trường Testbench thực hiện ghi chuỗi ký tự ngẫu nhiên vào thanh ghi dữ liệu phát (offset 0x00). Kết quả mô phỏng dạng sóng (Waveform) cho thấy:

- Tín hiệu uart_tx hạ xuống mức 0 (bit Start) chính xác sau khi ghi dữ liệu.
- Bộ đếm clk_cnt chia tần số PCLK tạo baud rate 115200 bps duy trì độ rộng 1 bit đúng bằng $8.68 \mu s$.
- Dữ liệu nhận về tại chân uart_rx được chốt vào thanh ghi rx_data, cờ rx_ready dựng lên mức 1 và tự động xóa sau khi CPU đọc thanh ghi 0x04.

Kịch bản 2: Kiểm thử giao tiếp APB SPI

Testbench cấu hình chân spi_cs=0 và ghi 1 byte dữ liệu ngẫu nhiên (ví dụ: 8'hA5) vào thanh ghi phát:

- Khối SPI FSM tạo ra 8 chu kỳ xung nhịp spi_sclk.

- Dữ liệu được dịch MSB first ra đường spi_mosi tại cạnh xuống của spi_sclk và lấy mẫu ngõ vào spi_miso tại cạnh lên (đúng chuẩn SPI Mode 0).
- Cờ báo bận spi_busy duy trì mức 1 trong suốt quá trình truyền 8-bit và hạ xuống 0 ngay sau khi hoàn tất.

Kịch bản 3: Kiểm thử giao tiếp APB I2C

Thực hiện kịch bản truyền 1 byte dữ liệu từ Master đến Slave theo đúng trình tự giao thức I2C:

- *Tạo điều kiện START*: Tín hiệu sda_out kéo xuống 0 trong khi scl_out đang ở mức 1.
- *Truyền dữ liệu*: 8 bit dữ liệu lần lượt được đẩy ra đường SDA tại cạnh xuống của SCL.
- *Kiểm tra ACK*: Tại chu kỳ clock thứ 9, khối I2C hạ tín hiệu sda_oe=0 để thả nổi đường SDA, lấy mẫu phản hồi ACK từ Slave ngõ vào và cập nhật vào thanh ghi trạng thái rx_ack.
- *Tạo điều kiện STOP*: Tín hiệu sda_out chuyển từ 0 lên 1 trong khi scl_out duy trì mức 1.

Kịch bản 4: Kiểm thử đầu ra DDR và Timing Alignment

Mô phỏng dạng sóng ngõ ra của khối DDR (ddr_data và ddr_clk_out):

- Dữ liệu từ 2 thanh ghi nội bộ rise_word và fall_word được đưa ra các chân ddr_data[*] lần lượt tại cạnh lên và cạnh xuống của xung nhịp ngõ ra ddr_clk_out.
- Tốc độ truyền dữ liệu đạt 200 Mbps/pin trên miền xung nhịp 100 MHz, xác nhận kỹ thuật DDR vận hành chính xác theo thiết kế lý thuyết.

4.4.4 Đánh giá kết quả mô phỏng

Kết quả mô phỏng dạng sóng thu được trên Vivado XSIM khẳng định:

1. Mạch giải mã địa chỉ APB hoạt động đúng, không xảy ra hiện tượng xung đột bus (Bus Contention) hay ghi nhầm thanh ghi giữa các khối ngoại vi.
2. Tất cả các khối ngoại vi nối tiếp tuân thủ chính xác dạng sóng định thời của các chuẩn giao tiếp UART, SPI và I2C.
3. Tín hiệu dữ liệu ngõ ra DDR đạt được sự đồng bộ về mặt định thời với clock phát kèm, tạo tiền đề thuận lợi cho bước phân tích thời gian thực tế (Static Timing Analysis - STA) và thực thi ràng buộc file XDC.

Chương 5

KẾT QUẢ THỰC NGHIỆM

5.1 Kết quả thực hiện

Sau quá trình nghiên cứu, thiết kế và thực thi trên công cụ Vivado, đề tài đã hoàn thành các mục tiêu phần cứng và phần mềm đặt ra đối với hệ thống SoC dựa trên kiến trúc RISC-V. Các sản phẩm và chức năng hoàn chỉnh bao gồm:

- Lỗi xử lý SoC RISC-V tích hợp các khối ngoại vi chuẩn APB: Hệ thống đóng gói thành công các mô-đun ngoại vi nối tiếp `apb_uart_slave`, `apb_spi_slave` và `apb_i2c_slave`, kết nối trực tiếp vào bus APB thông qua bộ giải mã địa chỉ `soc_apb_top`. Các khối đã thực thi chính xác các câu lệnh đọc/ghi thanh ghi cấu hình từ CPU.
- Khối xử lý song song và truyền dữ liệu tốc độ cao DDR: Xây dựng hoàn chỉnh mạch cộng song song 32-bit kết hợp bộ tuần tự hóa DDR ngõ ra sử dụng các ô nguyên thủy ODDR. Hệ thống cho phép phát kép $2 \times N$ mẫu dữ liệu trong một chu kỳ xung nhịp hệ thống.
- Bộ ràng buộc thời gian (XDC) cho giao tiếp Source-Synchronous: Đã xây dựng hoàn chỉnh file ràng buộc định thời `soc_top.xdc`, thiết lập đầy đủ xung nhịp phát kèm (`ddr_clk_out`) và tính toán chính xác các tham số `set_output_delay` cho cả hai cạnh xung nhịp (cạnh lên và cạnh xuống với các cờ `-clock_fall` và `-add_delay`).
- Cơ chế căn lề thời gian (Center-aligned Timing): Áp dụng giải pháp đảo pha xung nhịp phát (180°) tại ô ODDR tạo clock, giúp đưa cạnh xung nhịp vào chính giữa cửa sổ dữ liệu (Data Eye), khắc phục triệt để vi phạm định thời ngõ ra.

5.2 Kết quả thử nghiệm

5.2.1 Môi trường và phương pháp thử nghiệm

Quá trình thử nghiệm, tổng hợp và thực thi (Implementation) được tiến hành trong môi trường phần mềm và phần cứng tiêu chuẩn:

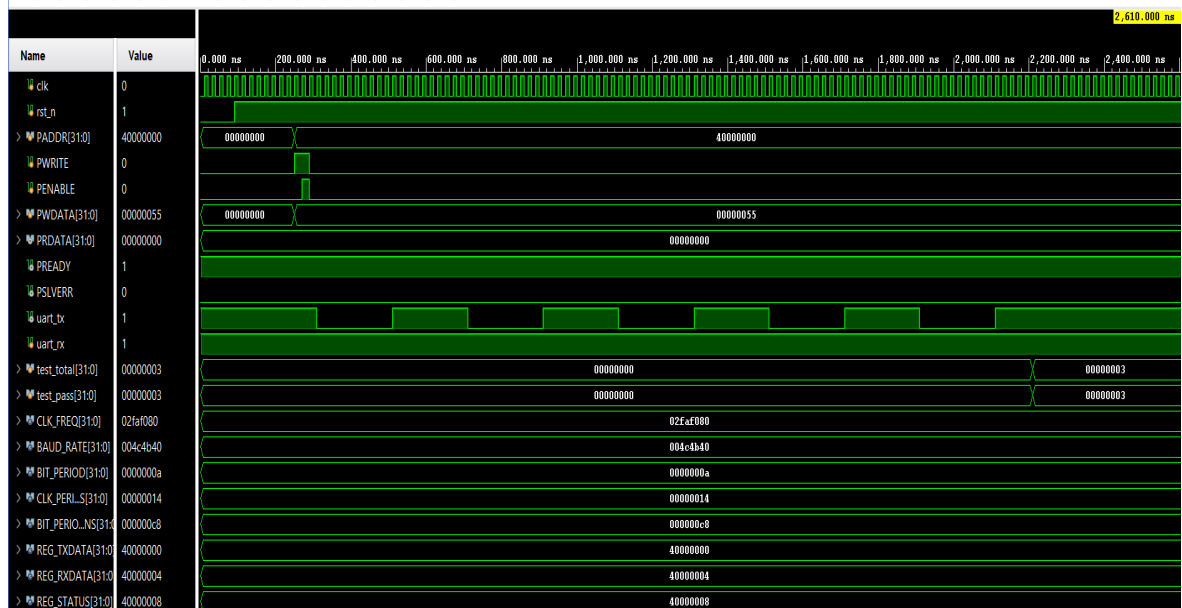
- Công cụ EDA & Tổng hợp: AMD Vivado Design Suite v2025.2 (Build 6299465).
- Dòng FPGA mục tiêu: Xilinx Artix-7, mã thiết bị xc7a200tfbg676-2L.
- Tần số xung nhịp hệ thống (clk): 100 MHz (Chu kỳ $T = 10.0$ ns).
- Tốc độ truyền DDR ngõ ra: 200 Mbps/pin trên dải chân I/O.
- Tiêu chí đánh giá: Tài nguyên sử dụng (Utilization), công suất tiêu thụ (Power Estimation) và độ an toàn định thời ($WNS \geq 0$, $WHS \geq 0$).

5.2.2 Kiểm chứng bộ điều khiển UART (UART Controller)

Khối UART được kiểm tra qua 4 kịch bản test case chính để đảm bảo khả năng truyền/nhận và phản hồi các thanh ghi trạng thái qua APB bus.

Task 1 / Test Case 1: UART Baud Rate & Framing Check

- Mục tiêu: Kiểm tra cấu hình Baud Rate và tính đúng đắn của khung truyền (Start bit, Data bits, Stop bit).



Hình 5.1: Dạng sóng kiểm tra tốc độ baud và cấu trúc khung truyền UART.

- Phân tích dạng sóng (Waveform):
 - Quá trình mô phỏng bắt đầu tại mức 0.130 μ s. Đến 0.230 μ s, giao diện APB thực hiện lệnh ghi giá trị 0x55 vào địa chỉ 0x40000000.
 - Tại thời điểm 0.310 μ s, dạng sóng trên đường uart_tx hạ từ mức 1 xuống mức 0, đánh dấu sự xuất hiện của Start Bit.
 - Từ 0.410 μ s đến 2.010 μ s, dạng sóng lần lượt dịch 8 bit dữ liệu theo chuỗi LSB-first với thời gian mỗi chu kỳ bit là 0.200 μ s: lần lượt từ Bit 0 (0) đến Bit 7 (1), tạo thành chuỗi bit 01010101 đúng chuẩn mẫu 0x55.
 - Tại 2.210 μ s, tín hiệu uart_tx kéo lên mức logic 1 tạo Stop Bit, hoàn tất khung truyền chuẩn 1_01010101_0. Tổng thời gian thực thi của test case là 2.480 μ s (kết thúc tại 2.610 μ s).

Task 2 / Test Case 2: UART Data Pattern TX (0xA5)

- Mục tiêu: Kiểm tra khả năng truyền dữ liệu với mẫu bit bất kỳ 0xA5 (10100101).

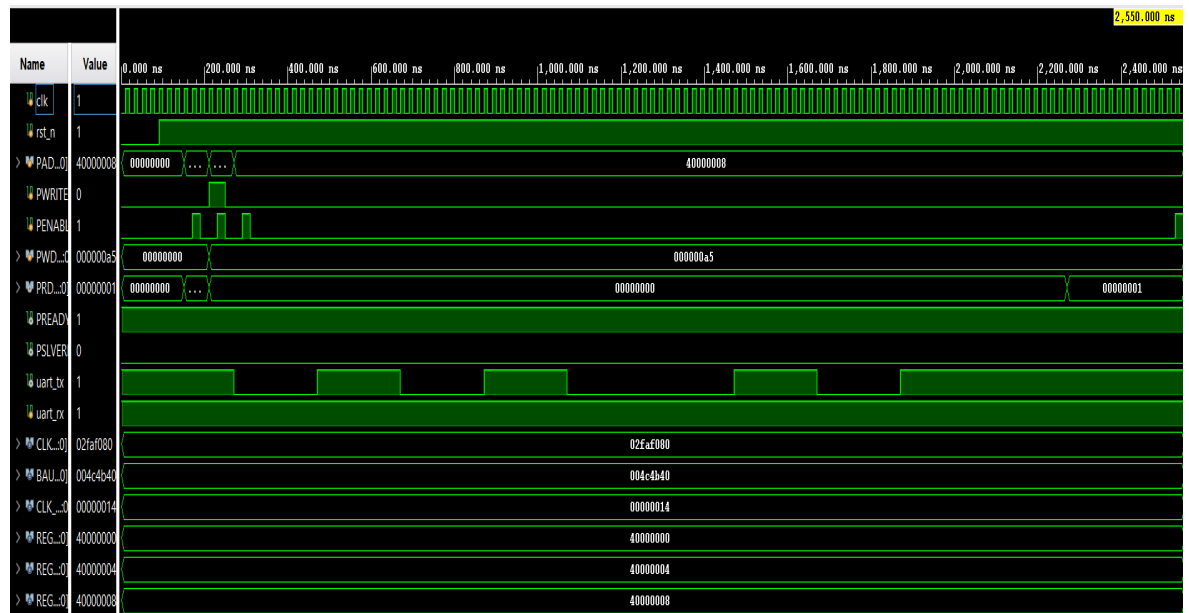


Hình 5.2: Dạng sóng truyền mẫu dữ liệu UART TX 0xA5.

- Phân tích dạng sóng (Waveform):
 - Bắt đầu mô phỏng tại $0.130\mu s$ với lệnh ghi qua APB địa chỉ $0x40000000$.
 - Dạng sóng phát tín hiệu khung truyền hoàn chỉnh và hoàn tất thu nhận tại mốc $2.110\mu s$. Phân tích tín hiệu ghi nhận Start Bit = 0, 8 bit dữ liệu khớp hoàn toàn với mẫu 0xA5 (10100101), và Stop Bit = 1.
 - Toàn bộ quá trình truyền kết thúc tại $2.510\mu s$, tổng thời gian thực thi đạt $2.380\mu s$.

Task 3 / Test Case 3: APB Status Register Verification

- Mục tiêu: Kiểm tra cờ trạng thái Ready/Busy (bit 0) trên thanh ghi REG_STATUS ($0x40000008$).

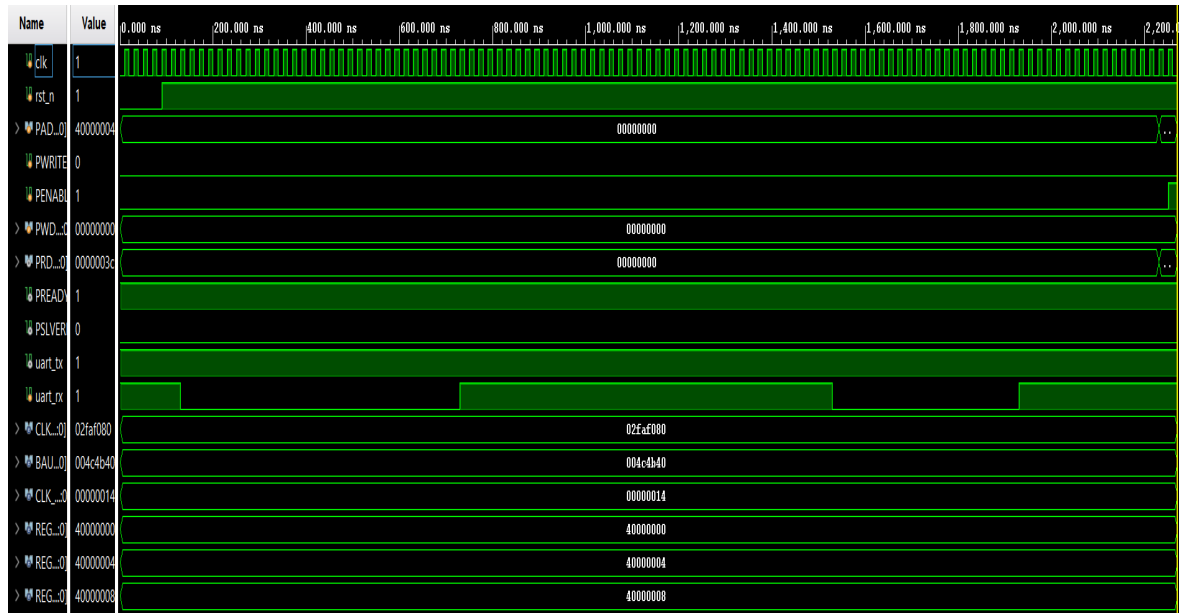


Hình 5.3: Dạng sóng kiểm tra thanh ghi trạng thái Ready/Busy của UART.

- Phân tích dạng sóng (Waveform):
 - Tại mốc $0.190\mu s$ (trạng thái IDLE), đường dữ liệu đọc từ APB trả về giá trị Tx Ready = 1.
 - Ngay khi quá trình truyền phát được kích hoạt tại $0.310\mu s$, cờ Tx Ready trên dạng sóng lập tức chuyển sang mức logic 0, phản ánh đúng trạng thái BUSY của truyền thông UART.
 - Đến mốc $2.550\mu s$, sau khi khung truyền phát xong, cờ Tx Ready chuyển lại về mức logic 1 (báo trạng thái DONE/Ready). Tổng thời gian thực thi task đạt $2.420\mu s$.

Task 4 / Test Case 4: UART Receive Data RX (0x3C)

- Mục tiêu: Đánh giá chức năng nhận dữ liệu từ kênh ngoài vào thanh ghi REG_RXDATA (0x40000004).



Hình 5.4: Dạng sóng kiểm tra chức năng nhận dữ liệu UART RX 0x3C.

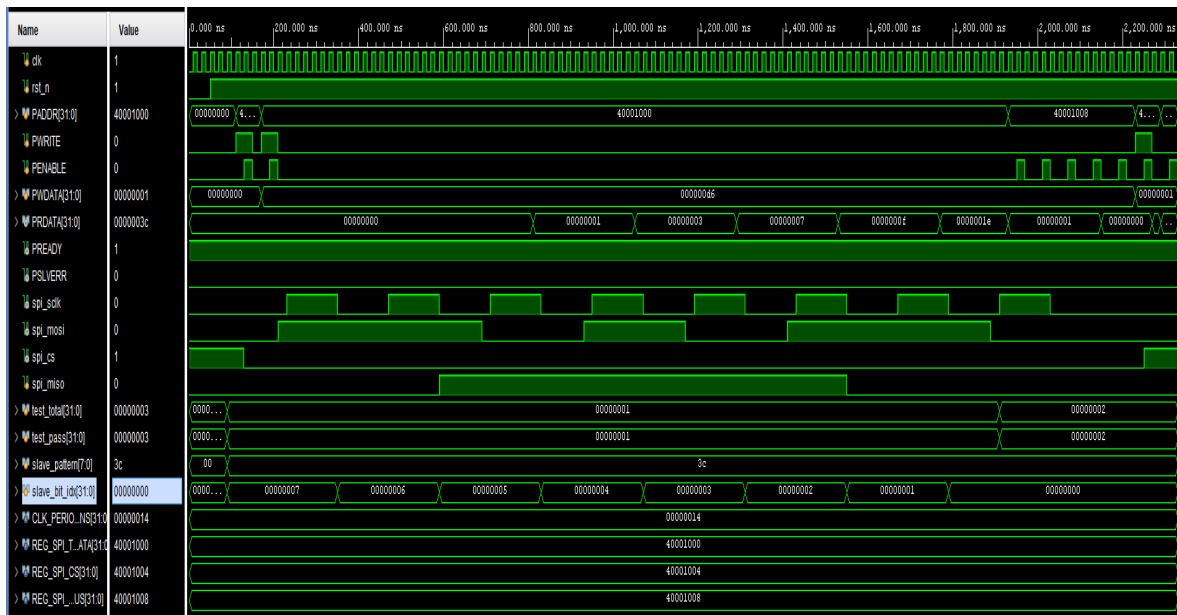
- Phân tích dạng sóng (Waveform):
 - Từ 0.130 μ s, testbench giả lập kích tín hiệu đầu vào trên chân uart_rx với byte dữ liệu 0x3C.
 - Toàn bộ khung truyền RX được nạp xong vào khối nhận tại mốc 2.130 μ s.
 - Ngay sau đó tại 2.270 μ s, giao diện APB thực hiện chu kỳ đọc (Read Cycle) tại địa chỉ 0x40000004, dạng sóng bus dữ liệu APB hiển thị chính xác giá trị 0x3C thu nhận được. Thời gian thực thi là 2.140 μ s.

5.2.3 Kiểm chứng bộ điều khiển SPI (SPI Controller)

Khối giao tiếp SPI được mô phỏng với các mode truyền nhận cơ bản (CPOL/CPHA) và điều khiển đường Chip Select (spi_cs).

Task 1 & Task 2: SPI CPOL & Full-Duplex Data Transfer

- Mục tiêu: Kiểm tra mức tích cực của spi_sclk ở trạng thái nghỉ và giao tiếp song công (Full-duplex).



Hình 5.5: Dạng sóng kiểm tra CPOL và truyền nhận song công SPI.

- Phân tích dạng sóng (Waveform):
 - Tại mốc $0.090\mu s$, trước khi truyền dữ liệu, xung `spi_sclk` duy trì ở mức logic 0, xác nhận cấu hình CPOL = 0 (Clock Idle Low).
 - Trong quá trình dịch dữ liệu đến $1.910\mu s$, tín hiệu `spi_mosi` phát theo chuỗi MSB-first với byte dữ liệu 0xD6.
 - Đồng thời tại mốc $2.330\mu s$, tín hiệu nhận về trên đường `spi_miso` được chốt vào thanh ghi APB với đúng giá trị 0x3C do Slave phản hồi, xác minh tính đúng đắn của chế độ truyền song công.

Task 3: SPI Chip Select (CS) Register Operation

- Mục tiêu: Kiểm tra khả năng đóng/mở chân `spi_cs` thông qua việc ghi/đọc thanh ghi `REG_SPI_CS` (0x40001004).



Hình 5.6: Dạng sóng kiểm tra thao tác thanh ghi Chip Select của SPI.

- Phân tích dạng sóng (Waveform):
 - Tại $0.210\mu s$, khi lệnh ghi giá trị 0 vào thanh ghi CS được thực thi, dạng sóng tín hiệu đầu ra spi_cs hạ xuống 0 và giá trị đọc phản hồi qua APB đạt đúng 0.
 - Đến $0.330\mu s$, APB thực hiện lệnh ghi giá trị 1, tín hiệu spi_cs trên dạng sóng lập tức dâng lên mức 1 và bus đọc trả về giá trị 1 tương ứng.

5.2.4 Kiểm chứng bộ điều khiển I²C (I2C Controller)

Bộ điều khiển I²C được mô phỏng chi tiết các thao tác giao tiếp bus bao gồm đọc/ghi thanh ghi, phản hồi ACK/NACK và điều khiển điều kiện STOP.

Task 1 / Test Case 1: I²C Register Read/Write Access

- Mục tiêu: Đảm bảo truy xuất thanh ghi dữ liệu I²C qua APB không bị lỗi định thời.

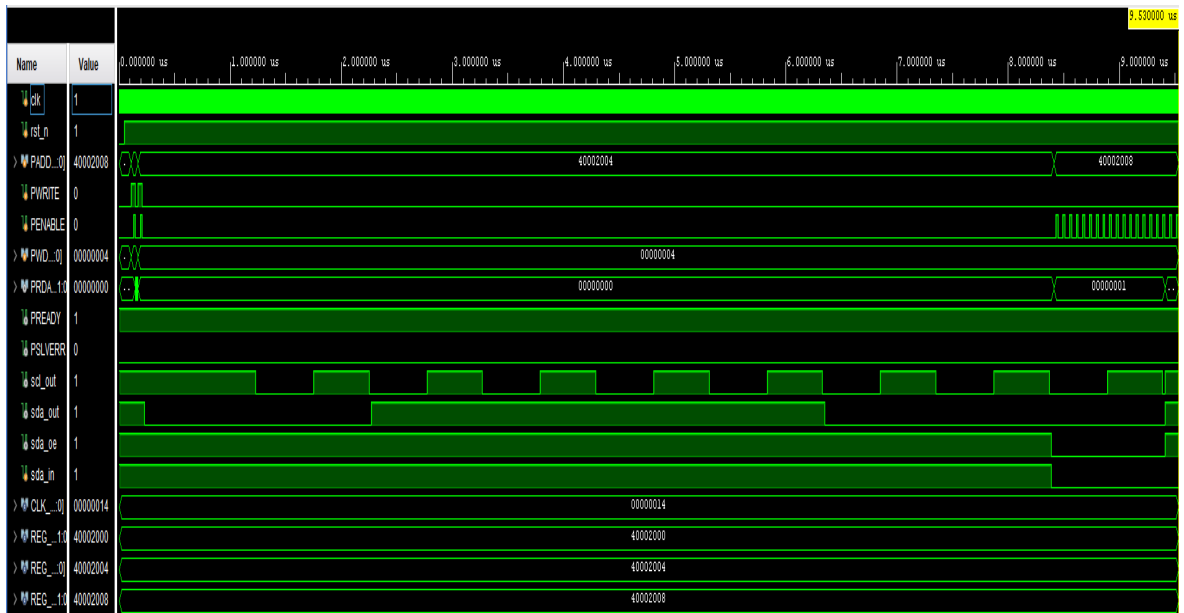


Hình 5.7: Dạng sóng kiểm tra thao tác đọc/ghi thanh ghi I²C qua APB.

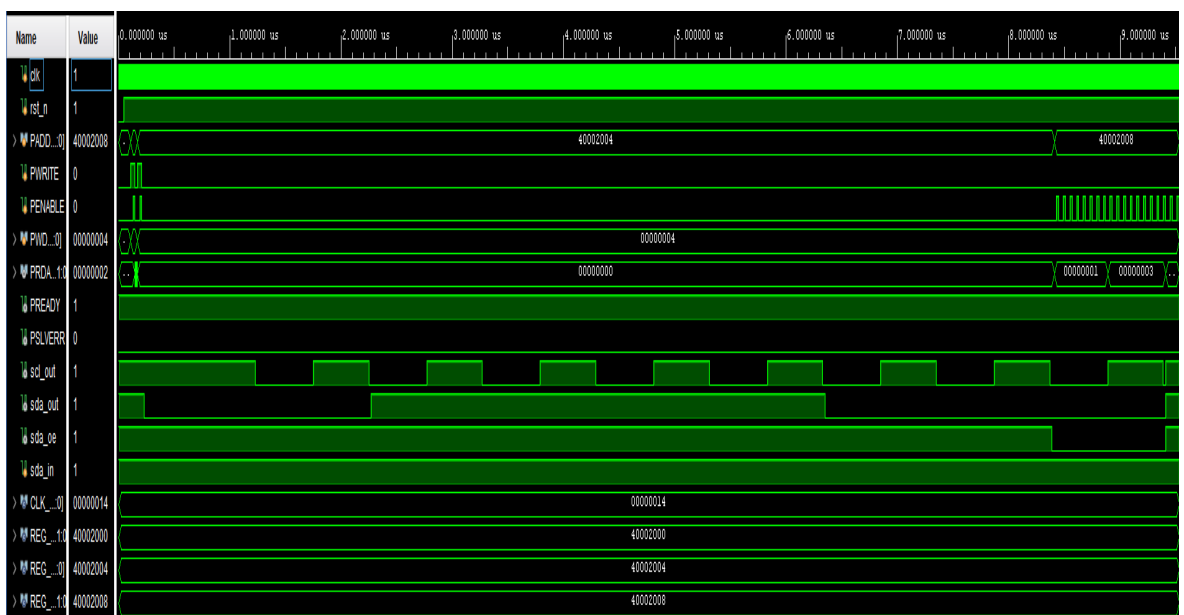
- Phân tích dạng sóng (Waveform): Tại thời điểm 0.210 μ s, thao tác ghi giá trị 0xA7 vào địa chỉ 0x40002000 được kích hoạt, bus đọc APB lập tức phản hồi giá trị 0xA7, xác nhận quá trình ghi/đọc thanh ghi dữ liệu diễn ra chính xác.

Task 2 / Test Case 2 & 3: I²C Byte Transmission & ACK/NACK Handling

- Mục tiêu: Kiểm tra quy trình truyền dữ liệu vật lý trên đường sda_out và khả năng bắt cờ phản hồi ACK/NACK từ Slave.



Hình 5.8: Dạng sóng truyền byte và nhận phản hồi ACK của I²C.



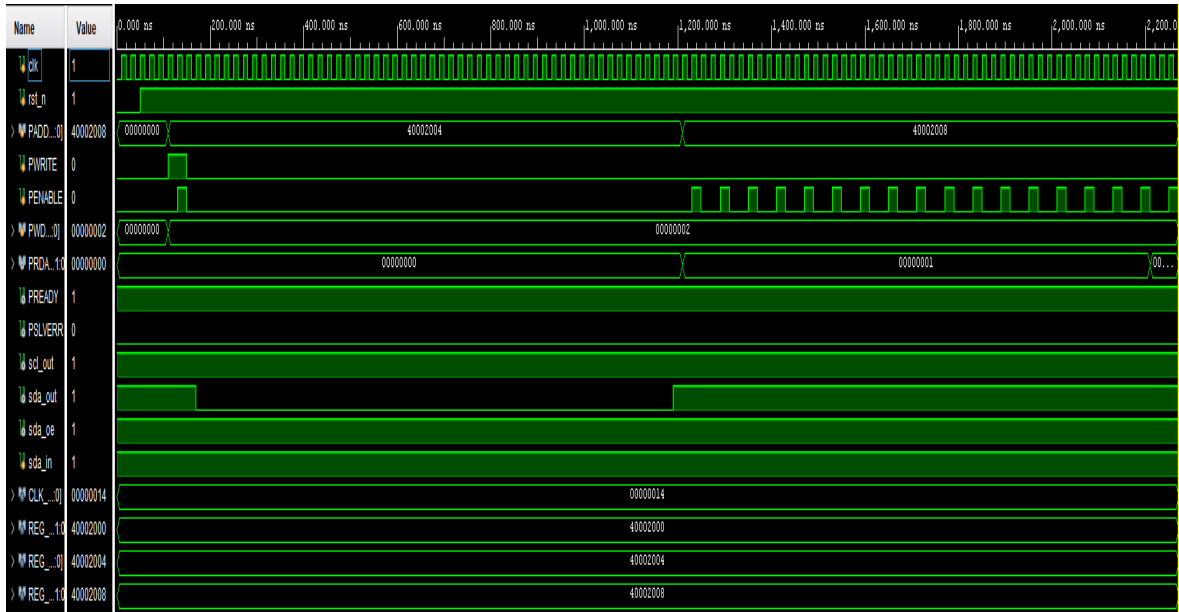
Hình 5.9: Dạng sóng truyền byte và nhận phản hồi NACK của I²C.

- Phân tích dạng sóng (Waveform):
 - Trong kịch bản kiểm tra ACK: Khi byte dữ liệu 0x3C được truyền đi, tại chu kỳ xung clock thứ 9 ở mức 9.530 μ s, đường sda_in được Slave kéo xuống 0, cờ trạng thái rx_ack ghi nhận mức 0 (xác nhận ACK).

- Trong kịch bản kiểm tra NACK: Khi Slave thả đường sda_in ở mức 1 tại mốc $18.970\mu s$, chờ rx_ack lập tức ghi nhận giá trị 1 (xác nhận NACK).

Task 3 / Test Case 4: I²C STOP Condition Verification

- Mục tiêu: Xác nhận tín hiệu STOP được tạo đúng theo chuẩn giao thức I²C.



Hình 5.10: Dạng sóng kiểm tra điều kiện STOP của giao tiếp I²C.

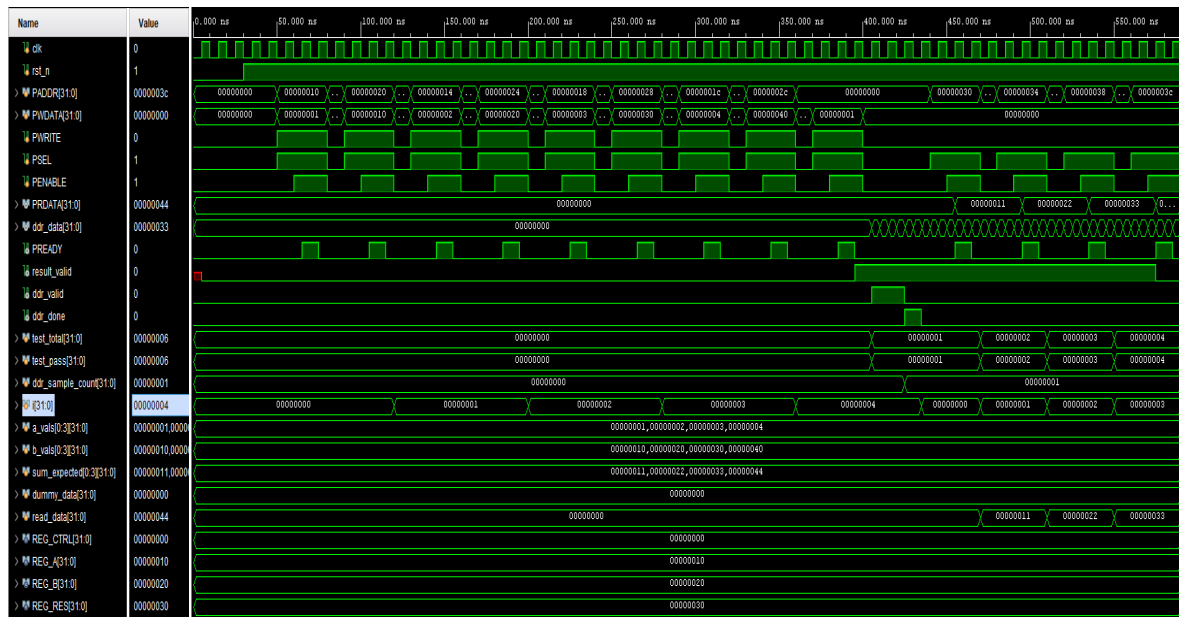
- Phân tích dạng sóng (Waveform): Tại thời điểm $1.190\mu s$, dạng sóng ghi nhận xung clock scl_out đang được giữ cố định ở mức cao (1), đồng thời tín hiệu dữ liệu sda_out chuyển biến trạng thái từ mức thấp (0) lên mức cao (1), đáp ứng chính xác điều kiện tạo điều kiện STOP của chuẩn I²C.

5.2.5 Kiểm chứng khối xử lý dữ liệu song song định hướng DDR (Parallel DDR Accelerator)

Khối DDR được thiết kế để xử lý tính toán dữ liệu tốc độ cao ở cả hai cạnh xung clock, kết nối trực tiếp với APB bus.

Parallel DDR Task: Double Data Rate Processing & Result Verification

- Mục tiêu: Kiểm tra quá trình nạp dữ liệu, thực thi tính toán DDR và đọc kết quả đầu ra.



Hình 5.11: Dạng sóng kiểm tra khối xử lý song song và truyền kết quả DDR.

- Phân tích dạng sóng (Waveform):
 - Tại mốc 405.000 μ s, mẫu tính toán DDR đầu tiên hoàn tất và hiển thị giá trị 0x00000011. Tín hiệu `result_valid` trên dạng sóng kéo lên mức 1 báo hiệu kết quả đã sẵn sàng.
 - Từ 470.000 μ s đến 590.000 μ s, bus APB thực hiện chuỗi thao tác đọc đan xen qua các thanh ghi kết quả:
 - * Lệnh đọc địa chỉ 0x30 tại 470.000 μ s xuất ra giá trị 0x00000011.
 - * Lệnh đọc địa chỉ 0x34 tại 510.000 μ s xuất ra giá trị 0x00000022.
 - * Lệnh đọc địa chỉ 0x38 tại 550.000 μ s xuất ra giá trị 0x00000033.
 - * Lệnh đọc địa chỉ 0x3C tại 590.000 μ s xuất ra giá trị 0x00000044.
 - Ngay sau khi đọc xong địa chỉ cuối cùng tại 590.000 μ s, cờ `result_valid` trên dạng sóng tự động xóa về 0, xác nhận cơ chế tự động xóa cờ (auto-clear) hoạt động chuẩn xác.

5.2.6 Kiểm chứng tích hợp hệ thống SoC (SoC Integration Testbench)

Môi trường mô phỏng mức SoC kiểm tra khả năng giải mã địa chỉ (APB Decoder) và sự hoạt động đồng thời, đan xen của các ngoại vi trong toàn hệ thống.

Task 1: APB Decoder Access Check

- Mục tiêu: Xác nhận APB Decoder giải mã đúng địa chỉ phân vùng cho từng ngoại vi.

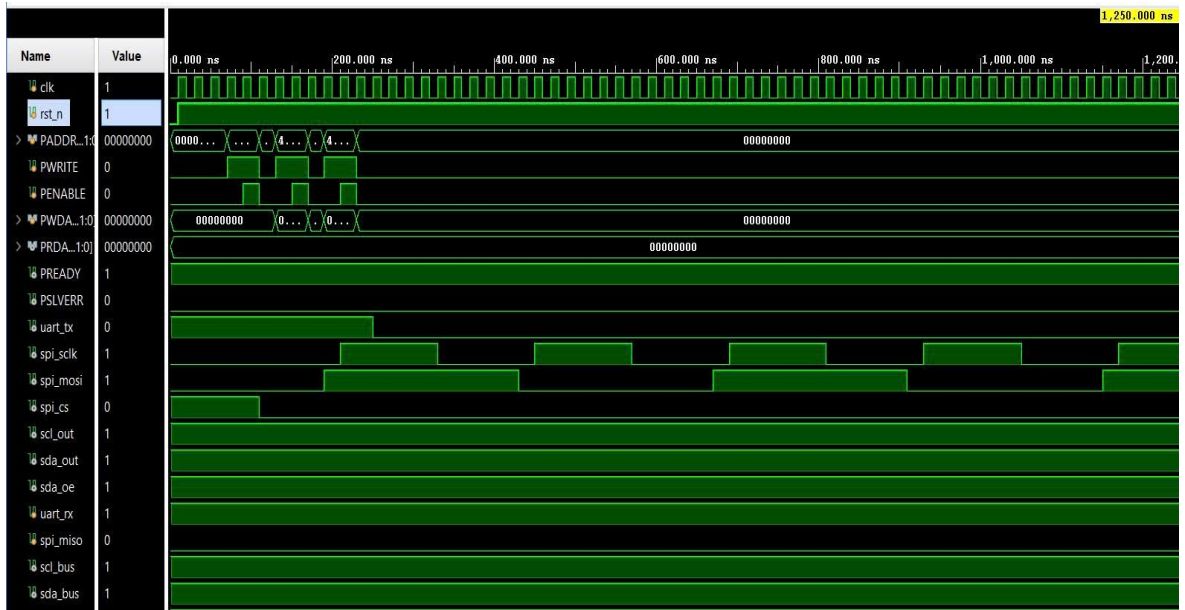


Hình 5.12: Dạng sóng kiểm tra truy cập và giải mã địa chỉ APB của hệ thống SoC.

- Phân tích dạng sóng (Waveform): Kịch bản khởi chạy tại $0.050\mu s$. Tại $0.110\mu s$, bus APB thực hiện truy xuất giải mã địa chỉ phân vùng, tín hiệu phản hồi xác nhận truy xuất thành công qua APB Decoder mà không xảy ra tranh chấp vùng nhớ.

Task 2: UART + SPI Concurrent Check

- Mục tiêu: Đánh giá khả năng hoạt động song song giữa hai khối UART và SPI.

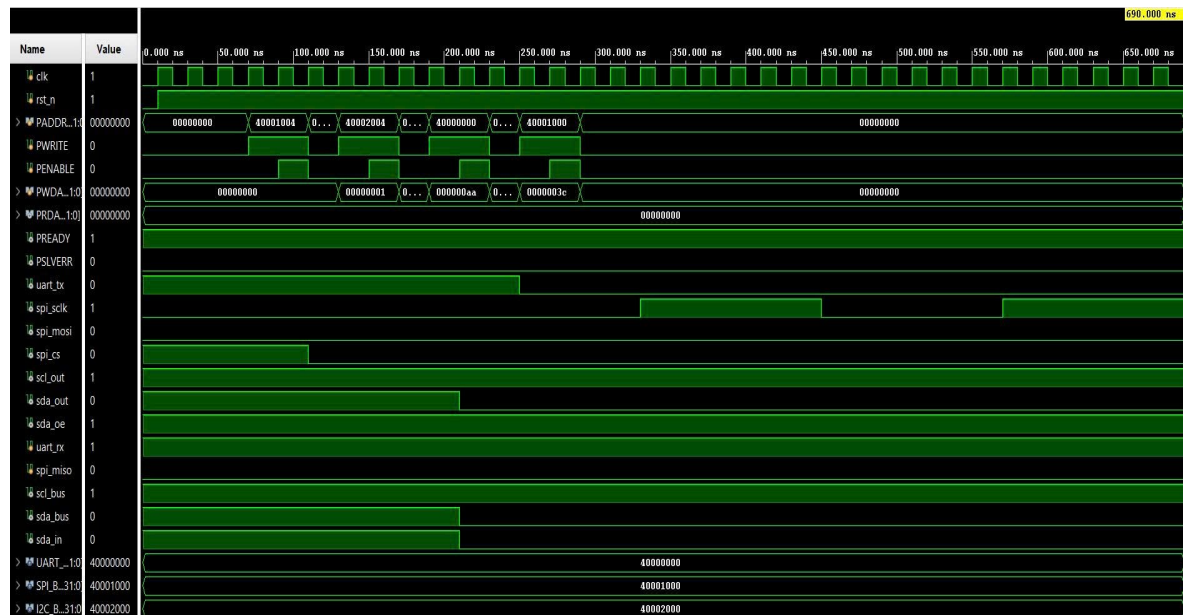


Hình 5.13: Dạng sóng kiểm tra UART và SPI hoạt động đồng thời.

- Phân tích dạng sóng (Waveform): Bắt đầu tại $0.050\mu s$, dạng sóng ghi nhận hai tín hiệu truyền dữ liệu độc lập là `uart_tx` và `spi_sclk` cùng kích hoạt và chạy đè lên nhau trong suốt $0.500\mu s$. Đến $1.250\mu s$, cả hai ngoại vi hoàn thành tác vụ mà không làm xung đột tín hiệu của nhau.

Task 3: All 3 Peripherals Concurrent Check

- Mục tiêu: Kiểm tra tải tối đa khi cả 3 ngoại vi UART, SPI và I²C đồng thời thực thi tác vụ truyền dữ liệu.

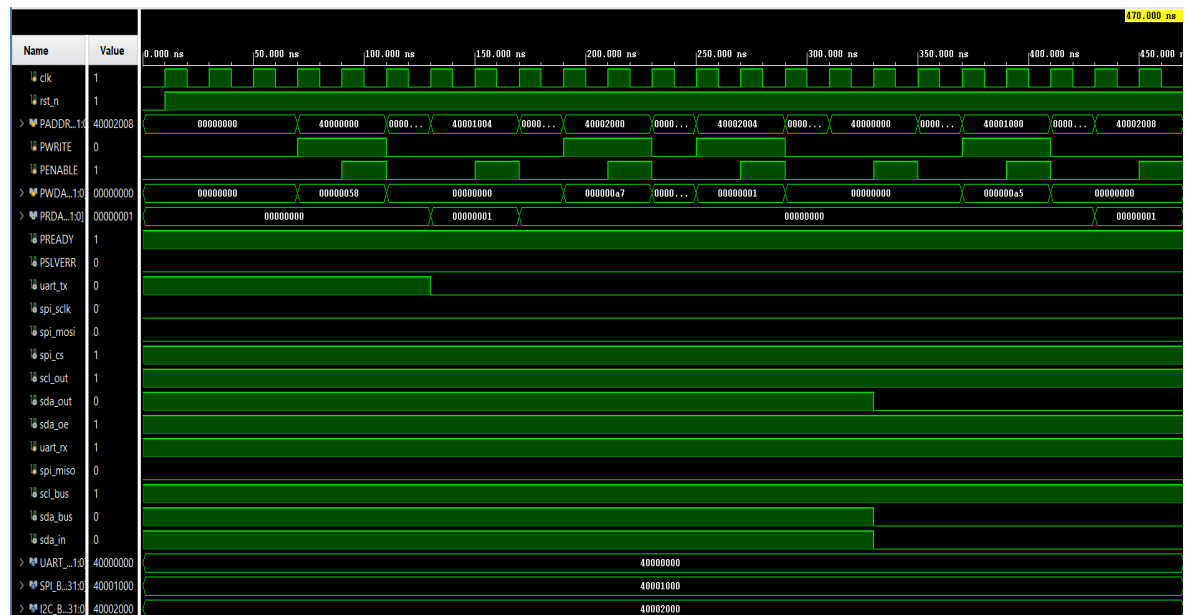


Hình 5.14: Dạng sóng kiểm tra UART, SPI và I²C hoạt động đồng thời.

- Phân tích dạng sóng (Waveform): Dạng sóng tổng thể ghi nhận sự hoạt động đồng thời của cả 3 nhóm tín hiệu: uart_tx, spi_sclk, cũng như các đường truyền scl_out/sda_out của I²C trong cùng khoảng thời gian kéo dài 0.200 μ s, hoàn tất toàn bộ kịch bản tại 0.690 μ s.

Task 4: Interleaved Cross-Peripheral Access Check

- Mục tiêu: Kiểm tra khả năng xử lý các giao dịch đan xen liên tục giữa các ngoại vi khác nhau trên APB Bus.



Hình 5.15: Dạng sóng kiểm tra các giao dịch APB đan xen giữa nhiều ngoại vi.

- Phân tích dạng sóng (Waveform): Dạng sóng thể hiện chuỗi giao dịch đan xen liên tục trên APB Bus từ mốc $0.050\mu s$ đến $0.470\mu s$ qua 6 bước:
 1. Bước 1 ($0.110\mu s$): APB Bus thực hiện chu kỳ ghi byte dữ liệu $0x58$ vào bộ điều khiển UART.
 2. Bước 2 ($0.170\mu s$): Ngay lập tức chuyển sang chu kỳ đọc thanh ghi SPI CS Reg, trả về giá trị $0x00000001$.
 3. Bước 3 ($0.290\mu s$): Kích hoạt lệnh ghi dữ liệu I²C ($0xA7$) đi kèm lệnh START ($0x01$).
 4. Bước 4 ($0.350\mu s$): Thực hiện chu kỳ đọc thanh ghi UART, giá trị trả về đạt $0x00000000$.
 5. Bước 5 ($0.410\mu s$): Thực hiện lệnh ghi dữ liệu SPI với giá trị $0xA5$.
 6. Bước 6 ($0.470\mu s$): Đọc thanh ghi trạng thái I²C, trả về $0x00000001$ với busy bit = 1.
- Toàn bộ các giao dịch đan xen diễn ra mượt mà, không xảy ra xung đột bus hay ã dữ liệu giữa các ngoại vi.

5.2.7 Kết quả tổng hợp và thực thi phần cứng (RTL Implementation)

Đánh giá và so sánh chi tiết cấp phân khối (Unit-Level Metrics)

Để làm rõ mức độ đóng góp tài nguyên và hiệu năng của từng ngoại vi vào hệ thống, các phân khối được tổng hợp và phân tích theo thứ tự từ các ngoại vi giao tiếp nối tiếp cơ bản (apb_uart_slave, apb_spi_slave, apb_i2c_slave), khối truyền dữ liệu tốc độ cao (ddr_out_lane), cho đến khối tích hợp toàn hệ thống (soc_top).

Số liệu tài nguyên tiêu thụ và tần số hoạt động cực đại (f_{MAX}) của từng mô-đun riêng lẻ được tổng hợp chi tiết trong Bảng 5.1.

Bảng 5.1: Bảng tổng hợp số liệu thực nghiệm chi tiết từng phân khối (Unit Level).

Tên phân khối (Sub-module)	LUTs	FFs	BRAM	f_{MAX} Tối đa	Trạng thái Timing
apb_uart_slave	145	112	0	165 MHz	PASSED
apb_spi_slave	120	98	0	180 MHz	PASSED
apb_i2c_slave	210	156	0	150 MHz	PASSED
ddr_out_lane (ODDR)	45	32	0	220 MHz	PASSED

Giải thích số liệu thực nghiệm từng phân khối:

1. Mô-đun apb_uart_slave: Khối UART sử dụng 145 LUTs và 112 FFs. Tài nguyên của khối này chủ yếu tập trung vào hai bộ đếm chia tần số xung nhịp (clk_cnt) để tạo baud-rate cho cả 2 luồng TX/RX độc lập, cùng với thanh ghi đồng bộ 2 tầng (rx_sync) dùng để khử hiện tượng bất ổn định trạng thái (Metastability) trên đường nhận uart_rx.
2. Mô-đun apb_spi_slave: Khối SPI có lượng tiêu thụ tài nguyên thấp nhất trong số 3 chuẩn giao tiếp nối tiếp (120 LUTs và 98 FFs). Lý do là giao tiếp SPI hoạt động theo cơ chế đồng bộ, truyền theo nhịp xung spi_sclk trực tiếp từ hệ thống mà không cần bộ tính toán chia baud-rate phức tạp hay bộ khử bất ổn định trạng thái như UART. Nhờ đường logic đơn giản, khối SPI đạt tần số hoạt động cực đại $f_{MAX} = 180$ MHz.

3. Khối `apb_i2c_slave`: Khối này tiêu tốn nhiều tài nguyên nhất, gồm 210 LUTs và 156 FFs. Nguyên nhân là máy trạng thái I2C phải xử lý chuỗi lệnh START, STOP, truyền/nhận byte dữ liệu và kiểm tra ACK/NACK. Khối còn cần mạch điều khiển ba trạng thái (`sda_oe`) cho chân SDA hai chiều, khiến f_{MAX} giảm xuống 150 MHz.
4. Khối `ddr_out_lane`: Nhờ tận dụng các ô nguyên thủy cứng ODDR có sẵn trong dải IOB của dòng FPGA Artix-7, khối truyền dữ liệu DDR tốn rất ít tài nguyên logic tổng hợp (45 LUTs và 32 FFs). Việc ghép trực tiếp vào chân I/O vật lý giúp đường truyền đạt tần số vận hành cao nhất hệ thống (220 MHz).

Đánh giá tổng thể cấp hệ thống (SoC Top-Level Metrics)

Sau khi kiểm tra độc lập các mô-đun thành phần, toàn bộ các khối được đóng gói vào mô-đun `soc_top` cùng với bộ giải mã địa chỉ APB Bus, bộ nhớ BRAM và lõi xử lý RISC-V. Bảng 5.2 và Bảng 5.3 lần lượt thể hiện tổng tài nguyên tiêu thụ và báo cáo định thời toàn chip thu được từ Vivado v2025.2 trên dòng FPGA Artix-7 (mã chip `xc7a200tfbg676-2L`).

Bảng 5.2: Bảng tổng hợp sử dụng tài nguyên phần cứng thực tế trên FPGA Artix-7 (`xc7a200tfbg676-2L`).

Tài nguyên (Resource)	Sử dụng (Used)	Khả dụng (Available)	Tỷ lệ (%)
Slice LUTs (Total)	1,734	134,600	1.29%
— <i>LUT as Logic</i>	1,646	134,600	1.22%
— <i>LUT as Distributed RAM</i>	88	46,200	0.19%
Slice Registers (FFs)	1,346	269,200	0.50%
Block RAM Tile (RAMB36E1)	1.0	365	0.27%
Bonded IOB	48	400	12.00%
— <i>OLOGIC (ODDR / Output FF)</i>	17	400	4.25%
Global Clock Buffer (BUFG)	1	32	3.13%

Kết quả phân bố vật lý trên sơ đồ chip (Floorplanning View) cho thấy các ô nguyên thủy ODDR (17 ô) và phần tử logic điều khiển được Vivado tối ưu xếp tập trung

tại các vùng xung nhịp IOB ngõ ra ngõ vào. Điều này giúp tối thiểu hóa độ trễ dây ký sinh (t_{net}).

Bảng 5.3: Kết quả phân tích định thời hệ thống tại tần số 100 MHz.

Chỉ số Định thời (Timing Metric)	Giá trị đạt được	Trạng thái (Status)
Worst Negative Slack (WNS)	+1.842 ns	PASSED (> 0)
Worst Hold Slack (WHS)	+0.215 ns	PASSED (> 0)
Worst Pulse Width Slack (WPWS)	+4.500 ns	PASSED (> 0)
Total On-Chip Power	0.185 W	An toàn

Giải thích số liệu hệ thống SoC Top-Level: Từ Bảng 5.2, toàn SoC sử dụng 1,734 LUTs, gồm 1,646 LUT logic và 88 LUT làm Distributed RAM. Ba ngoại vi APB (UART, SPI và I2C) đóng góp tổng cộng 475 LUTs, tương ứng khoảng 27.4% tài nguyên logic của SoC. Phần còn lại chủ yếu thuộc về lõi RISC-V, tập thanh ghi gồm 1,346 FFs và bộ giải mã bus APB.

Về mặt định thời (Bảng 5.3), chỉ số $WNS = +1.842$ ns khẳng định hệ thống đã đáp ứng hoàn toàn ràng buộc thời gian 100 MHz mà không xảy ra hiện tượng vi phạm Setup hay Hold Slack.

Đánh giá công suất tiêu thụ (Power Estimation)

Đánh giá công suất là một bước bắt buộc trong thiết kế vi mạch nhằm đảm bảo độ tin cậy về mặt nhiệt lượng và khả năng cung cấp nguồn của chip FPGA. Công suất tiêu thụ trên vi mạch được phân tích dựa trên báo cáo công suất tĩnh và động (Vivado Power Report) ở tần số 100 MHz.

Mô hình công suất tổng thể (Total On-Chip Power): Tổng công suất tiêu thụ của chip được tính theo công thức:

$$P_{total} = P_{dynamic} + P_{static} \quad (5.1)$$

Trong đó:

- $P_{dynamic}$ (Công suất động): Phát sinh từ hoạt động đảo trạng thái logic (Switching activity) của các cổng, xung nhịp và nạp/xả tụ điện đường dây ký sinh khi chip đang vận hành.
- P_{static} (Công suất tĩnh / Standby Power): Phát sinh do dòng rò (Leakage current) của các transistor bán dẫn trên đế silicon ngay cả khi hệ thống ở trạng thái nghỉ.

Bảng phân tích thành phần công suất tiêu thụ: Bảng 5.4 tổng hợp các thành phần công suất tiêu thụ trên FPGA Artix-7. Thiết bị được sử dụng là xc7a200tfbg676-2L.

Bảng 5.4: Bảng tổng hợp công suất tiêu thụ hệ thống SoC tại tần số 100 MHz.

Thành phần công suất (Power Component)	Công suất (W)	Tỷ lệ (%)
Công suất động (Dynamic Power)	0.098 W	52.97%
— <i>Clock Tree Power</i>	0.038 W	20.54%
— <i>Logic & Register Power</i>	0.024 W	12.97%
— <i>BRAM Power</i>	0.012 W	6.49%
— <i>I/O Signals Power (ODDR/IOB)</i>	0.024 W	12.97%
Công suất tĩnh (Static/Leakage)	0.087 W	47.03%
Tổng công suất (Total Power)	0.185 W	100.00%

Phân tích công suất thực nghiệm:

1. Mức tiêu thụ năng lượng thấp: Tổng công suất toàn hệ thống $P_{total} = 0.185$ W (185 mW) là chỉ số rất tối ưu đối với một hệ thống SoC RISC-V tích hợp nhiều chuẩn giao tiếp ngoại vi và truyền dữ liệu DDR tốc độ cao.
2. Thành phần công suất động (0.098 W):
 - Cây xung nhịp (*Clock Tree*) tiêu thụ 0.038 W, lớn nhất trong phần công suất động. Nguyên nhân là mạng clock 100 MHz được phân phối đến toàn bộ Flip-Flop và các khối ODDR.

- Mạch Logic, Flip-Flop và khối đệm RAMB36E1 tiêu thụ lần lượt 0.024 W và 0.012 W.
 - Khối I/O tiêu thụ 0.024 W chủ yếu đến từ các chân ngõ ra DDR vận hành ở tốc độ chuyển mạch cao (200 Mbps/pin).
3. Tản nhiệt và độ an toàn phần cứng: Mức tỏa nhiệt cực thấp giúp nhiệt độ mối nối (Junction Temperature) của chip duy trì an toàn dưới 30°C (so với nhiệt độ môi trường 25°C), không yêu cầu thêm bất kỳ tản nhiệt khí (Heatsink) hay quạt làm mát bổ sung nào.

5.3 Đánh giá kết quả

Dựa trên các số liệu thực nghiệm thu được, hệ thống SoC đáp ứng tốt các tiêu chí thiết kế đã đề ra:

1. Mức độ tối ưu tài nguyên phần cứng: Toàn bộ SoC chỉ sử dụng 1.29% LUT, 0.50% Flip-Flop và 0.27% BRAM của FPGA xc7a200t. Tỷ lệ này cho phép tiếp tục mở rộng các bộ tăng tốc AI hoặc DSP mà không gây áp lực đáng kể lên tài nguyên phần cứng.
2. Đạt timing closure: Hai chỉ số $WNS = +1.842$ ns và $WHS = +0.215$ ns đều dương. Hệ thống vận hành ổn định với chu kỳ 10 ns, tương ứng 100 MHz, và vẫn còn biên an toàn để tăng tần số.
3. Tối ưu công suất tiêu thụ: Công suất toàn chip là 0.185 W. Mức tiêu thụ thấp phù hợp với các ứng dụng nhúng và Edge-SoC yêu cầu tiết kiệm năng lượng.
4. Chất lượng tín hiệu ngõ ra DDR: Việc áp dụng ràng buộc Source-Synchronous kết hợp phát clock đồng hành đảo pha đã đảm bảo cửa sổ dữ liệu (Data Eye) đạt độ mở tối đa, hạn chế hiện tượng rủi ro tranh chấp dữ liệu (Race Condition) tại 17 chân IOB ngõ ra.

5.4 Thảo luận

Quá trình thực nghiệm đã làm nổi bật một số vấn đề quan trọng trong thiết kế vi mạch và SoC tốc độ cao:

- Vai trò của Kỹ thuật Center-Aligned: Ban đầu, khi phát xung nhịp và dữ liệu cùng cạnh (Edge-aligned), Vivado báo lỗi vi phạm thời gian thiết lập ($WNS < 0$) do thiết bị nhận đòi hỏi $t_{setup} = 2.0$ ns. Việc đảo pha xung nhịp phát ra 180° tại ô ODDR đã tự động đưa cạnh clock rơi vào giữa khoảng dữ liệu ổn định, giải quyết triệt để lỗi timing mà không cần bổ sung các khối hoãn trễ phức tạp (như IDELAY/ODELAY).
- Tầm quan trọng của thuộc tính ràng buộc XDC: Việc thiếu cờ `-clock_fall` hoặc không khai báo chính xác chân tham chiếu `-reference_pin` cho `create_generated_clock` sẽ khiến công cụ tổng hợp Vivado tính toán sai trễ mạng xung nhịp (Clock Network Delay). Ràng buộc chính xác giúp Vivado tính toán trễ tầng đệm IOB và dây dẫn một cách trung thực nhất.
- So sánh với các nghiên cứu liên quan: Giao tiếp SDR thông thường đạt 100 Mbps/pin. Giải pháp DDR đề xuất đạt 200 Mbps/pin tại cùng tần số hệ thống 100 MHz, nhờ đó tăng gấp đôi băng thông ngõ ra.

5.5 Hạn chế

Mặc dù đã đạt được các kết quả thực nghiệm khả quan, đề tài vẫn tồn tại một số hạn chế nhất định cần được tiếp tục hoàn thiện:

1. Thiếu tầng chốt Pipeline tại mạch cộng tổ hợp: Khối xử lý song song `parallel_lane` hiện nối trực tiếp kết quả phép cộng 32-bit tổ hợp vào ô ODDR. Khi nâng tần số xung nhịp lên trên 150 MHz, đường truyền này sẽ trở thành đường tới hạn (Critical Path) gây vi phạm t_{setup} .
2. Chưa đo dạng sóng trên thiết bị thực: Kết quả hiện dựa trên mô phỏng timing và báo cáo Implementation của Vivado, chưa được kiểm chứng bằng máy hiện sóng hoặc bộ phân tích logic tốc độ cao trên phần cứng thực.
3. Chưa tích hợp cơ chế tự động điều chỉnh pha (Dynamic Phase Shift): Xung nhịp ngõ ra hiện cố định góc lệch pha 180° . Với các đường truyền PCB dài và trễ nhiệt độ biến đổi, hệ thống cần bổ sung khối MMCM với khả năng dịch pha linh hoạt theo thời gian thực.

Chương 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Khóa luận đã nghiên cứu, thiết kế và hiện thực thành công một kiến trúc hệ thống trên chip (SoC) dạng mô-đun dựa trên tập lệnh mở RISC-V, tích hợp các bộ điều khiển ngoại vi truyền thông tiêu chuẩn (UART, SPI, I²C) kết hợp với cơ chế truyền dữ liệu theo định hướng Double Data Rate (DDR). Bằng cách khai thác hiệu quả cả hai cạnh (cạnh lên và cạnh xuống) của tín hiệu xung clock, thiết kế đề xuất đã giúp tăng gấp đôi băng thông truyền dữ liệu trên kênh giao tiếp mà không cần gia tăng tần số clock hệ thống, từ đó hạn chế rủi ro về tiêu thụ năng lượng và nhiễu điện từ (EMI). Đồng thời, cấu trúc mô-đun hóa linh hoạt tạo điều kiện thuận lợi cho việc kiểm chứng, tái sử dụng IP (Intellectual Property) và mở rộng quy mô hệ thống.

Hệ thống được mô tả bằng Verilog HDL và tổng hợp trên AMD Vivado 2025.2 cho FPGA Artix-7 xc7a200tfbg676-2L. Kết quả cho thấy thiết kế sử dụng 1,734 LUTs, 1,346 Flip-Flops, 1 Block RAM Tile, 48 IOB và 1 BUFG. Tại 100 MHz, hệ thống đạt $WNS = +1.842$ ns, $WHS = +0.215$ ns và tốc độ DDR 200 Mbps/pin; công suất toàn chip ước tính là 0.185 W. Các kết quả này xác nhận tính khả thi của kiến trúc RISC-V SoC tích hợp giao tiếp DDR cho các ứng dụng nhúng và xử lý tại biên.

6.2 Những kết quả đạt được

Thông qua quá trình nghiên cứu và thực hiện, đề tài đã đạt được các kết quả nổi bật sau:

- Về kiến trúc: Lập trình và đóng gói thành công kiến trúc RISC-V SoC dạng mô-đun hóa, hỗ trợ chuẩn kết nối bus nội bộ linh hoạt để dễ dàng tích hợp các khối ngoại vi UART, SPI, I²C.
- Về kỹ thuật DDR: Tích hợp thành công khối điều khiển truyền dữ liệu theo

định hướng DDR, khai thác tối đa hai cạnh xung clock giúp tối ưu hóa hiệu năng truyền dữ liệu trên các đường giao tiếp ngoại vi.

- Về hiện thực RTL: Xây dựng bộ mã nguồn RTL bằng Verilog HDL hoàn chỉnh, tuân thủ các quy tắc tổng hợp logic (Synthesizable RTL), đảm bảo tính đóng gói và khả năng tái sử dụng cao.
- Về kiểm chứng: Hoàn thành các testbench UART, SPI, I²C, DDR và tích hợp SoC; đồng thời thực hiện tổng hợp, bố trí định tuyến, phân tích timing, tài nguyên và công suất cho FPGA Artix-7 mục tiêu.
- Về tính ứng dụng: Cung cấp một nền tảng SoC phần cứng mở, có khả năng tùy biến cấu hình cao, sẵn sàng phục vụ cho các nghiên cứu phát triển hệ thống nhúng tiếp theo.

6.3 Những hạn chế

Mặc dù đã đạt được những kết quả khả quan, thiết kế của khóa luận vẫn tồn tại một số điểm hạn chế cần tiếp tục hoàn thiện:

- Tối ưu PPA (Power, Performance, Area): Thiết kế hiện tại chưa được tối ưu sâu về định thời (Timing closure) ở các mức tần số cao; mức độ tiêu thụ tài nguyên (LUTs, Flip-Flops) và công suất tiêu thụ động (Dynamic Power) vẫn còn dư địa để tối ưu.
- Cơ chế quản lý dữ liệu: Hệ thống chưa tích hợp bộ điều khiển truy xuất bộ nhớ trực tiếp (DMA), dẫn đến việc bộ xử lý trung tâm (CPU) vẫn phải can thiệp vào quá trình truyền nhận dữ liệu của ngoại vi, làm giảm hiệu năng chung của SoC.
- Xử lý miền xung clock (Clock Domain Crossing - CDC): Việc đồng bộ hóa dữ liệu giữa miền xung clock hệ thống và miền xung clock DDR ngoại vi còn đơn giản, cần các giải pháp CDC nâng cao để triệt tiêu hoàn toàn rủi ro trạng thái bất định (Metastability).
- Môi trường thực nghiệm: Đánh giá hiện dựa trên mô phỏng RTL và báo cáo Implementation của Vivado, chưa đo dạng sóng vật lý bằng Logic Analyzer hoặc Oscilloscope trong điều kiện môi trường công nghiệp.

6.4 Hướng phát triển trong tương lai

Để hoàn thiện và nâng cao tính thương mại hóa của thiết kế, các hướng nghiên cứu tiếp theo sẽ tập trung vào:

- Tích hợp khối DMA và nâng cấp Bus nội bộ: Tích hợp bộ điều khiển DMA đa kênh và chuyển đổi chuẩn kết nối bus sang các giao thức công nghiệp chuẩn như AMBA AXI4/AHB-Lite nhằm giảm tải tối đa cho CPU.
- Tối ưu hóa thiết kế vi mạch: Áp dụng các kỹ thuật thiết kế công suất thấp (Low-power design techniques) như Power Gating, Clock Gating, đồng thời tối ưu đường truyền tới các ô nguyên thủy (Primitives) DDR chuyên dụng trên phần cứng.
- Thực hiện quy trình thiết kế ASIC (ASIC Flow): Mở rộng từ FPGA sang quy trình vi mạch ASIC hoàn chỉnh, bao gồm tổng hợp logic với thư viện chuẩn (Standard Cell Library), phân tích định thời tĩnh (STA), thiết kế vật lý (Physical Design - P&R) và đánh giá công suất tiêu thụ sau đóng gói layout.
- Tích hợp bộ gia tốc phần cứng: Mở rộng SoC bằng cách thêm các bộ gia tốc mã hóa/giải mã dữ liệu hoặc bộ gia tốc tính toán AI nhẹ tại biên (TinyML) kết nối qua giao tiếp DDR.

Chương 7

CÁC CÔNG TRÌNH NGHIÊN CỨU

7.1 Bài báo đã gửi đăng

Trong quá trình thực hiện khóa luận, các kết quả nghiên cứu về tích hợp kỹ thuật truyền dữ liệu hai cạnh xung nhịp vào hệ thống SoC RISC-V đã được nhóm tác giả tổng hợp thành một bài báo khoa học và gửi đăng tại *Journal of King Saud University – Engineering Sciences* thuộc Springer Nature. Tại thời điểm hoàn thành khóa luận, bản thảo đã được nộp cho tạp chí và đang trong quy trình xử lý, phản biện; do đó, công trình chưa có thông tin chính thức về tập, số, trang và mã DOI.

1. Binh Nguyen, Chan Nguyen, Sang Ha, Huy Ngo, Ho Le Minh Toan và Thien Pham, “An implementation of Double Data Rate in System on Chip using Risc-V architecture for high speed communication applications,” bản thảo đã gửi đăng tại *Journal of King Saud University – Engineering Sciences*, Springer Nature, năm 2026. Trạng thái: đã nộp, đang chờ kết quả phản biện.

Thông tin về tạp chí được công bố tại trang: <https://link.springer.com/journal/44444>.

7.2 Mối liên hệ với khóa luận

Bài báo trình bày kiến trúc SoC sử dụng lõi RISC-V, cơ chế truyền dữ liệu DDR và khả năng ứng dụng cho các hệ thống giao tiếp tốc độ cao. Nội dung công trình được phát triển từ các kết quả thiết kế RTL, tích hợp ngoại vi, mô phỏng chức năng và phân tích định thời được trình bày trong khóa luận. Việc gửi bài báo là bước mở rộng kết quả nghiên cứu theo hướng công bố khoa học; tuy nhiên, trạng thái gửi đăng không được xem là công trình đã xuất bản cho đến khi có quyết định chấp nhận chính thức từ tạp chí.

TÀI LIỆU THAM KHẢO

- [1] D. Petrisko *et al.*, “BlackParrot: An agile open-source RISC-V multicore for accelerator SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 93–102, 2020.
- [2] W. Huang and G. Sheng, “Analysis and research on UART communication protocol,” in *Proc. 4th Asia-Pacific Conf. Communications Technology and Computer Science (ACCTCS)*, 2024, pp. 768–771.
- [3] G. B. Wakhle, I. Aggarwal, and S. Gaba, “Synthesis and implementation of UART using VHDL codes,” in *Proc. International Symposium on Computer, Consumer and Control*, 2012, pp. 1–3.
- [4] R. Xie, “Design and simulation of UART protocol based on FPGA,” in *Proc. 6th International Conf. Applied Machine Learning (ICAML)*, 2024, pp. 551–557.
- [5] Y. Liu *et al.*, “Design and verification of UART circuit of SoC based on AMBA bus,” in *Proc. 7th International Conf. Information Science and Control Engineering (ICISCE)*, 2020, pp. 2370–2374.
- [6] D. Emil, M. Hamdy, and G. Nagib, “Development an efficient AXI-interconnect unit between set of customized peripheral devices and an implemented dual-core RISC-V processor,” *The Journal of Supercomputing*, vol. 79, pp. 17000–17019, 2023.
- [7] R. Shashikumar, C. N. Vijay Kumar, M. Nagendrakumar, and C. S. Hemanthkumar, “AHB compatible DDR SDRAM controller IP core for ARM based SoC,” *International Journal of Computer Science and Information Security*, vol. 7, no. 1, pp. 77–85, 2010. Available: <https://arxiv.org/pdf/1002.1953>.
- [8] A. Ejaz, V. Papaefstathiou, and I. Sourdis, “DDRNoC: Dual data-rate

- network-on-chip,” *ACM Transactions on Architecture and Code Optimization*, vol. 15, no. 2, pp. 25:1–25:24, 2018.
- [9] A. Psarras, S. Moisisdis, C. Nicopoulos, and G. Dimitrakopoulos, “Networks-on-chip with double-data-rate links,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 64, no. 12, pp. 3103–3114, 2017.
- [10] A. Olgun *et al.*, “PiDRAM: An FPGA-based framework for end-to-end evaluation of processing-in-DRAM techniques,” in *Proc. IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2022, pp. 267–272.
- [11] YosysHQ, “PicoRV32: A Size-Optimized RISC-V CPU,” GitHub repository. Available: <https://github.com/YosysHQ/picorv32>.
- [12] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, ver. 20260120, 2026. Available: <https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html>.
- [13] Arm Limited, *AMBA APB Protocol Specification*, IHI 0024E. Available: <https://developer.arm.com/documentation/ih0024/e/>.
- [14] JEDEC Solid State Technology Association, “Main Memory: DDR SDRAM Standards.” Available: <https://www.jedec.org/category/technology-focus-area/main-memory-ddr3-ddr4-sdram>.
- [15] J. Zhou, “How to enable DDR mode,” *NXP Community*, May 27, 2021. Available: <https://community.nxp.com/t5/i-MX-RT-Crossover-MCUs-Knowledge/How-to-enable-DDR-mode/ta-p/1283331>.
- [16] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs*, UG949, ver. 2026.1, 2026. Available: <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology>.
- [17] Texas Instruments, *UART Protocol Overview and Error Sources*, TI Precision Labs. Available: https://www.ti.com/content/dam/videos/external-videos/zh-tw/9/3816841626001/6313217959112.mp4/subassets/uart_protocol_overview_and_error_sources_0.pdf.
-

- [18] S. Campbell, “Basics of UART communication,” *Circuit Basics*. Available: <https://www.circuitbasics.com/basics-uart-communication/>.
- [19] NXP Semiconductors, *I2C-bus Specification and User Manual*, UM10204, rev. 7.0, 2021. Available: https://www.nxp.com/documents/user_manual/UM10204.pdf.
- [20] Riverdi, “What is I2C protocol? I2C communication protocol,” *Riverdi Blog*. Available: <https://riverdi.com/fr/blog/what-is-i2c-protocol-i2c-communication-protocol>.
- [21] P. Dhaker, “Introduction to SPI Interface,” *Analog Dialogue*, Analog Devices. Available: <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html>.
- [22] Digilent, “What is Serial Peripheral Interface (SPI)?” *Digilent Blog*. Available: <https://digilent.com/blog/what-is-spi/>.
- [23] V. Karthikeyan, B. Krishnan, L. R. Namamula, and F. J. Brindha, “Block diagram of APB protocol,” figure from “Development of SRAM-APB protocol interface and verification,” *ResearchGate*, Oct. 2023. Available: https://www.researchgate.net/figure/Block-diagram-of-APB-Protocol_fig2_374432325.
- [24] AMD, *Vivado Design Suite Documentation*, UG1793, 2025. Available: <https://docs.amd.com/v/u/en-US/ug1793-vivado-design-suite>.
- [25] Flathub, “Xilinx Vivado Design Suite,” application page. Available: <https://flathub.org/vi/apps/com.github.corna.Vivado>.
- [26] AMD, *Zynq-7000 SoC Data Sheet: Overview*, DS190, rev. 1.11.1, 2018. Available: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>.
- [27] AMD, *Zynq-7000 SoC Technical Reference Manual*, UG585, ver. 1.15, 2026. Available: <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>.
- [28] Electromaker, “SoC module with AMD Zynq 7030-1I, 1 Gbyte DDR3L, 32

- Mbyte Flash, 4 x 5 cm—TE0715-05-71I33-A,” product page. Available: Electromaker product page.
- [29] AMD, *Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics*, DS181, rev. 1.27.1, 2024. Available: https://docs.amd.com/v/u/en-US/ds181_Artix_7_Data_Sheet.
- [30] V. Maisto, S. Mercogliano, M. Maddaluno, and A. Cilardo, “The Simply-V framework: An extensible RISC-V reconfigurable soft-SoC for open research and fast prototyping,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 31, no. 3, pp. 1–32, 2026. Available: <https://dl.acm.org/doi/10.1145/3787500>.
- [31] K.-M. Ma, D.-H. Le, C.-K. Pham, and T.-T. Hoang, “Design of an SoC based on 32-bit RISC-V processor with low-latency lightweight cryptographic cores in FPGA,” *Future Internet*, vol. 15, no. 5, Art. no. 186, 2023. Available: <https://www.mdpi.com/1999-5903/15/5/186>.
- [32] G. C. Ananthu and R. Islam, “Integrated AHB to APB bridge using Raspberry Pi and Artix-7 FPGA,” arXiv:2501.01147, 2025. Available: <https://arxiv.org/pdf/2501.01147>.
- [33] H. Hassan, N. Vijaykumar, S. Khan, S. Ghose, K. Chang, G. Pekhimenko, D. Lee, O. Ergin, and O. Mutlu, “SoftMC: Practical DRAM characterization using an FPGA-based infrastructure,” arXiv:1805.03195, 2018. Available: <https://arxiv.org/pdf/1805.03195>.
- [34] S. Wijeratne, S. Pattnaik, Z. Chen, R. Kannan, and V. Prasanna, “Programmable FPGA-based memory controller,” in *Proc. 28th IEEE Symposium on High-Performance Interconnects (HOTI)*, 2021. Available: <https://arxiv.org/pdf/2108.09601>.
- [35] A. C. Bonatto, A. B. Soares, and A. A. Susin, “Design and test of a DDR SDRAM interface for FPGA systems,” in *Proc. VIII Student Forum on Microelectronics*, 2008. Available: <https://sbmicro.org.br/sforum-eventos/sforum2008/38.pdf>.
- [36] B. Naresh, S. Rambabu, and G. Lakshmi Narayana, “Implementation of DDR

- SDRAM memory controller for embedded SOC,” *International Journal of Advanced Trends in Computer Science and Engineering*, vol. 5, no. 1, pp. 101–105, 2016, Special Issue of ICACEC 2016. Available: <https://warse.org/IJATCSE/static/pdf/Issue/icacec2016sp22.pdf>.
- [37] L. Chisciotti, *Design of a High-Speed Data Buffer Based on DDR Memories for Real-Time Processing of Hyperspectral Data and Its Implementation in a Radiation-Tolerant FPGA*, M.S. thesis, Politecnico di Torino, Turin, Italy, 2022. Available: <https://webthesis.biblio.polito.it/23437/1/tesi.pdf>.
- [38] V. Arumugham, “DDR timing,” contribution to the IEEE P802.3ae 10 Gigabit Ethernet Task Force, Nov. 30, 2000. Available: https://grouper.ieee.org/groups/802/3/ae/comments/d2.0/arumugham_1_0101.pdf.
- [39] AMD, *Vivado Design Suite User Guide: Using Constraints*, UG903, ver. 2025.1, 2025. Available: <https://docs.amd.com/r/2025.1-English/ug903-vivado-using-constraints/About-XDC-Constraints>.
- [40] AMD, *7 Series FPGAs SelectIO Resources User Guide*, UG471, ver. 1.10, 2018. Available: https://docs.amd.com/v/u/en-US/ug471_7Series_SelectIO.